

A Type-and-Effect System for Temporal Dependency Analysis of Render-based Reactive Programs

JUNE WUNDER, Boston University, USA

ANKUSH DAS, Boston University, USA

MARCO GABOARDI, Boston University, USA

Reactive programming frameworks such as React allow developers to build interactive applications by declaratively specifying how outputs depend on changing inputs. Although this model makes it easy to reason about what an application computes, the temporal behavior of reactive programs—when updates occur and how they propagate—remains difficult to understand and verify. Applications implicitly rely on timing assumptions buried in framework runtimes, leading to subtle bugs such as stale reads, transient inconsistencies, order-dependent behavior, and unintended feedback cycles. To address these challenges, this paper presents Willow, a core calculus for reactive programming inspired by React. Willow gives a time-aware operational semantics that models computation in terms of renders, the fundamental evaluation step in which components produce user interface descriptions, and pairs it with a novel type-and-effect system that statically tracks timing behavior as effects. A next modality $\circ$ expresses delays measured not only in renders but in any unit the host environment exposes—renders, network round-trips, or wall-clock milliseconds. A family of modalities tracks the full lifecycle of event handlers: when they are registered, when they fire, when pending events are canceled and when handlers are removed. A key insight is that the resulting effects form a temporal dependency graph, letting standard graph algorithms statically detect render cascades and inter-render loops that cause non-termination or performance degradation. We formalize Willow and prove preservation of the effect system with respect to the time-aware semantics. We also implement a prototype checker with automatic effect inference and evaluate it on representative reactive patterns such as debouncing, form inputs, and API-driven updates. Our results demonstrate that time-aware typing provides a practical foundation for reasoning about the temporal correctness of reactive programs.

1 Introduction

Modern interactive software is increasingly structured as reactive programs [4, 21, 42] that continuously respond to streams of events originating from users, sensors, and network services. Frameworks such as React [6, 31] have popularized a declarative model for building such systems, in which programs describe how outputs depend on changing inputs rather than explicitly orchestrating control flow. This programming style has proven highly effective for building complex interactive applications, particularly in web and mobile environments [13, 33].

Despite its widespread adoption, reasoning about the temporal behavior of reactive programs remains challenging. Reactive applications implicitly encode timing assumptions about when updates occur, how quickly state propagates, and how computations interact with asynchronous events. Timing dependencies can be intentionally or accidentally *mutually recursive*, i.e., an update to variable x leads to an update to y which, in turn, leads to an update to x . State changes or other side effects are often conditional on other pieces of program state, which makes programs more difficult to reason about. Due to these complexities, timing behaviors are embedded within framework semantics and runtime scheduling policies. This leaves timing reasoning to be done through informal means, developers lack principled tools for predicting or verifying the temporal properties of their programs.

To make matters worse, reactive programs are particularly susceptible to bugs that arise from a lack of understanding of precise timing behavior. For instance, computations may observe stale

state [7] if updates have not yet propagated, or transient inconsistencies (glitches) when dependent values are updated at different times. Programs may also exhibit order-dependence [22], where the outcome of a computation depends on the scheduler’s choice of evaluation order, or delayed-effect violations, where effects occur later than the program assumes due to deferred execution or batching. Graphical user interfaces have long been recognized as difficult to test [3, 30], owing to the combinatorial space of possible event sequences and orderings. Timing-dependent behaviors are among the hardest cases because they emerge from the interaction between the program’s dependency structure and the framework’s execution model.

A second source of temporal complexity is the *lifecycle* of event handlers themselves. Event handlers must be repeatedly torn down and re-bound as program state changes. Deciding which state changes invalidate each handler is left entirely to the programmer. Programmers can also forget to remove old event handlers, which gives rise to logic bugs and memory leaks. Events also fire at different frequencies, and programmers must guard against high-frequency firings or unintended double inputs.

This motivates the need for static analyses that make the temporal behavior of reactive computations explicit. To this end, this paper introduces Willow, a simplified core calculus inspired by React, to reason about the temporal behavior of reactive programs. At its core, Willow models time using *renders*, i.e., when a component is evaluated to produce a description of the user interface. Concretely, a render is triggered whenever one of the state variables changes. These renders are treated as the fundamental computation step.

Our first contribution is an operational semantics that makes the timing of renders and events a first-class part of the model. Previous semantics [27, 29] faithfully reproduce React’s runtime behavior, so timing is emergent from execution. We instead design a (simplified) semantics that centers timing, making the ordering and delay of renders and events explicit in the semantic objects themselves. Our semantics models execution in two phases: the render phase where code runs, and the housekeeping phase where state updates and events are managed. Variables are considered immutable during the render phase and are modified during housekeeping between renders. A novel *instrumented* semantics tracks variable dependencies and distinguishes between event-triggered evaluation steps and the propagation of updates through reactive dependencies. This formulation makes the timing behavior of reactive programs explicit and provides a foundation for static reasoning.

Building on this semantics, we introduce a novel type-and-effect system that tracks timing information. Effects capture when states *may* change; when events are bound, removed, or cancelled; and the timing of scheduled events. The effect of a computation intuitively describes when its observable consequences may occur relative to the event that triggered it.

The key idea of our type system is to integrate state and event dependencies with quantitative delay information. For instance, if an update to x causes an update to y after 1 render, this effect on y is expressed as $\circ^{1r}@x$ where $@x$ denotes an update to x . Similarly, if a text input box has a `onChange` handler that updates state z , we convey that as $\Box^{\text{change}(\text{input})}(\circ^{1r}@z)$. Scheduled events are their own effect, simply a label with some specifying data in a tuple. The modality $\Box^e(F)$ marks the registration of a *persistent* listener, and its counterpart $\Diamond^e(F)$ marks a *one-time* listener. We also add modalities $\times e$ and $\oslash e$ to mark removing all listeners from an event, and cancelling one firing of an event e . Finally, these effects can be composed in sequence with $F_1 * F_2$ and branches are marked by $F_1 + F_2$. Effects then propagate through expressions in a manner analogous to traditional effect systems. This approach enables modular reasoning about reactive programs, allowing the timing properties of larger components to be inferred from those of their parts.

A key advantage of Willow is that captured effects can be viewed as a *temporal dependency graph* that captures variable dependency over the entire execution, not just a single render. This enables standard graph algorithms to analyze performance and dataflow of programs. We use these algorithms to (i) detect long render cascades, where one event drives many sequential renders and degrades performance, (ii) flag inter-render loops, where updates repeatedly trigger one another, (iii) warn when expensive handlers are bound to high-frequency events, (iv) confirm that stale event handlers are always cleaned up, and (v) analyze the timing of an app’s first render. As a result, Willow statically detects critical performance defects missed by compiler transformations [32] and other current tools.

To evaluate the practicality of our approach, we formalize Willow and the proposed effect system and prove key metatheoretic properties. In particular, we establish type preservation, showing that well-typed programs respect the timing guarantees described by their effects. To minimize programmer burden, we have also implemented a prototype *type-and-effect inference algorithm* in Haskell that infers effects of all examples presented in the paper.

In summary, this paper makes the following contributions:

- A core calculus for reactive programming featuring a time-aware operational semantics making explicit the temporal and causal structure of updates (Section 4),
- A type-and-effect system for tracking timing constraints, enabling static reasoning about when computations produce observable effects (Section 5),
- Metatheoretic results establishing preservation of the effect system with respect to an instrumented semantics (Section 6),
- Post-typecheck analyses that can statically catch common bugs and warn programmers what to inspect for fixing timing logic errors (Section 5.4),
- A prototype type-and-effect inference algorithm to demonstrate feasibility for implementation in real React programs (Section 7),
- The type-checking of real-world examples, namely a stuck loading state, a request race condition, and an update loop in a realistic signup form, demonstrating how the system statically finds and draws attention to timing bugs (Section 7).

2 Overview

Willow’s core calculus is inspired by React’s programming model and aims at capturing the essential structure of render-based reactive programs. Before describing Willow in detail, we provide a brief background on React.

React [31] provides a declarative model for building user interfaces (UIs) for websites and applications. In this model, programmers describe how the UI should be derived from some application state, and the framework automatically updates the interface whenever that state changes.

React programs are structured around *components*: functions that take inputs and return a ‘piece’ of UI to render. Program execution is then divided into a sequence of *renders*. During each render, a component computes a snapshot of the UI based on the current values of its state variables and inputs. React then compares the resulting interface with the previous one and applies the minimal set of updates needed to the Document Object Model (DOM), i.e., visible page. This render cycle occurs repeatedly as the program responds to events, giving the user the impression of a continuously interactive system. Renders typically happen at least 60 times per second, meaning each render can only take up to 16 ms to execute, compute and apply the minimal updates.

React also provides several *hooks*, which are primitive mechanisms for managing state and side effects. Two of the most fundamental are `useState` and `useEffect` hooks. The `useState` hook allows components to store values that persist across renders, while `useEffect` allows code to execute in

response to changes in specific pieces of state. The difference between a component and a hook in React is that a component returns render-able HTML whereas a hook is primarily used for encapsulation of the program logic to make programs more readable.

Setters in React. The `useState` hook is typically used in declarations of the form:

```
1  const [x, setX] = useState(e);
```

Here, `x` denotes a state variable and `setX` is a *setter* function that schedules updates to the state variable. State is *immutable during a render*: updates are not applied immediately but instead scheduled to occur between renders. The expression `e` gives an initial value to `x` the first time the component is rendered. Setter functions are usually given a function of type $\tau \rightarrow \tau$ that computes a new value from a value in the previous state. Thus, setters allow the type signature $(\tau \rightarrow \tau) \rightarrow \text{unit}$. When a setter receives a function argument, e.g., `setX(x => x + 1)`, the function is evaluated between renders using the most recent state value. Multiple setter calls during a single render are queued and applied sequentially before the next render begins. For instance, if a variable `x` has value 3 at the start of a render and `setX(x => x + 1)` was invoked twice during that render, then `x` would remain 3 throughout the render, but next render would observe the value 5 for `x`.

Effects in React. React is purposefully designed to resemble functional programming. Stateful variables are immutable during renders because each render is intended to be a snapshot in time of the overall execution. Side effects, such as state updates, are therefore typically performed in response to events or within *effect hooks*. This is where `useEffect` comes in handy.

```
1  useEffect(() => {
2    ...side effects...
3    return () => {...cleanup...}
4  }, [x, y, z...])
```

The first argument is an effectful function, while the second argument specifies an array of *dependencies*. Whenever the value of one of these dependencies changes between renders, React schedules the effect function to execute. The effect function may optionally return a cleanup function that runs before the effect is re-executed and cleans up the previous render's effect. This enables programs to react to state updates and cause side effects in response.

An example: Moving Dot. The react model based on components and hooks provides flexibility but reasoning about renders with side effects is difficult. Effects depend on state changes that occur in previous renders, and multiple pieces of state may interact indirectly through chains of effects. As applications grow larger, understanding the temporal behavior of a program becomes increasingly challenging. To illustrate these issues, consider the following (simplified) program, written using `useEffect` to display a dot that follows the location of the user's click, together with a checkbox controlling whether movement is enabled.

```
1  export default function MovingDot() {
2    const [position, setPosition] = useState({ x: 0, y: 0 });
3    const [canMove, setCanMove] = useState(true);
4    let handleClick = (e) => {
5      setPosition(_ => ({ x: e.clientX, y: e.clientY }));
6      useEffect(() => {
7        if (canMove) { document.addEventListener("click", handleClick); }
8        return () => document.removeEventListener("click", handleClick);
9      }, [canMove]);
10   return ( <div>
```

```

11      <input type="checkbox" checked={canMove}
12          onChange={ (e) => setCanMove(_ => e.target.checked)} />
13      <div style={transform: `${position.x}, ${position.y}`} ... />
14    </div> ); }

```

Lines 2–3: State Declarations. The component declares two state variables along with their setters: `position` holds the dot’s current coordinates, initialized to $\{x:0, y:0\}$, and `canMove` is a boolean flag, initialized to `true`, that controls whether a click should be allowed to move the dot.

Lines 4–6: Click Handler. The `handleClick` function that takes a DOM event `e`, calls `setPosition` in the updater form: the argument `_ => {x: e.clientX, y: e.clientY}` ignores the previous position and returns the click coordinates. Because this is a setter call, the update is *not* applied immediately; it is queued and will take effect at the start of the next render.

Lines 7–10: Effect Block. `useEffect` declares a side-effecting block with dependency list `[canMove]`. React runs this block after any render in which `canMove` has changed (and once on the initial mount). Each time the block runs, it first executes the cleanup function `removeEventListener` returned by the *previous* invocation, if any, to remove the record of the old `handleClick` before setting up a new one. The body then conditionally re-records `handleClick`: if `canMove` is `true`, `addEventListener` attaches `handleClick` to the document’s click event; if `canMove` is `false`, no new listener is attached.

Lines 11–15: Rendered Output. The `<input type="checkbox">` renders the toggle; its `onChange` handler calls `setCanMove` with the checkbox’s new `checked` value, scheduling a `canMove` update for the next render, which will in turn trigger a re-run of the effect block. The positioned `<div>` renders the dot itself, placed at the coordinates stored in `position`.

The correctness of this example relies on several temporal assumptions:

- i. When the user clicks the page, the registered listener calls `setPosition`, but `position` only updates on the *next* render, not during the event.
- ii. When `canMove` flips, the effect block runs first the cleanup function to remove the old listener; only after that the body decides whether to attach a new one. Forgetting the cleanup would correspond to a program logic mismatch and a potential memory leak.
- iii. The points in time when a click can move the dot are not determined by the code itself; it is determined by when the effect block last ran and what `canMove`’s value was at that point.

This kind of effect block programming is a common idiom in modern React, and, unfortunately, also a common source of bugs. To use it correctly, a programmer must track which renders re-run the effect, which of those runs trigger the cleanup, and in what order the resulting side effects occur relative to one another and to the surrounding renders. This is infeasible in large applications.

Introducing Willow. To address the challenges discussed above, Willow introduces a novel type-and-effect system that tracks how state variables may change over renders. We encode the timing semantics of Willow in this effect system, so the above list of assumptions can be known at compile time. The key idea is to represent potential state updates explicitly as effects describing *when* (i.e., after how many renders) an update may occur. For a variable `x`, the base effect `@x` indicates that `x` *may change*, and `○1r@x` indicates that `x` may have a different value in exactly the next render. We write `·` for the *empty effect*, indicating that no state update may occur.

As usual in effect systems, function types carry an effect. We use the notation $\tau_1 \rightarrow \tau_2 \mid F$ to describe a function whose argument type is τ_1 , return type is τ_2 , and whose effect is F . Function effects are only triggered when a function is *called*; until then they are encapsulated in the type. Setters are primitive to Willow and we can assign them a type based on their timing behavior: $(\tau \rightarrow \tau \mid \cdot) \rightarrow \text{unit} \mid \text{○}^{1r}@x$. Setters have `○1r@x` as their effect to describe semantically that setters

are evaluated between renders, and in the render after a setter is called its variable may have been altered by the closure given to the setter. Also note that the inner function of type $\tau \rightarrow \tau$ has the empty effect $\cdot$, since the setter of x may only update x .

Multiple effects can be composed using the sequencing $*$ operator. Concretely, $F_1 * F_2$ denotes that *both* effects F_1 and F_2 occur in sequence. For instance, consider the expression: `setX(f); setY(f)`. The effect for this program would be written as $\circ^{1r}@x * \circ^{1r}@y$. Intuitively, this can be read as: in the next render, x may have changed *and also* in the next render, y may have changed.

Effects can be composed with the $+$ operator to handle branches: $F_1 + F_2$ indicates that either F_1 or F_2 may happen. For instance, consider `if b then setX(f) else setY(f)`. The effect for this program is written as $\circ^{1r}@x + \circ^{1r}@y$, since either x or y may change in the next render.

The constructs above describe *synchronous* state changes: setter calls that are queued during a render and flushed deterministically before the next one begins, requiring no external trigger. *Asynchronous* events, like a DOM click, a timer expiry, or a network response may arrive at any point in time, interleaved between renders, or not at all. Their occurrence is contingent on the outside world, not on the program's own execution. To describe these, the effect language provides a second layer of modalities indexed by *event labels* rather than by render counts. The two layers meet at effect of the form $\Box^e(\circ^{1r}@x)$: contingent on an external event e firing, x may change in the render that follows.

To see how Willow's type-and-effect system can help in practice let us revisit the MovingDot example. In Willow we can write it as follows.

```

1 comp MovingDot () : html {
2   state position, setPosition default (0, 0);
3   state canMove, setCanMove default true;
4   let handleClick =  $\lambda e$ . setPosition( $\lambda\_.$  (e.clientX, e.clientY));
5   on canMove do {
6     remove click<#doc>;
7     if canMove then (bind click<#doc> handleClick) else () };
8   return ( ... /* same html as in React */ ); }
```

The Willow implementation is similar to the one in React we presented before with a few differences. State declarations specify the default initial value explicitly, and we use λ -expressions in place of `(e) => ...` closures. The `useEffect` block of React is replaced by an *on-block* of the form `on canMove do { ... }` which fires whenever its watched variable `canMove` changes. An *on-block* in Willow executes *cleanup at the beginning*, if needed. In our example `remove` is called first, and only after that the rest of the body is executed. Notice that the conditional registration is now the body of the *on-block*, not a check inside `handleClick`.

The argument `click<#doc>` is an *event label*: a tagged identifier for a class of external events. The label has the form $\ell(\bar{v})$, where ℓ names the kind of event (`click`, `timeout`, `req`, ...) and $\bar{v}$ is a tuple of statically-known values identifying *which* event of that kind (here, the document node `#doc`; for a request, a URL). Only statically-known values may appear in the tuple: dynamic values such as a freshly-generated timer id cannot enter the effect layer, so events like `timeout<>` carry an empty tuple.

The type-and-effect system reveals both how user interactions influence program state and how the lifecycle of the click subscription itself is managed. The relevant effects that Willow infers for

the MovingDot example are the following:

```

bind click<#doc> handleClick :  $\square^{\text{click}\langle\#doc\rangle}(\circ^{1r}\text{@position})$ 
  remove click<#doc> :  $\mathbf{X}\text{click}\langle\#doc\rangle$ 
  on canMove block body :  $\mathbf{X}\text{click}\langle\#doc\rangle * (\square^{\text{click}\langle\#doc\rangle}(\circ^{1r}\text{@position}) + (\cdot))$ 
  onChange :  $\circ^{1r}\text{@canMove}$ 

```

The construct `bind` registers a *persistent* handler against an event label. Every time the event fires, the handler runs. We describe this with the graded square modality $\square^{\text{click}\langle\#doc\rangle}(F_h)$, read “every time `click<#doc>` fires, F_h happens.” Here F_h is the effect of the handler body. In our example, `handleClick` calls `setPosition`, so $F_h = \circ^{1r}\text{@position}$. The construct `remove` unregisters all handlers for the named event. We describe this with the effect $\mathbf{X}\text{click}\langle\#doc\rangle$, read “the registration for `click<#doc>` no longer fires its handler.” Willow’s type-and-effect system encodes directly the timing assumptions we mentioned earlier from React.

- i. We see in the effect of click listener that it modifies position on the next render because $\circ^{1r}\text{@position}$ signifies that “in the next render position may change.”
- ii. When `canMove` changes we see that $\mathbf{X}\text{click}\langle\#doc\rangle$ happens before the new event handler is registered, so we know that stale event handlers are cleaned up.
- iii. We see in the effect of the on-block that the event handler is only sometimes registered $(\square^{\text{click}\langle\#doc\rangle}(\circ^{1r}\text{@position}) + (\cdot))$, which indicates that our handler is conditional.

By exposing this structure, Willow helps programmers reason about how reactive programs evolve.

A second example: Debounce. To illustrate one-time listeners in Willow, we consider a Debounce component, which takes a quickly-changing input value and exposes a slow value that updates only when value has not changed for 100ms. A debounce protects expensive downstream work from executing too frequently. This can be implemented in Willow as follows:

```

1 comp Debounce (value: any) : any {
2   state slow, setSlow default value;
3   on value do {
4     clearTimeout();
5     setTimeout( $\lambda\_.$  setSlow( $\lambda\_.$  value))
6   };
7   return slow;
8 }

```

We first initialize a slow stateful variable, which will store our slowed down value. We bind an on-block to the fast *value*. In this block, we create a timeout (running for 100ms) to update the slow state, but first we cancel our previous timeout.

The builtins `setTimeout` and `clearTimeout` have types:

```

setTimeout f :  $\diamond^{\text{timeout}\langle\rangle}(F_f) * \circ^{100\text{ms}}\text{timeout}\langle\rangle$ 
clearTimeout() :  $\oslash\text{timeout}\langle\rangle * \mathbf{X}\text{timeout}\langle\rangle$ 

```

Unlike `bind`, `setTimeout` registers a *one-shot* handler. When `timeout<>` fires the callback runs once and the registration is consumed. We describe this behavior with the graded diamond $\diamond^{\text{timeout}\langle\rangle}(F)$, read “contingent on `timeout<>` firing, F happens once.” Here F is the effect of the callback body: `setSlow` produces $\circ^{1r}\text{@slow}$. The accompanying $\circ^{100\text{ms}}$ records that the event `timeout<>` itself fires after 100ms of wall-clock time. Willow’s $\circ$ modality is indexed not only by

renders r , but by any declared time unit — renders, milliseconds, network round-trips — so the system can describe asynchronous schedules alongside synchronous render counts.

For timeouts, a `timeout()` event is scheduled and a handler is registered. To clean up both, `clearTimeout` produces cancel and remove effects $\circ \text{timeout}() * \times \text{timeout}()$. The cancel effect will prevent the `timeout()` event from firing, and the remove effect clears all current event listeners to the event. Putting it together, the cascading effect of value in `Debounce` is

$$\circ \text{timeout}() * \times \text{timeout}() * \diamond \text{timeout}() (\circ^{1r} @ \text{slow}) * \circ^{100\text{ms}} \text{timeout}()$$

which reads: when value changes (1) any pending `timeout()` is cancelled and its handlers are unregistered (2) a fresh `timeout()` is scheduled to fire after 100ms (3) contingent on it firing, in the next render `slow` may change.

The payoff of the debounce example is that the protection survives composition. Suppose a parent component watches `slow` from a `Debounce` child and issues a network request:

```
comp slowInput = Debounce(input);
on slowInput do { fetch("/api/search?q=" + slowInput, ...) };
```

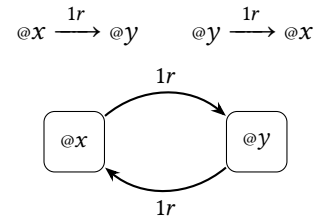
Then the effect of input carries the prefix $\diamond^{\text{timeout}()}(\circ^{1r}(\circ^{1n}F))$, so the network call F is visibly behind a debounce timer. If input is changed by a frequent event (e.g. an `onChange` on an `<input/>`), a parent or library author can see in the type that the downstream fetch is protected; if a debounce were missing, the type would be missing the leading $\diamond$ and Willow’s post-hoc analyses could warn the programmer.

Temporal Dependency Graphs. React’s `useEffect` is intentionally an “escape hatch” from the declarative paradigm (see David Khourshid’s “Goodbye `useEffect`” [25]). For most of React we rely on predictable functional reactive programming, and then carve out a section of our program where that predictability no longer holds. Synchronizing with external systems can have unexpected behavior, effects may clash with sibling effects, and cleanup happens some unknowable amount of time in the future.

Willow’s type-and-effect system can be used to explore the *temporal dependency graph*. In this graph, nodes represent potential state changes, and edges model delays between two nodes. This way, the graph visualizes effects over *multiple renders*, even though our effects only capture dependencies over a single render. By recursively expanding these effects over the graph, we can compute the *full effect* of any event.

Temporal dependency graphs can be used to automatically detect inter-render loops, analyze the performance of event handlers and an app’s initial render, and ensure stale event handlers are not left behind. Each of these analyses can be performed via off-the-shelf graph walking algorithms. The simplest example that the effect system catches is a state-change loop, showcased in the following example and the corresponding looping graph.

```
1 comp MutualRecursion (clock: int) : unit {
2   state x, setX default 0;
3   state y, setY default 0;
4   on x do { setY addOne };
5   on y do { setX addOne };
6   return ();
7 }
```



Components	C	$::=$	$\text{comp } A(x_1 : \tau_1, x_2 : \tau_2, \dots, x_n : \tau_n) : \tau \{ p_1 \ p_2 \ \dots \ p_n \ \text{return } x \}$
Declarations	p	$::=$	$\text{let } x = e; \mid \text{state } x, \text{ setX default } e;$ $\text{on } x_1, x_2, \dots, x_n \text{ do } \{ e \}; \mid \text{comp } x = A(x_1, x_2, \dots, x_n);$
Expressions	e	$::=$	$x \mid c \in \{\text{true}, \text{false}, ()\} \mid e_1 \ e_2 \mid e_1; e_2 \mid \lambda x. e$ $\text{if } x \text{ then } e_1 \text{ else } e_2 \mid (e_1, e_2) \mid \text{fst } e \mid \text{snd } e$ $\text{bind } \ell(\bar{v}) \ e \mid \text{once } \ell(\bar{v}) \ e \mid \text{cancel } \ell(\bar{v}) \mid \text{remove } \ell(\bar{v})$

Fig. 1. Syntax of Willow Programs

The effect of changing x is $\circ^{1r} @ y$; and vice versa, the effect for changing y is $\circ^{1r} @ x$. From the graph, we can infer that the full effect of modifying x is an infinite chain of modifications of x and y . Willow can auto-detect this kind of error and warn the programmer at compile time.

3 Formal Syntax

Willow’s syntax is inspired by React with a few simplifications. Like React, Willow programs are composed of components. The last declared component is considered as the main toplevel component of the program. Term C denotes a component named A that takes a sequence of arguments (similar to a function), and contains a series of declarations, terminated by return of a variable.

Declarations p have four possibilities: a simple let declaration $\text{let } x = e$; binds variable x to the value of e . A state declaration binds a state variable x and its setter setX , and is required to include a default initial value in the form of an expression. Effect blocks take a list of variables that are monitored such that when any of them change value compared to the previous render, the effect block’s expression e is executed. Finally, we also support *subcomponents* that enable a programmer to use any of the previously declared components. The declaration $\text{comp } x = A(x_1, x_2, \dots, x_n)$; executes component A on arguments $x_1, \dots, x_n$ and binds the returned value to x .

Expressions in Willow appear in let, state, and effect block declarations. Most of the expressions are standard, namely variables, constants, function applications, and lambda-expressions. If-expressions are written as $\text{if } x \text{ then } e_1 \text{ else } e_2$ which branches on the value of x . We support pairs of the form (e_1, e_2) which can be projected out using fst and snd respectively. Finally, Willow provides primitives for interacting with the event layer. The expressions $\text{bind } \ell(\bar{v}) \ e$ and $\text{once } \ell(\bar{v}) \ e$ register the closure e as a handler for the event $\ell(\bar{v})$: bind installs a persistent listener that runs on every firing, while once installs a one-shot listener that runs at most once. The expressions $\text{cancel } \ell(\bar{v})$ and $\text{remove } \ell(\bar{v})$ tear down event work: cancel suppresses a single pending firing of $\ell(\bar{v})$, while remove unregisters every listener attached to $\ell(\bar{v})$.

4 Semantics

We have created a formal render-based semantics for Willow inspired by React and many other frontend JavaScript libraries/frameworks. Render-based semantics are generally used in the context of frontend user interface coding, so we discuss our semantics in terms relevant to this paradigm.

Render-based semantics work by running a program’s code every time a piece of program state changes. Whatever the program returns is considered the output for that render. In frontend web development the output of a program is HTML. In many frameworks there is a virtual copy of the HTML visible to the user called the “Virtual DOM.” The returned HTML is diffed with this Virtual DOM to find the exact changes that need to be made to the visible HTML, and these changes are made in one atomic step. To keep Willow environment-agnostic we don’t formalize what happens

with the output of a Willow program. We assume that there is a higher runtime that Willow exists within and it handles the returned outputs on each render.

Real-world render-based frameworks make heavy use of caching and memoization in order to improve performance. We ignore these optimizations because pieces of software such as React Compiler [32] have substantially reduced the need to introduce these optimizations manually.

The declarative render-based programming style revolves around the existence of an event queue. For example in the JavaScript runtime, to initiate a network request a programmer calls `fetch` with a URL and request information and attaches a closure via `.then` to run when the response arrives. The in-flight request is handled outside JavaScript by the host environment, and when it completes the closure is queued to run on the next turn of the event loop. The resulting execution order, with microtasks interleaved against the task queue and React’s own priority-lane scheduler on top, can be as difficult to predict as understanding any given JavaScript runtime. We simplify these render-based semantics with a more concrete execution order.

In our formal semantics, Willow’s internal event queue holds closures associated with setter calls; event interactions introduced by the event primitives `bind`, `once`, `cancel`, `remove`; and events being fired $\ell(\bar{v})$. Willow’s queue is therefore concerned only with the events its primitives introduce. Events from the external environment move into Willow’s event queue via the “external scheduler” which is a queue of events that Willow flushes between renders.

There are two phases to Willow program execution: (1) The render phase, in which programmer-written code executes and the event queue is built, (2) The housekeeping phase, in which the scheduler and event queue are flushed and program state is updated. These two phases are looped over and over as new inputs arrive from the external environment.

4.1 Semantics Derivations

In all the semantic derivations there is a program signature Σ that does not change throughout program execution. Σ is a mapping from component names to a tuple of the full component’s code, its effect environment Δ , and type environment Γ which we discuss in § 5.

$$A : (\text{comp } A \ (\bar{x} : \bar{\tau}) : \tau_r \{ p \}, \Delta, \Gamma)$$

Component judgment. $\Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C \ (\bar{x} = \bar{v}) : \tau \{ p \} \text{ s } U$ takes one step of component evaluation. Here $\mathcal{L}$: listener map from event labels to registered listeners tagged $\square$ persistent or $\diamond$ one-shot, V_e : snapshot of the value environment from the end of the previous render, V_s : state-variable environment seeded from V_e and mutated by setter calls during housekeeping, $\mathcal{C}$: cancellation multiset with one entry per pending `cancel` suppressing the next dispatch of the named event, s : the execution status either rendered if the event queue is not finished processing or waiting if housekeeping is over, and U : Willow’s event queue of setter closures and event-primitive operations awaiting housekeeping.

Declaration judgment. $\Sigma ; [V_e \mid V_s] ; V ; c \vdash p \Downarrow v ; U ; V'$ evaluates declarations p at component path c under working environment V , threading V_e and V_s through unchanged. It produces the returned value v , the update queue U accumulated as a side effect, and the final value environment V' witnessed at return. Here we introduce V : working environment of variable values in scope during the current render, c : dotted component path used as the key for setter dispatch, V' : value environment produced after all declarations in p are evaluated.

Expression judgment. $\Sigma ; V \vdash e \Downarrow v ; U$ evaluates expression e under working environment V to value v , producing update queue U as a side effect.

Effect vs. state environments. The pair V_e, V_s carries values between renders. During the render phase they are read-only and during the housekeeping phase they are modified. V_s is read to evaluate the state declaration, the state’s value is taken from V_s and bound with its name in V . Willow

writes to V_s while flushing setters from the event queue during housekeeping. V_e is a frozen snapshot of the value environment taken just before housekeeping, when choosing whether to execute an effect block the bound variables are checked for differences between V_e and V . V_s begins each housekeeping phase equal to V_e and is mutated as setter calls are popped from the update queue.

Component paths. Setter evaluation happens between renders, so each setter must be paired with the state variable it targets. However, because Willow allows components to be used multiple times as subcomponents, we need to be able to address each piece of state with a globally unique name. The component path c provides this unique identifier. We write $c.x$ for the path obtained by appending x to c , with “.” as separator and no leading dot. At the top level the path is empty and $c.x = x$, while inside a subcomponent bound to variable y in the parent component the path is $c'.y$ where c' is the parent’s path and $c.x = c'.y.x$. The path attached to each setter is the key for that state variable in V_s and V_e at the top level, so setter dispatch is exact-key lookup.

4.2 Component

The component-level rules drive (1) the housekeeping phase in which the event queue is flushed, and (2) the transition between renders in which fresh arguments arrive and a new render is executed. We show the most interesting rules in this section collected in Figure 2 and leave the rest to the appendix. Rules at this level only deal with executing the main component of the program. We evaluate subcomponents at the declaration level of the semantics. We make this choice so that all variable names are relative to the top main component in this rules level, which makes handling state variables more straightforward.

We start with showing how setters are evaluated between renders. When a setter is called at the expression level, it does not immediately change the value of its corresponding state variable but instead is entered into the event queue to be evaluated during housekeeping. In `FLUSH FIRST UPDATE` we pop off a setter call event from the queue and execute its closure.

Before evaluating its closure we first remove all setters from the environment, because setters are not allowed to have any effects. The value returned from the closure is used as the new value for the y state, which is achieved by updating y in V_s . If $v \neq v'$ then y ’s value will differ between V_s and V_e , so for the next render y will trigger the effects it is bound to.

The `FLUSH EXTERNAL EVENTS` rule shows how event-effects are flushed from the external scheduler and enter Willow’s event queue. We take $\mathcal{E}$ as a given object Willow has no control over. $\mathcal{E}$ is allowed to fill with labelled events such as mouse clicks, network request successes or failures, etc, and Willow handles these once a render at the start of housekeeping. Here $U_{\mathcal{E}}$ is a series of pairs $(\ell\langle\overline{v_1}\rangle, v_2)$ of labelled events with the values they carry. An example could be $(\text{click}[\text{doc}], (100, 200))$ for a document click at position $x=100$ $y=200$.

The `POP LISTEN` rule shows how event listeners are added. The `bind` and `once` expressions do not instantly bind a closure to an event, but instead add a `listen( $\ell\langle\overline{v_2}\rangle, c, m$ )` event to the queue. The elements of the listen event’s tuple are $\ell\langle\overline{v_2}\rangle$ the event to be listened to, c the closure to execute on fire, and m the modality either $\square$ or $\diamond$ depending on if the listener was created with `bind` or `once`. The closure and modality are then added to the ordered set of event handlers

The `POP EVENT` rule shows how an event handler is executed in response to an event firing $(\ell\langle\overline{v_1}\rangle, v_2)$. First, we require that the event has not been cancelled so we enforce that the cancellation multiset does not contain any instances of $\ell\langle\overline{v_1}\rangle$. Next we retrieve all listeners of the event $(\langle\lambda x_1.e_1, V_1\rangle, m_1)$ from the listener mapping $\mathcal{L}$. The closures are all executed and given v_2 as their argument, these are allowed to have any side effect. We enforce that the queues are “listener safe” which means that they are not allowed to output a new labelled event $\ell'\langle\overline{v'}\rangle$ in their event queue.

$$\begin{array}{c}
\text{(FLUSH FIRST UPDATE)} \\
\frac{\Sigma ; \mathcal{L} ; V_e ; V_s, y = v ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ rendered } \text{setter}_y((\lambda y.e, V)), U' \quad V' = \text{removeSetters}(V) \quad \Sigma ; V', y = v \vdash e \Downarrow v' ; \cdot}{\Sigma ; \mathcal{L} ; V_e ; V_s, y = v' ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ rendered } U'} \\
\\
\text{(FLUSH EXTERNAL EVENTS)} \\
\frac{\Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ rendered } U \quad \mathcal{E} \Rightarrow_{\text{ext}} U_{\mathcal{E}}}{\Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ rendered } U_{\mathcal{E}}, U} \\
\\
\text{(POP LISTEN)} \\
\frac{\Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v_1}) : \tau \{ p \} \text{ rendered } \text{listen}(\ell(\overline{v_2}), c, m), U' \quad m \in \{\Diamond, \Box\} \quad \mathcal{L}' = \mathcal{L}[\ell(\overline{v_2}) \mapsto \mathcal{L}(\ell(\overline{v_2})) \cup \{(c, m)\}]}{\Sigma ; \mathcal{L}' ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v_1}) : \tau \{ p \} \text{ rendered } U'} \\
\\
\text{(POP EVENT)} \\
\frac{\Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ rendered } (\ell(\overline{v_1}), v_2), U' \quad \mathcal{C}(\ell(\overline{v_1})) = 0 \quad \mathcal{L}(\ell(\overline{v_1})) = \{(\lambda x_1.e_1, V_1), m_1\}, \dots, (\lambda x_k.e_k, V_k), m_k\}}{\Sigma ; V_i, x_i = v_2 \vdash e_i \Downarrow () ; U_i \quad \vdash U_i \text{ lsafe} \quad \mathcal{L}' = \mathcal{L}[\ell(\overline{v_1}) \mapsto \{(\lambda x_i.e_i, V_i), m_i, F_i\} \mid m_i = \Box\}]} \\
\Sigma ; \mathcal{L}' ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ rendered } U', U_1, \dots, U_k \\
\\
\text{(WAITING TO RENDERED)} \\
\frac{\Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ waiting} \cdot \quad \Sigma ; [V_e \mid V_s] ; x = v' ; "" \vdash p \Downarrow v_r ; U' ; V' \quad \exists i \text{ s.t. } v_i \neq v'_i}{\Sigma ; \mathcal{L} ; V' ; V' ; \mathcal{C} \vdash \text{comp } C(\overline{x = v'}) : \tau \{ p \} \text{ rendered } U'}
\end{array}$$

Fig. 2. Selected component-level semantics rules

The event queues produced by executing the listeners are appended to the program event queue to be evaluated. Finally, only listeners marked with the $\Box$ modality are kept in the listener mapping.

Once the event queue is fully flushed the state transitions from rendered to waiting where Willow waits for new top-level arguments. Now we review where a render occurs. In WAITING TO RENDERED, Willow receives new top-level arguments from the external program. To write this in the semantics we require that the previous render had arguments v and the next render will have arguments v' where at least one argument differs. Next we can evaluate the main component's declarations with the declaration evaluation judgement, receive a new event queue to be processed, and start housekeeping over again. This rule is where we see the value environment from the end of the previous render V' become the initial value for V_e and V_s .

4.3 Declarations

Declarations evaluate under the working environment V . At this layer we read from the effect and state environments V_e and V_s and accumulate an update queue as a side effect. We review re-render behavior for space purposes because the first render behavior is straightforward; the rules appear in Figure 3.

In STATE RERENDER, a re-rendered state variable declaration's default expression is ignored. The default expression is used on the first render to bootstrap an initial value, but now we use the value in V_s that has been modified by setter calls during housekeeping. To V we add $x = v_0$

$$\begin{array}{c}
\text{(STATE RERENDER)} \\
\frac{\Sigma ; [V_e \mid V_s] ; V, x = v_0, \text{setX} = \text{setter}_{c.x} ; c \vdash p \Downarrow v ; U ; V' \quad x = v_0 \in V_s, x = v_e \in V_e}{\Sigma ; [V_e \mid V_s] ; V ; c \vdash \text{state } x, \text{setX default } e ; p \Downarrow v ; U ; V'} \\
\\
\text{(EFFECT RERENDER, NO CHANGES)} \\
\frac{\Sigma ; [V_e \mid V_s] ; V ; c \vdash p \Downarrow v ; U ; V' \quad \overline{x = v} \in V_e \quad \overline{x = v'} \in V \quad \forall i, v_i = v'_i}{\Sigma ; [V_e \mid V_s] ; V ; c \vdash \text{on } x_1, x_2, \dots, x_n \text{ do } \{e\} ; p \Downarrow v ; U ; V'} \\
\\
\text{(EFFECT RERENDER, YES CHANGES)} \\
\frac{\begin{array}{c} \Sigma ; [V_e \mid V_s] ; V ; c \vdash p \Downarrow v ; U' ; V' \\ \Sigma ; V \vdash e \Downarrow v_e ; U \quad \overline{x = v} \in V_e \quad \overline{x = v'} \in V \quad \exists i, v_i \neq v'_i \end{array}}{\Sigma ; [V_e \mid V_s] ; V ; c \vdash \text{on } x_1, x_2, \dots, x_n \text{ do } \{e\} ; p \Downarrow v ; U, U' ; V'} \\
\\
\text{(SUBCOMP)} \\
\frac{\begin{array}{c} \Sigma ; [V'_e \mid V'_s] ; \overline{x' = v} ; c.y \vdash p_A \Downarrow v_A ; U_y ; V'_y \\ \Sigma ; [V_e \mid V_s] ; V, y = v_A ; c \vdash p \Downarrow v ; U ; V' \\ V'_e = \text{removePrefix}("y.", \text{startsWith}("y.", V_e)) \quad V'_s = \text{removePrefix}("y.", \text{startsWith}("y.", V_s)) \\ V''_y = \text{addPrefix}("y.", V'_y) \quad \overline{x = v} \in V \quad A : (\text{comp } A(\overline{x'}) : \tau \{p_A\}, \Delta, \Gamma) \in \Sigma \end{array}}{\Sigma ; [V_e \mid V_s] ; V ; c \vdash \text{comp } y = A(\overline{x}) ; p \Downarrow v ; U_y, U ; V' \cup V''_y}
\end{array}$$

Fig. 3. Re-render declaration-level semantics rules.

from V_s , and a setter value with label $c.x$. Setters appear to act like normal functions in the type system, but in the semantics they are a dummy value that holds the global path to its state. This is the bridge between renders: setters dispatched during the previous housekeeping have already updated V_s , so the next render sees their effects without re-running initialization.

Next we see how effect blocks are executed. This is the only declaration that's allowed to perform side-effectful functions in Willow. Here we check for differing values of the watched variables $\overline{x}$ between V_e and V . If there are no differences then we are in the EFFECT RERENDER, NO CHANGES case and we skip the declaration. If at least one variable has changed then we are in EFFECT RERENDER, YES CHANGES and move to evaluate the effect block's body $\Sigma ; V \vdash e \Downarrow v_e ; U$. This expression can have a side effect, which we add to the event queue to be processed in housekeeping, the return value of this expression is discarded.

The subcomponent accepts the relevant subsets of V_e and V_s from its parent. The subcomponent returns a value, an update queue, and its final value environment, which need to be prefixed with the parent's path. Reading SUBCOMP top to bottom:

- (1) Retrieve the subcomponent's code p_A from Σ .
- (2) Calculate V'_e and V'_s by restricting to entries whose keys begin with "y." and strip the prefix.
- (3) Evaluate p_A under V'_e, V'_s , and a working environment of the subcomponent's arguments.
- (4) Re-prefix the final value environment V''_y with "y.", yielding V''_y .
- (5) Bind y to the subcomponent's return v_A , then evaluate the remaining parent declarations.
- (6) Place the subcomponent's queue U_y before the parent's queue U and union V''_y into V' so that the subcomponent's state is stored for the next render.

$$\begin{array}{c}
\text{(SEQ)} \\
\frac{\Sigma ; V \vdash e_1 \Downarrow v_1 ; U_1 \quad \Sigma ; V \vdash e_2 \Downarrow v_2 ; U_2}{\Sigma ; V \vdash e_1 ; e_2 \Downarrow v_2 ; U_1, U_2} \\
\text{(FUNCTION APP)} \\
\frac{\Sigma ; V \vdash e_1 \Downarrow \langle \lambda x. e_3, V' \rangle ; U_1 \quad \Sigma ; V \vdash e_2 \Downarrow v_2 ; U_2 \quad \Sigma ; V', x : v_2 \vdash e_3 \Downarrow v_3 ; U_3}{\Sigma ; V \vdash e_1 e_2 \Downarrow v_3 ; U_1, U_2, U_3} \\
\text{(SETTER APP)} \\
\frac{\Sigma ; V \vdash e_1 \Downarrow \text{setter}_x ; U_1 \quad \Sigma ; V \vdash e_2 \Downarrow \langle \lambda y. e_3, V' \rangle ; U_2}{\Sigma ; V \vdash e_1 e_2 \Downarrow () ; U_1, U_2, \text{setter}_x(\langle \lambda y. e_3, V' \rangle)} \\
\text{(BIND)} \\
\frac{\Sigma ; V \vdash e \Downarrow \langle \lambda x. e', V' \rangle ; U}{\Sigma ; V \vdash \text{bind } \ell(\bar{v}) e \Downarrow () ; U, \text{listen}(\ell(\bar{v}), \langle \lambda x. e', V' \rangle, \square)} \\
\text{(CANCEL)} \quad \text{(REMOVE)} \\
\frac{\Sigma ; V \vdash \text{cancel } \ell(\bar{v}) \Downarrow () ; \text{cancel}(\ell(\bar{v})) \quad \Sigma ; V \vdash \text{remove } \ell(\bar{v}) \Downarrow () ; \text{remove}(\ell(\bar{v}))}{\Sigma ; V \vdash \text{cancel } \ell(\bar{v}) \Downarrow () ; \text{cancel}(\ell(\bar{v})) \quad \Sigma ; V \vdash \text{remove } \ell(\bar{v}) \Downarrow () ; \text{remove}(\ell(\bar{v}))}
\end{array}$$

Fig. 4. Selected expression-level semantics rules.

4.4 Expression

Most of the expression semantics of Willow are unchanged from a normal expression semantics of any other language. Willow expressions create an update queue as a side effect which is ordered in execution order the same as the update queue for declarations. Take for example the `SEQ` rule in Figure 4.

The first and second expressions are evaluated, they return values, and produce update queues U_1 and U_2 as a side effect. In this particular rule we discard v_1 , return v_2 , and as a side effect produce the queue U_1, U_2 because the events in U_1 happened first.

The `FUNCTION APP` rule is normal with added event queue handling. When a function application is a normal closure value then the closure is evaluated immediately and its effect is placed in the event queue. In the `SETTER APP` rule we see how the setter dummy value is used. This rule requires its argument to be a closure value, and places the closure onto the event queue rather than executing it right now. As we saw earlier this closure will be used to modify the value of x between renders.

Finally we see the `BIND`, `CANCEL`, and `REMOVE` rules; `ONCE` is exactly like `bind` but outputs an $\diamond$ label instead of an $\square$. These each have their own corresponding event that they place on the queue. We reviewed the `listen` event earlier, and review the `cancel` and `remove` events in the appendix. `bind` and `once` both evaluate their argument expression to a closure and emit a `listen` event tagged with their corresponding modality. `cancel` and `remove` take no expression argument and unconditionally emit their corresponding queue item.

5 Type-and-Effect System

Willow's type-and-effect system answers *when*-questions about a render-based program: when can a state change cascade into another, when are event-handlers registered and removed, when does a feedback loop appear, and which effects fire on the first render.

Willow has two base effects, the state change effect $@x$ and the event effect $\ell(\bar{v})$. The state change effect signifies that a state's setter function is called so its variable x may change. An event effect

Types	τ	$::=$	$\text{unit} \mid \text{bool} \mid \tau_1 \times \tau_2 \mid (\tau_1 \rightarrow \tau_2 \mid F)$
Effects	F	$::=$	$\cdot \mid @x \mid \ell\langle\bar{v}\rangle \mid \circ^{Nu} F \mid F_1 * F_2 \mid F_1 + F_2$ $\diamond^{\ell\langle\bar{v}\rangle}(F) \mid \square^{\ell\langle\bar{v}\rangle}(F) \mid \oslash \ell\langle\bar{v}\rangle \mid \times \ell\langle\bar{v}\rangle$
Event Labels	$\ell\langle\bar{v}\rangle$	$::=$	$\ell\langle\bar{v}\rangle$ where $\bar{v}$ is a tuple of base values
Time Unit	u	$::=$	$r \mid n \mid \text{ms} \mid \dots$

Fig. 5. Grammar of Willow Types and Effects

corresponds to an event in the environment Willow is embedded in. On the web this can be a keypress, a mouse movement, a network request.

The primary vehicle of timing analysis is the temporal next modality $\circ^N$ whose superscript N counts time units, letting effects describe not only *what* may happen but *when*. Units include renders (r), network requests (n), wall-clock milliseconds (ms), and any unit the host environment cares to introduce. The syntax for effects is based upon the next operator in temporal logic [36].

The event handler modalities $\diamond^{\ell\langle\bar{v}\rangle}$ “eventually” and $\square^{\ell\langle\bar{v}\rangle}$ “always” describe when an event handler will be registered. The superscripts given to *eventually* and *always* are the name of the event being bound to: a click, a key press, a promise’s resolution or error. The modalities $\oslash \ell\langle\bar{v}\rangle$ “cancel” and $\times \ell\langle\bar{v}\rangle$ “remove” balance with these to ensure no stale event handlers remain after each render. Willow’s type-and-effect system creates a unified abstraction over both event handlers and pending events via these modalities.

Effects move between two positions in Willow. The arrow type in Willow ($\tau_1 \rightarrow \tau_2 \mid F$) is extended by a latent effect annotation F . This annotation is the effect that may occur when the arrow’s closure is executed. The typing of an expression $\Gamma \vdash e : \tau \mid F$ reads “under Γ , expression e has type τ and produces effect F .” We say that when an expression is wrapped in a closure as a function body then the latent effect of the expression is captured into the type system, and when a function is called then its effect is released.

Sequential effects are denoted with $*$ and branches in programs are denoted with $+$. The LHS and RHS of a sequencing operator denote which effect will be put on Willow’s event queue first (in $F_1 * F_2$, F_1 happens first). This means that in general effects are not commutative across $*$.

Effects are introduced to the Willow type-and-effect system through type annotations on built-in functions. Most syntax constructs combine the effects of their children using the sequencing and branching constructors. The built-ins that introduce effects are setters, network calls, and the event-layer primitives `bind`, `once`, `cancel`, and `remove`.

$$\begin{array}{c}
 \Gamma \vdash e_1 : (\tau_1 \rightarrow \tau_2 \mid F) \mid F_1 \quad \Gamma \vdash e_2 : \tau_1 \mid F_2 \\
 \text{(APP)} \quad \frac{}{\Gamma \vdash e_1 e_2 : \tau_2 \mid F_1 * F_2 * F} \quad \text{(FN)} \quad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2 \mid F}{\Gamma \vdash \lambda x. e : (\tau_1 \rightarrow \tau_2 \mid F) \mid \cdot}
 \end{array}$$

Both APP and FN are the typical function introduction and application rules, except extended with our effect system. The application’s effect combines three pieces via the sequencing combinator $*$: F_1 from evaluating e_1 , F_2 from evaluating e_2 , and the closure’s latent effect F . In the function introduction rule FN we see how the effect of the function body moves from the latent position in its typing judgement to the latent position in the arrow type. A closure expression has no effect on its own so the closure expression has the empty effect.

Time units appear as a superscript on the next modality $\circ$ and only identical units collapse, so $\circ^{1r}(\circ^{1r}F)$ can be written as $\circ^{2r}F$. We deliberately keep units in Willow incompatible: a network request is more taxing than one extra render, and some milliseconds of delay matter for perceived responsiveness but have no performance implication. Nesting modalities sequences in time:

$\circ^{1r}(\circ^{1n}F)$ kicks off a network request during the next render and F happens after it returns. Sequencing and branching place each child effect relative to a shared “zero time”: in $(\circ^{1r}F_1 * \circ^{1n}F_2)$, F_1 runs whenever the next render occurs, and F_2 runs once a network request resolves. $(\circ^{1r}F_1 * \circ^{1r}F_2)$ can be simplified to $\circ^{1r}(F_1 * F_2)$. New time units are introduced by built-in functions. In a real-world scenario programmers could type-alias their own units to make library APIs read better.

Below we give the type for a possible function `asyncCompute`, a function that takes two closures: one to run on success and one on failure. The function returns unit immediately and after one unit of computation time, will either succeed or fail.

```
asyncCompute : ∀F1, F2. (int → unit | F1) -> (int → unit | F2) -> unit
              |  $\circ^{1u}(\text{comp}[\text{suc}] + \text{comp}[\text{err}])$ 
              *  $\diamond_{\text{comp}[\text{suc}]}(F1 * \text{Xcomp}[\text{err}]) * \diamond_{\text{comp}[\text{err}]}(F2 * \text{Xcomp}[\text{suc}])$ 
```

The effect of `asyncCompute` shows our unknown unit of computation as “1u” in the superscript of a next modality. After one unit of time we can see that either the `comp[suc]` event or the `comp[err]` event is promised to fire. At the time of function call, two single-shot event handler registrations are put onto the event queue. They are bound to the `comp[suc]` success or `comp[err]` error events. On success the `comp[suc]` event will fire so the first argument will be called, its effect $F1$ will happen, and all handlers for `comp[err]` will be removed. The reverse goes for the `comp[err]` event. Through analysis of Willow’s effect for `asyncCompute` we can see (1) when each event will occur, (2) every event is handled, and (3) all handlers are cleaned up.

Immediate, cascading, and full effects. Willow distinguishes three kinds of effect. The *immediate* effect of a function is its own latent side effect.

A variable’s *cascading* effect describes what watching effect blocks will do when that variable changes. For example, on x do `{ setY addOne }`; contributes $\circ^{1r}@y$ to the cascading effect of x . When x changes the block fires (during the next render), the setter enqueues a closure, and one render later y may change. Cascading effects live in Δ alongside the variable’s dependencies, written $x[\text{deps}] \vdash \text{cascade}$. State variables have no dependencies; let-bound variables do. To determine a variable’s dependencies we parameterize Willow over a given dataflow function df . df takes an expression as input, and returns a set of variable names that may alter the value of the expression when they change. The simplest instantiation of df is just the set of free variables.

The *full* effect of a variable or function collects every effect that may be triggered, transitively, by a single mutation or event firing. It is recovered post-typecheck by a graph expansion over Δ that tracks visited nodes to mark loops. For example, given $x[] \vdash \circ^{1r}@z$, $y[z] \vdash \cdot$, $z[] \vdash \circ^{1r}@x$ — “ x triggers a state change in z one render later, y depends on z so it is recomputed whenever z changes, and z may mutate x after one render” — the full effect of x changing is $\circ^{1r}(@z * @y * \circ^{1r}\text{loop}[x])$, where `loop[x]` marks the point at which the expansion revisits x . Loops are surfaced rather than swept away. Under a reactive paradigm a cycle in the cascading graph is usually a bug. Preservation (the metatheory section) makes the full effect honest, any well-typed program that steps continues to type with an effect bounded by the original under subeffecting, so the full effect computed at type-check time is a genuine over-approximation of what runs.

Subeffecting. The relation $F \leq F'$ used in the rules above is Willow’s subeffecting relation. F' is the more informative side, it makes more commitments about what may happen than F does. Under this reading $+$ acts as a meet: a branch over-approximation can be replaced by either side, so $F_1 + F_2 \leq F_i$. Dually, $*$ acts as a join, with $F \leq F_1 * F_2$ whenever F refines either side. The next modality $\circ^N$ is monotone and obeys $\circ^0 F = F$ and $\circ^{N_1} \circ^{N_2} = \circ^{N_1 + N_2}$, the graded-monad laws. Subeffecting on $@x$, $\ell(\bar{v})$, and the event-guarded constructs is similarly familiar: the empty effect sits below each, and the body of $\square$ and $\diamond$ is monotone. The full rules, including the dual side for $@x$ and the four event-guarded leaves, are in Appendix.

$$\begin{array}{c}
\text{(TY STATE DECL)} \\
\hline
\Gamma \vdash e : \tau \mid \cdot \quad x[] \mid F \in \Delta \quad \Gamma' = \Gamma, x : \tau, \text{setX} : ((\tau \rightarrow \tau \mid \cdot) \rightarrow \text{unit} \mid \circ^{1r} @x) \\
\hline
\Sigma ; \Gamma ; \Delta \vdash \text{state } x, \text{setX default } e; \Rightarrow \Gamma' \\
\text{(TY LET DECL)} \quad \text{(TY ON DECL)} \\
\hline
\Gamma \vdash e : \tau \mid \cdot \quad \bar{y} = df(e) \quad x[\bar{y}] \mid F \in \Delta \quad \Gamma \vdash e : \text{unit} \mid F \quad \forall i \in [n], \Delta \vdash @x_i \Rightarrow F \\
\hline
\Sigma ; \Gamma ; \Delta \vdash \text{let } x = e; \Rightarrow \Gamma, x : \tau \quad \Sigma ; \Gamma ; \Delta \vdash \text{on } x_1, x_2, \dots, x_n \text{ do } \{e\}; \Rightarrow \Gamma \\
\text{(TY SUBCOMP DECL)} \\
\hline
\Delta \vdash @x_i \Rightarrow @y.z_i \quad \Delta \vdash @y.\text{return} \Rightarrow @y \quad A : (\text{comp } A(\overline{z_i : \tau_i}) : \tau_r \{p\}, \Delta_A, \Gamma_A) \in \Sigma \\
\hline
\overline{x_i : \tau_i} \in \Gamma \quad \overline{v_\Delta} = fv(\Delta_A) \quad \Delta_A[y.v_{\Delta_i}/v_{\Delta_i}] \subseteq \Delta \quad \overline{v_\Gamma} = fv(\Gamma_A) \\
\hline
\Sigma ; \Gamma ; \Delta \vdash \text{comp } y = A(x_1, x_2, \dots, x_n); \Rightarrow \Gamma, \Gamma_A[y.v_{\Gamma_i}/v_{\Gamma_i}], y : \tau_r
\end{array}$$

Fig. 6. Declaration-level typing rules.

5.1 Declaration-Level Typing Rules

The declaration typing derivation is $\Sigma ; \Delta ; \Gamma_1 \vdash p \Rightarrow \Gamma_2$, where Σ is the program signature, which maps each component name to its code, effect environment, and typing environment. Δ is the current component's effect environment; it maps variables to their dependencies and cascading effect $x[\text{deps}] \mid \text{cascade}$. We treat Δ as given, the typing rules check that effects in the program are subeffects of the effects given in Δ . The difficulty in finding a Δ candidate for a program is finding a minimal cascading effect. Typing rules enforce that each variable's cascading entry in Δ is consistent using the “causation” judgement $\Delta \vdash @x \Rightarrow F$. Our prototype inference/typechecker recovers Δ programmatically (§7).

$$\begin{array}{cc}
\text{(X CAUSES F)} \quad \frac{x[\dots] \mid F' \in \Delta \quad F \leq F'}{\Delta \vdash @x \Rightarrow F} & \text{(X CAUSES Y)} \quad \frac{y[\dots, x, \dots] \mid F' \in \Delta}{\Delta \vdash @x \Rightarrow @y}
\end{array}$$

$\Delta \vdash @x \Rightarrow F$ reads “a change in x is a sufficient cause for the effect F , given the cascading entries in Δ .” The first rule is the base case: x causes F directly when Δ records a cascade F' for x that is at least as informative as F under subeffecting. The second is the dependency case: x causes $@y$ when y 's dependencies include x , because if x changes y is recomputed, which is itself a state-change-like event.

TY STATE DECL (Figure 6) requires the default expression e to have no effect. State variables have no dependencies, so their entry in Δ looks like $x[] \mid F$. The cascade F records what effect blocks watching x will do when x changes. TY LET DECL similarly looks up x 's entry, but with dependencies $\bar{y}$ computed by df . A let-bound variable is assumed to change whenever any dependency does.

TY ON DECL types the body e with some effect F , then checks that each watched variable x_i “causes” F — i.e., F is accounted for by x_i 's cascading entry in Δ . This is the static side of Willow's preservation story: the cascade promised in Δ at type-check accounts for the effect block's body.

TY SUBCOMP DECL appears to do a lot but is mostly managing α -renaming of variables. This rule retrieves the subcomponent's entry from Σ , then enforces variable aliases. We enforce a timing guarantee that the supplied arguments to A are dependencies that change A 's parameters in the same render. We also do the same for A 's return value back to the variable bound to the result of A . This means that arguments and return values are passed as normal with no event-queue or timing interaction. We enforce that A 's effect environment Δ_A is a sub-environment of the current Δ (with some renaming). A 's typing environment Γ_A is not required of the caller; instead its renamed copy

$$\begin{array}{c}
\text{(SEQ)} \quad \frac{\Gamma \vdash e_1 : \tau_1 \mid F_1 \quad \Gamma \vdash e_2 : \tau_2 \mid F_2}{\Gamma \vdash e_1; e_2 : \tau_2 \mid F_1 * F_2} \quad \text{(BRANCH)} \quad \frac{\Gamma \vdash e_1 : \text{bool} \mid F_1 \quad \Gamma \vdash e_2 : \tau \mid F_2 \quad \Gamma \vdash e_3 : \tau \mid F_3}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau \mid F_1 * (F_2 + F_3)} \\
\text{(TY BIND)} \quad \frac{\Gamma \vdash e : (\tau \rightarrow \text{unit} \mid F) \mid F_e \quad \tau = \Sigma_E(\ell(\bar{v}))}{\Gamma \vdash \text{bind } \ell(\bar{v}) \ e : \text{unit} \mid F_e * \Box^{\ell(\bar{v})}(F)} \quad \text{(TY ONCE)} \quad \frac{\Gamma \vdash e : (\tau \rightarrow \text{unit} \mid F) \mid F_e \quad \tau = \Sigma_E(\ell(\bar{v}))}{\Gamma \vdash \text{once } \ell(\bar{v}) \ e : \text{unit} \mid F_e * \Diamond^{\ell(\bar{v})}(F)} \\
\text{(TY CANCEL)} \quad \frac{\Gamma \vdash \text{bind } \ell(\bar{v}) \ e : \text{unit} \mid F_e * \Box^{\ell(\bar{v})}(F)}{\Gamma \vdash \text{cancel } \ell(\bar{v}) : \text{unit} \mid \oslash \ell(\bar{v})} \quad \text{(TY REMOVE)} \quad \frac{\Gamma \vdash \text{once } \ell(\bar{v}) \ e : \text{unit} \mid F_e * \Diamond^{\ell(\bar{v})}(F)}{\Gamma \vdash \text{remove } \ell(\bar{v}) : \text{unit} \mid \cancel{\ell(\bar{v})}}
\end{array}$$

Fig. 7. Selected expression-level typing rules, including the four event primitives.

is added to the output so the subcomponent's internal bindings become accessible as $y.v$ in the parent. All declaration-level variables in subcomponents are assumed to be unique, so we can use $y.$ as a unique prefix for the variables of this subcomponent without clashing with another instance of A in this scope.

5.2 Expression-Level Typing Rules

Earlier we reviewed the function application rule, we now apply three other important expression typing rules, shown in Figure 7.

The return type of e_1 is discarded and the return type of e_2 is returned. The effects of e_1 and e_2 are sequenced with $*$. **BRANCH** sequences the effect of the condition with a $+$ joining the effects of the branches. Exactly one of F_2 or F_3 may happen.

Shown in Figure 7, the **bind** and **once** rules require their argument to be a closure whose latent effect F becomes the body of the event-guarded modality, and whose parameter type matches the payload type $\Sigma_E(\ell(\bar{v}))$ carried by the event. The mode tag ($\Box$ versus $\Diamond$) matches the listener tag the semantics installs in the listener map $\mathcal{L}$. The **cancel** and **remove** rules have no premise: the event label is provided syntactically, and the semantics treats both operations as unconditional, so the effect annotation describes an *attempt* at teardown that may be a no-op at runtime.

5.3 Effect Polymorphism

Willow's prototype type-and-effect inference algorithm implements effect polymorphism through a basic Hindley-Milner style type inference algorithm used for effects. For Willow this means that both functions and components are allowed to parameterize effect variables that can be filled in at function call or at subcomponent declaration. Willow's formal type-and-effect system does not implement this feature for simplicity of the type system. Because Willow does not feature any form of recursion we do not have to handle effect-polymorphic recursion, so this makes our implementation easier. It is appropriate to use an effect in Willow when a function or component takes a function with arbitrary effect as a parameter.

```

both : ∀F. (int → unit | F) → (int × int) → unit | F * F
comp TextInput<F> (init: string, onChange: string → unit | F) : html {...}

```

Here we see a utility function **both** that applies its first argument to both elements of an integer pair. The effect produced is $F * F$ because the function parameter is called twice. The second small example is the type of a utility component **TextInput**, which one can imagine is given a typical React implementation of a text input element. An initial value for the input is supplied, and when

the input is changed by the user, `onChange` would be called. When *TextInput* is used as a subcomponent F can be inferred as a concrete effect, and Willow propagates the effect of the function given for `onChange`. Due to the way the subcomponent rule ensures that subcomponent effect environments (Δ_A) are contained within their parent environments (Δ) this does not cause any namespace clashes. In examples, we freely use effect polymorphism to prevent code duplication and improve readability.

5.4 Post-Typecheck Analysis

Once type-and-effect inference finishes, effects in both Δ and HTML can be viewed as graph nodes for standard graph analysis.

As mentioned earlier, Willow can automatically do loop detection, analyze the performance of event handlers, ensure stale event handlers are not left behind, and analyze the performance of an app's initial render. Each of these algorithms starts with identifying key expressions in a program and calculating the full effect of those expressions.

To turn an effect into a directional graph we view each effect as its own node in the graph, and use the effect environment as the list of edges in the graph. We can view the dependency array as a list of incoming edges, and view the cascading effect as the outgoing edges. The timing and event annotations on `next`, `eventually`, and `always` modalities can be viewed as weights in this graph. Earlier we outlined how to calculate the full effect of changing a variable, but this process can be used on the effect of any expression. The full effect is recovered by a simple graph expansion: if a node n is a descendant of its own expansion we place a `loop[n]` node and do not expand again. This is how we earlier recovered the full effect $\circ^{1r}(@z * @y * \circ^{1r}\text{loop}[x])$ of x .

Effect Summary We can make the full effect of a variable more readable to a programmer by hiding state change effects corresponding to variables in child subcomponents' scopes. Calculate the full effect of an expression, hide all variables that start with a "y." prefix, collapse all `next` modalities' superscripts so that $\circ^{1r}\circ^{1r}F$ becomes $\circ^{2r}F$.

Loop detection A common bug in declarative reactive programs is inter-render loops caused by two or more effect blocks triggering each other's effects each time they run. To the uninitiated this may sound as simple as not having "on x do setY ...; on y do setX ..." however this problem goes deeper. The reason effect blocks exist is to synchronize with external state, so effect blocks are where declarative reactive programs interface with programs of a different paradigm. When effect blocks only synchronize by either retrieving or pushing updates but not both then there is no problem. Bugs can arise when two way synchronization occurs between the client and server-side.

Take a case as simple as keeping user data up to date on the client side. A user's data may be updated by the current application or externally in a separate instance of the same app. This means a common pattern is to have a web socket listening for user profile updates so the app can update proactively to reflect changes. This socket should only update the client-side data though; if receiving data on the client side causes an update to be pushed to server-side then the client no longer controls whether it will receive that push back to itself, and a loop could form. With Willow we can give external APIs temporal effects so Willow's loop detection can catch these cases.

Event Checking Analysis Every type of event fires with a different distribution over time. Keyboard presses happen in short bursts, mouse movements are frequent and continual, network requests only resolve a single time. Knowing these distributions, we can warn programmers of performance intensive event handlers placed on key events.

Event handlers cleaned up Given the full effect of a variable, walk the effect's graph and ensure that all event handlers are matched by a corresponding `remove` modality. This includes walking both branches of the $+$ constructor separately and looking inside of the $\square$ and $\diamond$ modalities. Using

this simple graph walk we can revisit the `asyncCompute` example. We see, just with Willow’s type-and-effect system, that no event handlers are left after the function executes.

First Render Analysis First render is the part of program behavior users feel. Initial paint dominates perceived performance, Google’s Core Web Vitals now codify Largest Contentful Paint as a first-class web performance metric [19].

A component’s first render is the case where every variable counts as “changed,” so every effect block fires. This means that all program logic is evaluated at the time few pieces of state have initial values yet. In the initial render the application has not had time to fetch up-to-date data from the server, and has not even been able to pull data from local cache. Permission to play sound/video or use the microphone/camera is not established yet, whether previously given or not. This means that the initial render starts more asynchronous computations than any other time in an app’s lifecycle while in a completely unpopulated state. For programmers this can be a difficult mental shift, as usually programs are read in the context of steady-state execution. Willow’s type system can give programmers a static understanding of the first paint’s timing separate from the steady-state behavior.

6 Metatheory

We present a proof of preservation for Willow. Our preservation proof’s main guarantee can be summarized as follows: “when a variable changes, a sub-effect of its cascade given in Δ will appear in the next housekeeping phase.” To prove this we rely upon an instrumented semantics, which is extended with helpful side conditions and employs the use of a trace of update queues rather than a single update queue. The trace’s main function is to couple event queues with the effect that caused them. Given Δ has an entry $x[y] \vdash F$, then preservation tells us that when x changes we should see that y had changed. If there is an effect block bound to x then we should see it produce an update queue that fulfills some subeffect of F .

As a brief aside, the main extension to the language we require is that `bind` and `once` expressions are annotated with their listener’s effect. This information is already available in the typing of a program so we just also require it to be in the instrumented semantics. We use this to extend the listener mapping to be from an event label key to a tuple of (c, m, F) closure c , modality m , and F the effect of c as determined by the type system. This helps greatly when ensuring that event handlers have the effect they are typed to have.

Returning to traces: the records that make up traces are

- $ARG \bar{x} = \bar{v}_1 \rightarrow \bar{v}_2$ records which of the main component’s arguments changed this render.
- $LET(x) \leftarrow \{\bar{y}\}$ records a `let` binding and which variables it was affected by.
- $EFF(\bar{x}) \rightarrow \{U\}$ records that a list of variables $\bar{x}$ changed between this render and last render and triggered the event queue U to be added to the global event queue.
- $RET x$ records that the variable x was returned by a component.
- $EXT(\ell_1\langle\bar{v}_{11}\rangle, v_{21}), \dots, (\ell_n\langle\bar{v}_{1n}\rangle, v_{2n})$ marks the receipt of a batch of external event firings from the scheduler $\mathcal{E}$ and delimits a block of per-event processing records that follow.
- $FIRE(\ell\langle\bar{v}_1\rangle, v_2)$ records an event firing is waiting to be dispatched.
- $FIRE-CXL(\ell\langle\bar{v}_1\rangle, v_2)$ records that a firing was suppressed by the cancellation multiset $\mathcal{C}$.
- $FIRE-SUC(\ell\langle\bar{v}_1\rangle, v_2) \rightarrow \{(U_1, F_1), \dots, (U_k, F_k)\}$ records that a firing was successfully dispatched: for each registered listener i , the listener body produced update queue U_i , and the record pairs U_i with the effect F_i stored in the listener map $\mathcal{L}$ at registration time. Recording F_i in the trace is what allows the post-hoc Full Effect analysis to walk listener-registered effects through the operational trace.

The instrumented component semantic judgement is a well-typed configuration. It contains typing environments Σ, Δ, Γ_s . Σ and Δ have already been explained, Γ_s is the entire typing environment of the main component that is returned from type checking. $\mathcal{S}$ is a state tuple containing $(X_1, X_2, V_s, \mathcal{L}, \mathcal{C})$. X_1 and X_2 are the sets of variables that were modified during the housekeeping phases before the previous render and the most recent render respectively. $\mathcal{L}$ is extended in the way we discussed earlier. V_s and $\mathcal{C}$ are unchanged from the regular semantics. A is the name of the main component. *status* is either *waiting* or *rendered*. T_1 is the already processed part of the trace produced by the most recent render. T_2 is the trace that is yet to be processed.

$$\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x=v} \mid \text{status} \mid T_1 \mid T_2 \rangle$$

We now give the theorem statement for Preservation of Configurations and then describe relevant judgements directly after. The theorem formally states that given a well-typed program with main component A , at each step of the housekeeping phase (1) the effect environment and set of states/arguments modified in the previous housekeeping phase explain the trace produced by the most recent render (2) the already-processed section of the trace explains each variable that is marked as having been changed for next render.

Theorem 6.1 (Preservation for Configurations). *Let $\mathcal{K} = \langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x=v} \mid \text{status}_1 \mid T_1 \mid T_2 \rangle$ with $\mathcal{S} = (X_1, X_2, V_{s1}, \mathcal{L}, \mathcal{C})$, and suppose $\mathcal{K}$ is well-typed, i.e.*

$$\begin{aligned} \Sigma \Rightarrow \text{comp } A \Rightarrow (\Delta, \Gamma_s) \quad & (\Delta, X_1) \succ T_1, T_2 \quad (\Delta, X_1) \succ T_1 \succ X_2 \\ \Sigma; \Gamma_s \models_t V_{s1} \quad & \overline{\Sigma \models v : \tau} \quad \text{for every } (c, m, F) \in \mathcal{L}(\ell(\overline{v_1})) : \Sigma \models c : \Sigma_E(\ell(\overline{v_1})) \rightarrow \text{unit} \mid F \\ & \text{for every firing } (\ell(\overline{v_1}), v_2) \text{ in } \mathcal{E} \text{ or any FIRE record of } T_2 : \Sigma \models v_2 : \Sigma_E(\ell(\overline{v_1})). \end{aligned}$$

If $\mathcal{K} \rightarrow \mathcal{K}'$ with $\mathcal{K}' = \langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S}' \mid A \mid \overline{x=v'} \mid \text{status}_2 \mid T_3 \mid T_4 \rangle$ and $\mathcal{S}' = (X_3, X_4, V_{s2}, \mathcal{L}', \mathcal{C}')$, then the analogous well-typedness conditions hold of $\mathcal{K}'$.

Component typing $\Sigma \Rightarrow \text{comp } A \Rightarrow (\Delta, \Gamma_s)$ states that under the program signature Σ , the component A is typeable under effect environment Δ and typing environment Γ_s .

Trace justification $(\Delta, X) \succ T_1, T_2$ states that for each entry in the trace we can point to a variable in X and an effect in Δ that justify it. Δ contains effects that the type system expects to happen after variables change and X is the list of variables that changed, so this judgement requires that T_1, T_2 reflects the type-and-effect system's allowed effects.

Trace-driven variable-change set $(\Delta, X) \succ T_1 \succ X'$ This judgement states that not only should Δ, X justify T_1 but all three together should be able to justify each variable that is marked as having changed for the next render.

Value well-typedness $\Sigma; \Gamma \models_t V$ signifies that all variable-value mappings in V are well-typed under the variable-type mappings in Γ . $\Sigma \models v : \tau$ states that value v is well-typed under τ .

Listener-map well-formedness This requirement on the listener map tells us the closures stored in the map are well-typed under the arrow type $\Sigma_E(\ell(\overline{v})) \rightarrow \text{unit} \mid F$. The input to the arrow must be of the type that is carried by the event the closure is bound to. So for a closure bound to the event $\ell(\overline{v})$ the value carried by that event firing will have type $\Sigma_E(\ell(\overline{v}))$.

Payload typing for in-flight firings Here we require that the values associated with in-flight events are of the correct type $\Sigma_E(\ell(\overline{v}))$.

We do not constrain the scheduler or give guarantees about scheduled events being on time. This is by the nature of the scheduler being external, we must assume the scheduler is well-behaved. There is no constant unit conversion from milliseconds to renders or any other units, so any well-timed property on the scheduler would follow by tautology.

7 Implementation and Evaluation

We implement a prototype type-and-effect inference algorithm for Willow written in Haskell, and accompany it with the suite of analysis algorithms (Section 5.4) to explore how Willow could give programmers more information about their programs before runtime. Our inference algorithm requires minimal type annotations, only needing function and component arguments to be type-and-effect annotated. The effects of all programs in the paper are automatically inferred by the Willow inference checker.

To demonstrate how Willow’s type-and-effect system can help find timing bugs before runtime, we show a focused example of the username input on a signup form for a website. Our example will show a text input field where the user will write their desired username. The username will then be sent to the server over the network and the client will receive a boolean response of whether the username is available or taken. By dynamically checking the availability we make our form feel more responsive than if we required the user to validate all inputs at once when attempting to submit the form.

```

1 comp UsernameInput () : (bool, html) {
2   state username,      setUsername      default "";
3   state availableNames, setAvailableNames default new Set();
4   state status, setStatus default "idle";
5   comp slowUsername = Debounce(username);
6
7   let handleUsernameSuc = λ(name, isAvailable).
8     if isAvailable then setAvailableNames(λs. s.add(name)) else ()
9   let handleUsernameErr = λ_. setStatus(λ_. "error");
10  on slowUsername do {
11    if availableNames.has(slowUsername) then
12      setStatus(λ_. "idle")
13    else (
14      setStatus(λ_. "checking");
15      fetchUsernameCheck(slowUsername, handleUsernameSuc, handleUsernameErr));
16  return (availableNames.has(username), /* html */)

```

In lines 2-4 we declare our state variables. We need to track the user’s in-progress username, so we store that in the `username` state string. `availableNames` is a set we use to hold username candidates marked as available by the server. `status` will store the network request status of our username checks. On line 5 we declare a debounced version of `username`, this will be helpful to slow down the number of network requests to our username availability API endpoint. Internally the debounce schedules a timeout to fire its slow update; we refer to that timeout event by the label `db()` in the effects below.

On lines 7 and 8 we declare a handler for when a username check request succeeds. We receive a tuple of the checked name and a bool of whether it is available or not. If available we add the name to the `availableNames` set, and if not we do nothing. There are two failure states for a username check: first the network request itself could fail and second the name could be unavailable. We handled the second case already but we handle the network failure on line 9 by setting the status to “error”.

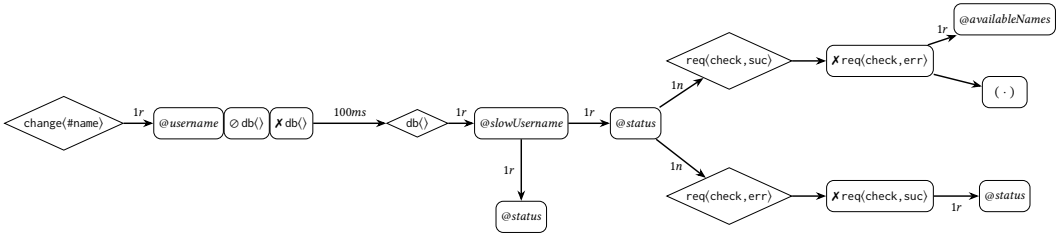
Whenever the user stops typing in the text input field for the username, the debounce will resolve, `slowUsername` will update to its new value, and the on-block on line 10 will execute. If we already have checked this username and it’s available then we can set the network status to idle (lines 11-12). Otherwise we set our status to “checking” on line 14 and initiate a new API call

to check availability on line 15. Here, `fetchUsernameCheck` has a similar type to `asyncCompute` from §5, except its events are `req⟨check, suc⟩` and `req⟨check, err⟩`.

Finally we return (1) a bool of whether the currently typed username is available and (2) the html of the username input which shows username availability, network request status, and an input.

```
<input id="name" value={username} onChange={λe. setUsername(λ_. e.target.value)} />
```

Now we can see what Willow tells us about this program.



Manual inspection of the full effect of our text input changing shows us a bug in our program. At every point in the network request we update `status` except for when the request is successful. Immediately when `slowUsername` is changed, both branches of our on-block update the status. Then after the network request the error case updates the status, but the success case does not. This means that on success the loading spinner would confusingly continue to spin while telling the user that their username is available and they don't have anything to wait for. This can be fixed by updating the `status` in the `handleUsernameSuc` closure.

```
λ(name, isAvailable). (setStatus(λ_. "idle"); if isAvailable...)
```

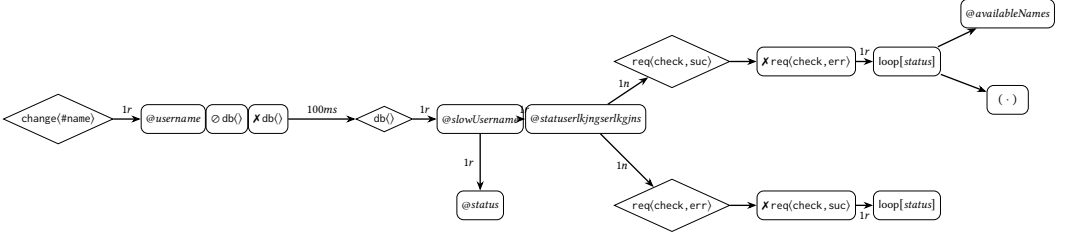
Looking at the types also gives the realization that we're allowing multiple network requests to share the same `status` variable. This means that we have created a race condition on the status of multiple network requests. In a traditional setting like React the solution would use pointers but in the stripped down setting of Willow we slow down. We can bind our username check status to check if there is already a network request in progress and do nothing if so.

```
1 on slowUsername, status do {
2   if availableNames.has(slowUsername) || status == "checking" then () else ...
```

This prevents us from kicking off more than one network request at a time and removes the race condition.

The last problem Willow flags is via loop detection. The full effect analysis automatically flags a loop in our code, so we have to do manual analysis to see if this is okay or not.

Now our analysis. When `availableNames` *does not include* `slowUsername` and `status == "idle"` then we want to check the username. This case leads us to the `else` branch which checks the username. The case in which `availableNames` *includes* `slowUsername` and `status == "idle"` happens when the user has settled on a username and the username has been checked. This case leads to the `then` branch which does nothing. The case in which `availableNames` *does not include* `slowUsername` and `status == "error"` brings us to the `else` branch and triggers another check. This is desirable because it's a built-in retry mechanism for the network error failure case. So Willow tells us there is a temporal loop in our code, and there is, but it is acting as our error-handling case so we can leave the code as is.



8 Related Work

React, and GUI Programming

Functional reactive programming (FRP) began with Fran [13], which modeled time-varying *behaviors* and discrete *events* as first-class values. This value-centric view of change was carried onto the web by Flapjax [33] and FrTime [7], which build dataflow graphs that automatically propagate updates, and the lineage continues through ultrametric and modal accounts of higher-order FRP [26]. Elm [9] specialized FRP for GUIs and popularized the model-view-update loop that later shaped Redux and the React ecosystem. Demetrescu et al. [11] bring dataflow constraints to imperative C/C++.

Most closely related are formal semantics for React itself. λ_{react} [29] gives a semantics for legacy class-based React, while React-tRace [27] models the modern hooks API, including the subtle ordering of effects and state updates. Willow’s semantics differ from Lee et al.’s to focus on the timing analysis. We attempt to make Willow’s temporal dependency graphs widely applicable to reactive programming while looking to React to see what the problem space is like. React-tRace fully encodes the component lifecycle of mounting and unmounting, as well as the highly asynchronous order of setter, effect, and subcomponent evaluation. If we were to directly use the tRace semantics we would have to introduce an entire layer of dynamic naming that would clutter our type system’s readability. We would like to implement Willow’s type-and-effect system for React tRace in the future if possible.

Adjoint Reactive GUI programming [20] is the nearest type-theoretic neighbor on the GUI side. It introduces a modality for receiving a value from an event and reasons about the time needed to obtain a value of a given type. However that work uses a coeffect discipline over values, Willow uses an effect discipline over computations.

A separate strand integrates session types with GUI and reactive programming. Event-Driven Multiparty Session Actors [16] use flow-sensitive effect typing for actors participating in several sessions at once, and Model-View-Update-Communicate [15] folds session typing into an Elm-style architecture.

Timing Analysis Willow’s graded next modality $\circ$ is most directly inspired by temporal session types. Das et al. [10] add timing modalities to session types to bound the parallel complexity of message-passing programs. The next operator itself is syntactically the next operator of temporal logic [36]. ESTEREL [4] and LUSTRE [21] compute over discrete logical instants and explicit clocks, building on the dataflow semantics of Kahn process networks [22]. Willow’s render is a comparable discrete tick, but Willow targets the asynchronous, externally-scheduled world of UI events and over-approximates branching as *may*-effects rather than assuming a single fixed synchronous clock. Modal FRP calculi such as Simply RaTT [2] use a Fitch-style $\circ$ modality to stage reactive computation across time steps and to rule out space leaks. Ahman [1] studies termination and resource locking for non-declarative functional reactive programs. The actor-reactor model [41] catalogs the pitfalls — glitches, stale reads, unintended feedback — that reactive languages routinely fall into; Willow attacks the same hazards statically, surfacing cyclic cascades and stale-listener

bugs as type-level effects. Time-indexed hardware systems suggest that Willow’s render model could generalize beyond software: the ephemeral history register [38] is a circuit-design construct similar to React’s scheduled state. Timeline types [35] index a type system by when signals arrive, albeit for statically-scheduled hardware rather than dynamically-scheduled UI events.

Effect systems Willow’s next modality $\circ$ is a graded modality in the sense of graded monads [24], and bounded linear logic [18] and its generalization to resource semirings [5, 17], which annotate types with quantities drawn from an algebraic structure. The link between graded temporal modalities and effects has been developed recently by Sekiyama et al., who give an algebraic axiomatization of temporal effects for higher-order recursive programs [40] and a dependent temporal type-and-effect system with answer-effect modification for delimited continuations [39]. Willow’s effects also build on the broader effect-system tradition: algebraic effects and handlers [23] and the effect-polymorphic language Frank [28] inform our treatment of effect polymorphism. Two systems for typing parallel and concurrent structure are methodologically close: Muller’s static prediction of parallel computation graphs [34] and the graph types for ADTs with futures of Rinaldi et al. [37], whose vertex structures describe dependencies among futures. We took heavy inspiration from this system while designing Willow, and we anticipate needing to incorporate vertex structures in the future if we are to allow dynamic mounting and unmounting of subcomponents. More broadly, Willow’s temporal dependency graphs connect to a long line of work on dependency analysis, from the program dependence graph [14] to semantics-based accounts of dependency [8] and their roots in secure information flow [12]. Willow differs in tracking *when* dependencies fire across renders rather than only which computations depend on which.

9 Conclusion

We have presented Willow, a core calculus for render-based reactive programming whose type-and-effect system makes the temporal behavior of reactive programs explicit. By treating the *render* as the fundamental unit of time and tracking state changes, event registrations, and scheduled events as graded effects, Willow turns timing properties that are normally buried in framework runtimes into static, checkable artifacts. The temporal next modality $\circ$ records *when*, in renders or any declared time unit, an observable update may occur, while the $\square$, $\diamond$, $\otimes$, and $\times$ modalities track the registration and teardown lifecycle of event listeners. We gave Willow a two-phase, time-aware operational semantics, proved preservation of the effect system against an instrumented version of that semantics, and showed that the inferred effects form *temporal dependency graphs* that standard graph algorithms can analyze for render cascades, inter-render loops, stale listeners, high-frequency handlers, and first-render timing. A prototype inference algorithm checks every example in the paper, and our signup-form case study shows Willow statically surfacing a stuck loading state, a request race condition, and an update loop.

Willow does not yet model the mounting and unmounting of components that Lee et al. [27] formalize. Lifting this restriction would require a dependent type system built on the vertex structures of Rinaldi et al. [37]. This would let Willow’s analysis apply directly to pre-existing render-based frameworks. By making the *when* of reactive computation a first-class, statically-analyzable property, Willow gives user interface programmers a principled way to understand the responsiveness of their applications at compile time and to spend less effort on manual temporal reasoning.

References

- [1] Danel Ahman. 2023. When Programs Have to Watch Paint Dry. In *Foundations of Software Science and Computation Structures*, Orna Kupferman and Pawel Sobocinski (Eds.). Springer Nature Switzerland, Cham, 1–23. doi:10.1007/978-3-031-30829-1_1

- [2] Patrick Bahr, Christian Uldal Graulund, and Rasmus Ejlers Møgelberg. 2019. Simply RaTT: a fitch-style modal calculus for reactive programming without space leaks. *Proceedings of the ACM on Programming Languages* 3, ICFP (July 2019), 1–27. doi:10.1145/3341713
- [3] Ishan Banerjee, Bao Nguyen, Vahid Garousi, and Atif Memon. 2013. Graphical user interface (GUI) testing: Systematic mapping and repository. *Information and Software Technology* 55, 10 (Oct. 2013), 1679–1694. doi:10.1016/j.infsof.2013.03.004
- [4] Gérard Berry and Georges Gonthier. 1992. The ESTEREL synchronous programming language: design, semantics, implementation. *Sci. Comput. Program.* 19, 2 (Nov. 1992), 87–152. doi:10.1016/0167-6423(92)90005-V
- [5] Aloïs Brunel, Marco Gaboardi, Damiano Mazza, and Steve Zdancewic. 2014. A Core Quantitative Coeffect Calculus. In *Programming Languages and Systems - 23rd European Symposium on Programming, ESOP 2014, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2014, Grenoble, France, April 5-13, 2014, Proceedings (Lecture Notes in Computer Science, Vol. 8410)*, Zhong Shao (Ed.). Springer, 351–370. doi:10.1007/978-3-642-54833-8_19
- [6] Lin Clark and React Team. 2017. React Fiber Architecture. <https://github.com/acdlite/react-fiber-architecture>.
- [7] Gregory H. Cooper and Shriram Krishnamurthi. 2006. Embedding dynamic dataflow in a call-by-value language. In *Proceedings of the 15th European Conference on Programming Languages and Systems (Vienna, Austria) (ESOP'06)*. Springer-Verlag, Berlin, Heidelberg, 294–308. doi:10.1007/11693024_20
- [8] Patrick Cousot. 2019. Abstract Semantic Dependency. In *Static Analysis - 26th International Symposium, SAS 2019, Porto, Portugal, October 8-11, 2019, Proceedings (Lecture Notes in Computer Science, Vol. 11822)*, Bor-Yuh Evan Chang (Ed.). Springer, 389–410. doi:10.1007/978-3-030-32304-2_19
- [9] Evan Czaplicki. 2012. Elm : Concurrent FRP for Functional GUIs. <https://www.semanticscholar.org/paper/Elm-%3A-Concurrent-FRP-for-Functional-GUIs-Czaplicki/1791a8a278b83c54425d7581cb45320feba5f4b0>
- [10] Ankush Das, Jan Hoffmann, and Frank Pfenning. 2018. Parallel complexity analysis with temporal session types. *Proc. ACM Program. Lang.* 2, ICFP (July 2018), 91:1–91:30. doi:10.1145/3236786
- [11] Camil Demetrescu, Irene Finocchi, and Andrea Ribichini. 2011. Reactive imperative programming with dataflow constraints. In *Proceedings of the 2011 ACM international conference on Object oriented programming systems languages and applications (OOPSLA '11)*. Association for Computing Machinery, New York, NY, USA, 407–426. doi:10.1145/2048066.2048100
- [12] Dorothy E. Denning and Peter J. Denning. 1977. Certification of Programs for Secure Information Flow. *Commun. ACM* 20, 7 (1977), 504–513. doi:10.1145/359636.359712
- [13] Conal Elliott and Paul Hudak. 1997. Functional reactive animation. In *Proceedings of the second ACM SIGPLAN international conference on Functional programming (ICFP '97)*. Association for Computing Machinery, New York, NY, USA, 263–273. doi:10.1145/258948.258973
- [14] Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. 1987. The Program Dependence Graph and Its Use in Optimization. *ACM Transactions on Programming Languages and Systems* 9, 3 (1987), 319–349. doi:10.1145/24039.24041
- [15] Simon Fowler. 2019. Model-View-Update-Communicate: Session Types meet the Elm Architecture. doi:10.48550/ARXIV.1910.11108 Version Number: 3.
- [16] Simon Fowler and Raymond Hu. 2017. Event-Driven Multiparty Session Actors. Presented at the 11th ACM SIGPLAN Workshop on Higher-Order Programming with Effects (HOPE 2023). No proceedings.
- [17] Dan R. Ghica and Alex I. Smith. 2014. Bounded Linear Types in a Resource Semiring. In *Programming Languages and Systems*, Zhong Shao (Ed.). Springer, Berlin, Heidelberg, 331–350. doi:10.1007/978-3-642-54833-8_18
- [18] Jean-Yves Girard, Andre Scedrov, and Philip J. Scott. 1992. Bounded linear logic: a modular approach to polynomial-time computability. *Theoretical Computer Science* 97, 1 (April 1992), 1–66. doi:10.1016/0304-3975(92)90386-T
- [19] Google. 2020. Largest Contentful Paint (LCP). web.dev. <https://web.dev/articles/lcp>. Core Web Vitals metric for perceived load speed; sites should target LCP of 2.5 seconds or less..
- [20] Christian Uldal Graulund, Dmitriy Szamozvancev, and Neel Krishnaswami. 2021. Adjoint Reactive GUI Programming. In *Foundations of Software Science and Computation Structures*, Stefan Kiefer and Christine Tasson (Eds.). Vol. 12650. Springer International Publishing, Cham, 289–309. doi:10.1007/978-3-030-71995-1_15 Series Title: Lecture Notes in Computer Science.
- [21] N. Halbwachs, P. Caspi, P. Raymond, and D. Pilaud. 1991. The synchronous data flow programming language LUSTRE. *Proc. IEEE* 79, 9 (1991), 1305–1320. doi:10.1109/5.97300
- [22] Gilles Kahn. 1974. The Semantics of a Simple Language for Parallel Programming. In *IFIP Congress*. <https://api.semanticscholar.org/CorpusID:18030506>
- [23] Ohad Kammar, Sam Lindley, and Nicolas Oury. 2013. Handlers in action. In *Proceedings of the 18th ACM SIGPLAN international conference on Functional programming (ICFP '13)*. Association for Computing Machinery, New York, NY, USA, 145–158. doi:10.1145/2500365.2500590

- [24] Shin-ya Katsumata. 2014. Parametric effect monads and semantics of effect systems. In *The 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '14, San Diego, CA, USA, January 20-21, 2014*, Suresh Jagannathan and Peter Sewell (Eds.). ACM, 633–646. doi:10.1145/2535838.2535846
- [25] David Khourshid. 2022. Goodbye, useEffect. Reactathon 2022 Conference Talk, Real World React (YouTube). <https://www.youtube.com/watch?v=HPoC-k7Rxwo>
- [26] Neelakantan R. Krishnaswami and Nick Benton. 2011. Ultrametric Semantics of Reactive Programs. In *2011 IEEE 26th Annual Symposium on Logic in Computer Science*. IEEE, Toronto, ON, Canada, 257–266. doi:10.1109/LICS.2011.38
- [27] Jay Lee, Joongwon Ahn, and Kwangkeun Yi. 2025. React-tRace: A Semantics for Understanding React Hooks. doi:10.1145/3763067 arXiv:2507.05234 [cs].
- [28] Sam Lindley, Conor McBride, and Craig McLaughlin. 2017. Do be do be do. doi:10.48550/arXiv.1611.09259 arXiv:1611.09259 [cs].
- [29] Magnus Madsen, Ondřej Lhoták, and Frank Tip. 2020. A Semantics for the Essence of React. In *LIPICs, Volume 166, ECOOP 2020*, Vol. 166. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 12:1–12:26. doi:10.4230/LIPICs.ECOOP.2020.12 Artwork Size: 26 pages, 814986 bytes Medium: application/pdf.
- [30] A.M. Memon. 2002. GUI testing: pitfalls and process. *Computer* 35, 8 (Aug. 2002), 87–88. doi:10.1109/MC.2002.1023795
- [31] Meta Platforms. 2013. React: A JavaScript Library for Building User Interfaces. <https://react.dev>.
- [32] Meta Platforms. 2024. React Compiler. <https://react.dev/learn/react-compiler/introduction>.
- [33] Leo A. Meyerovich, Arjun Guha, Jacob Baskin, Gregory H. Cooper, Michael Greenberg, Aleks Bromfield, and Shriram Krishnamurthi. 2009. Flapjax: a programming language for Ajax applications. In *Proceedings of the 24th ACM SIGPLAN Conference on Object Oriented Programming Systems Languages and Applications (Orlando, Florida, USA) (OOPSLA '09)*. Association for Computing Machinery, New York, NY, USA, 1–20. doi:10.1145/1640089.1640091
- [34] Stefan K. Muller. 2022. Static prediction of parallel computation graphs. *Proc. ACM Program. Lang.* 6, POPL (Jan. 2022), 46:1–46:31. doi:10.1145/3498708
- [35] Rachit Nigam, Pedro Henrique Azevedo de Amorim, and Adrian Sampson. 2023. Modular Hardware Design with Timeline Types. *Proc. ACM Program. Lang.* 7, PLDI (June 2023), 120:343–120:367. doi:10.1145/3591234
- [36] Amir Pnueli. 1977. The temporal logic of programs. In *Proceedings of the 18th Annual Symposium on Foundations of Computer Science (SFCS '77)*. IEEE Computer Society, USA, 46–57. doi:10.1109/SFCS.1977.32
- [37] Francis Rinaldi, June Wunder, Arthur Azevedo de Amorim, and Stefan K. Muller. 2024. Pipelines and Beyond: Graph Types for ADTs with Futures. *Proc. ACM Program. Lang.* 8, POPL (Jan. 2024), 17:482–17:511. doi:10.1145/3632859
- [38] Daniel L. Rosenband. 2004. The ephemeral history register: flexible scheduling for rule-based designs. In *Proceedings of the Second ACM/IEEE International Conference on Formal Methods and Models for Co-Design (MEMOCODE '04)*. IEEE Computer Society, USA, 189–198. doi:10.1109/MEMCOD.2004.1459853
- [39] Taro Sekiyama and Hiroshi Unno. 2023. Temporal Verification with Answer-Effect Modification: Dependent Temporal Type-and-Effect System with Delimited Continuations. *Proc. ACM Program. Lang.* 7, POPL (Jan. 2023), 71:2079–71:2110. doi:10.1145/3571264
- [40] Taro Sekiyama and Hiroshi Unno. 2025. Algebraic Temporal Effects: Temporal Verification of Recursively Typed Higher-Order Programs. *Proc. ACM Program. Lang.* 9, POPL (Jan. 2025), 78:2306–78:2336. doi:10.1145/3704914
- [41] Sam Van den Vonder, Joeri De Koster, Florian Myter, and Wolfgang De Meuter. 2017. Tackling the awkward squad for reactive programming: the actor-reactor model. In *Proceedings of the 4th ACM SIGPLAN International Workshop on Reactive and Event-Based Languages and Systems (REBLS 2017)*. Association for Computing Machinery, New York, NY, USA, 27–33. doi:10.1145/3141858.3141863
- [42] Zhanyong Wan and Paul Hudak. 2000. Functional reactive programming from first principles. In *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation (Vancouver, British Columbia, Canada) (PLDI '00)*. Association for Computing Machinery, New York, NY, USA, 242–252. doi:10.1145/349299.349331

A Collected Grammars

For reference, we collect here the full grammars of Willow, reproduced from the main text: the term syntax (Figure 8) and the grammar of types and effects (Figure 9).

Components	C	$::=$	$\text{comp } A(x_1 : \tau_1, x_2 : \tau_2, \dots, x_n : \tau_n) : \tau \{ p_1 \ p_2 \ \dots \ p_n \ \text{return } x \}$
Declarations	p	$::=$	$\text{let } x = e; \mid \text{state } x, \text{ set } X \text{ default } e;$ $\text{on } x_1, x_2, \dots, x_n \text{ do } \{ e \}; \mid \text{comp } x = A(x_1, x_2, \dots, x_n);$
Expressions	e	$::=$	$x \mid c \in \{\text{true}, \text{false}, ()\} \mid e_1 \ e_2 \mid e_1; e_2 \mid \lambda x. e$ $\text{if } x \text{ then } e_1 \text{ else } e_2 \mid (e_1, e_2) \mid \text{fst } e \mid \text{snd } e$ $\text{bind } \ell(\bar{v}) \ e \mid \text{once } \ell(\bar{v}) \ e \mid \text{cancel } \ell(\bar{v}) \mid \text{remove } \ell(\bar{v})$

Fig. 8. Syntax of Willow Programs (reproduced from Figure 1).

Types	τ	$::=$	$\text{unit} \mid \text{bool} \mid \tau_1 \times \tau_2 \mid (\tau_1 \rightarrow \tau_2 \mid F)$
Effects	F	$::=$	$\cdot \mid @x \mid \ell(\bar{v}) \mid \circ^{Nu} F \mid F_1 * F_2 \mid F_1 + F_2$ $\diamond^{\ell(\bar{v})}(F) \mid \square^{\ell(\bar{v})}(F) \mid \oslash \ell(\bar{v}) \mid \blacktimes \ell(\bar{v})$
Event Labels	$\ell(\bar{v})$	$::=$	$\ell(\bar{v})$ where $\bar{v}$ is a tuple of base values
Time Unit	u	$::=$	$r \mid n \mid \text{ms} \mid \dots$

Fig. 9. Grammar of Willow Types and Effects (reproduced from Figure 5).

B Types

B.1 Program

These rules establish program-level well-formedness. The judgement $\Sigma \Rightarrow \text{comp } A_1, \dots, \text{comp } A_n \Rightarrow \Sigma'$ checks a sequence of component definitions against a starting signature Σ , threading each type-checked component back into the signature so that later components may refer to earlier ones, and yields the final signature Σ' that maps every component name to its code, effect environment Δ , and typing environment Γ .

$$\begin{array}{c}
 \text{(WF COMP)} \\
 \frac{\Sigma \Rightarrow \text{comp } A(\overline{x_i : \tau_i}) : \tau \{ p_1, p_2, \dots, p_n \} \Rightarrow (\Delta, \Gamma)}{\Sigma \Rightarrow \text{comp } A \Rightarrow \Sigma, A : (\text{comp } A, \Delta, \Gamma)} \\
 \\
 \text{(WF PROGRAM)} \\
 \frac{\Sigma \Rightarrow \text{comp } A_1(\overline{x_i : \tau_i}) : \tau \{ p_1, p_2, \dots, p_n \} \Rightarrow (\Delta, \Gamma) \quad \Sigma, A_1 : (\text{comp } A_1, \Delta, \Gamma) \Rightarrow \text{comp } A_2, \dots, \text{comp } A_n \Rightarrow \Sigma'}{\Sigma \Rightarrow \text{comp } A_1, \dots, \text{comp } A_n \Rightarrow \Sigma'}
 \end{array}$$

Fig. 10. Program-level well-formedness.

B.2 Component

Component-level typing checks a single component against the program signature Σ . The judgement $\Sigma \Rightarrow \text{comp } A \Rightarrow (\Delta, \Gamma)$ says that, given the declared effect environment Δ , the component's declaration block type-checks and produces the typing environment Γ that binds its state, let, and subcomponent names.

$$\begin{array}{c} \text{(TY COMPONENT)} \\ \hline \Sigma ; \overline{x_i : \tau_i} ; \Delta \vdash p_1, p_2, \dots, p_n \Rightarrow \Gamma \\ \hline \Sigma \Rightarrow \text{comp } A(\overline{x_i : \tau_i}) : \tau \{ p_1, p_2, \dots, p_n \} \Rightarrow (\Delta, \Gamma) \end{array}$$

Fig. 11. Component-level typing.

B.3 Declaration

Declaration typing threads a typing environment through a component's declaration block. The judgement $\Sigma ; \Gamma ; \Delta \vdash p \Rightarrow \Gamma'$ checks a single declaration p under signature Σ , effect environment Δ , and incoming environment Γ , extending it to Γ' ; the sequencing judgement $\Sigma ; \Gamma ; \Delta \vdash p_1; \dots; \text{return } x : \tau$ chains these across a whole block that ends in a return of type τ . Each rule additionally checks, via the “causes” judgement (Section C.1), that the effects a declaration performs are permitted by Δ .

$$\begin{array}{c} \text{(TY RETURN DECL)} \\ \hline \Delta \vdash @x \Rightarrow @\text{return} \\ \hline \Sigma ; \Gamma, x : \tau ; \Delta \vdash \text{return } x : \tau \\ \hline \text{(DECLARATIONS)} \\ \hline \Sigma ; \Gamma_1 ; \Delta \vdash p_1 \Rightarrow \Gamma_2 \quad \Sigma ; \Gamma_2 ; \Delta \vdash p_2; \dots; p_n; \text{return } x : \tau \\ \hline \Sigma ; \Gamma_1 ; \Delta \vdash p_1; p_2; \dots; p_n; \text{return } x : \tau \\ \hline \text{(TY STATE DECL)} \\ \hline \Gamma \vdash e : \tau \mid \cdot \quad x[] \mid F \in \Delta \quad \Gamma' = \Gamma, x : \tau, \text{setX} : ((\tau \rightarrow \tau \mid \cdot) \rightarrow \text{unit} \mid \circ^{1r}@x) \\ \hline \Sigma ; \Gamma ; \Delta \vdash \text{state } x, \text{setX default } e; \Rightarrow \Gamma' \\ \hline \text{(TY LET DECL)} \\ \hline \Gamma \vdash e : \tau \mid \cdot \quad \bar{y} = df(e) \quad x[\bar{y}] \mid F \in \Delta \\ \hline \Sigma ; \Gamma ; \Delta \vdash \text{let } x = e; \Rightarrow \Gamma, x : \tau \\ \hline \text{(TY ON DECL)} \\ \hline \Gamma \vdash e : \text{unit} \mid F \quad \forall i \in [n], \Delta \vdash @x_i \Rightarrow F \\ \hline \Sigma ; \Gamma ; \Delta \vdash \text{on } x_1, x_2, \dots, x_n \text{ do } \{ e \}; \Rightarrow \Gamma \\ \hline \text{(TY SUBCOMP DECL)} \\ \hline A : (\text{comp } A(\overline{z_i : \tau_i}) : \tau_r \{ p \}, \Delta_A, \Gamma_A) \in \Sigma \\ \hline \Delta \vdash @x_i \Rightarrow @y.z_i \quad \Delta \vdash @y.\text{return} \Rightarrow @y \quad \overline{x_i} : \tau_i \in \Gamma \\ \hline \overline{v_\Delta} = fv(\Delta_A) \quad \Delta_A[y.v_{\Delta i}/v_{\Delta i}] \subseteq \Delta \quad \overline{v_\Gamma} = fv(\Gamma_A) \\ \hline \Sigma ; \Gamma ; \Delta \vdash \text{comp } y = A(x_1, x_2, \dots, x_n); \Rightarrow \Gamma, \Gamma_A[y.v_{\Gamma i}/v_{\Gamma i}], y : \tau_r \end{array}$$

Fig. 12. Declaration-level typing rules.

B.4 Expression

Expression typing is a standard type-and-effect system. The judgement $\Gamma \vdash e : \tau \mid F$ assigns expression e the type τ and the effect F describing the update-queue items its evaluation may emit — state changes, listener registrations, cancellations, and removals. Pure expressions carry the empty effect $\cdot$, sequencing and application combine sub-effects with $*$, branching combines them with $+$, and the event primitives `bind`, `once`, `cancel`, and `remove` introduce the event-guarded modalities and teardown effects.

$$\begin{array}{c}
\begin{array}{c} \text{(UNIT)} \\ \hline \Gamma \vdash () : \text{unit} \mid \cdot \end{array} \quad \begin{array}{c} \text{(BOOL)} \\ c \in \{\text{true}, \text{false}\} \\ \hline \Gamma \vdash c : \text{bool} \mid \cdot \end{array} \quad \begin{array}{c} \text{(BASE)} \\ c \in \alpha \\ \hline \Gamma \vdash c : \alpha \mid \cdot \end{array} \quad \begin{array}{c} \text{(VAR)} \\ \hline \Gamma, x : \tau \vdash x : \tau \mid \cdot \end{array} \\
\begin{array}{c} \text{(APP)} \\ \hline \Gamma \vdash e_1 : (\tau_1 \rightarrow \tau_2 \mid F) \mid F_1 \quad \Gamma \vdash e_2 : \tau_1 \mid F_2 \\ \hline \Gamma \vdash e_1 e_2 : \tau_2 \mid F_1 * F_2 * F \end{array} \quad \begin{array}{c} \text{(FN)} \\ \hline \Gamma, x : \tau_1 \vdash e : \tau_2 \mid F \\ \hline \Gamma \vdash \lambda x. e : (\tau_1 \rightarrow \tau_2 \mid F) \mid \cdot \end{array} \\
\begin{array}{c} \text{(SEQ)} \\ \hline \Gamma \vdash e_1 : \tau_1 \mid F_1 \quad \Gamma \vdash e_2 : \tau_2 \mid F_2 \\ \hline \Gamma \vdash e_1; e_2 : \tau_2 \mid F_1 * F_2 \end{array} \quad \begin{array}{c} \text{(BRANCH)} \\ \hline \Gamma \vdash e_1 : \text{bool} \mid F_1 \quad \Gamma \vdash e_2 : \tau \mid F_2 \quad \Gamma \vdash e_3 : \tau \mid F_3 \\ \hline \Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau \mid F_1 * (F_2 + F_3) \end{array} \\
\begin{array}{c} \text{(PROD)} \\ \hline \Gamma \vdash e_1 : \tau_1 \mid F_1 \quad \Gamma \vdash e_2 : \tau_2 \mid F_2 \\ \hline \Gamma \vdash (e_1, e_2) : \tau_1 \times \tau_2 \mid F_1 * F_2 \end{array} \quad \begin{array}{c} \text{(FST)} \\ \hline \Gamma \vdash e_1 : \tau_1 \times \tau_2 \mid F \\ \hline \Gamma \vdash \text{fst } e_1 : \tau_1 \mid F \end{array} \quad \begin{array}{c} \text{(SND)} \\ \hline \Gamma \vdash e_1 : \tau_1 \times \tau_2 \mid F \\ \hline \Gamma \vdash \text{snd } e_1 : \tau_2 \mid F \end{array} \\
\begin{array}{c} \text{(TY BIND)} \\ \hline \Gamma \vdash e : (\tau \rightarrow \text{unit} \mid F) \mid F_e \quad \tau = \Sigma_E(\ell(\bar{v})) \\ \hline \Gamma \vdash \text{bind } \ell(\bar{v}) e : \text{unit} \mid F_e * \square^{\ell(\bar{v})}(F) \end{array} \quad \begin{array}{c} \text{(TY ONCE)} \\ \hline \Gamma \vdash e : (\tau \rightarrow \text{unit} \mid F) \mid F_e \quad \tau = \Sigma_E(\ell(\bar{v})) \\ \hline \Gamma \vdash \text{once } \ell(\bar{v}) e : \text{unit} \mid F_e * \diamond^{\ell(\bar{v})}(F) \end{array} \\
\begin{array}{c} \text{(TY CANCEL)} \\ \hline \Gamma \vdash \text{cancel } \ell(\bar{v}) : \text{unit} \mid \oslash \ell(\bar{v}) \end{array} \quad \begin{array}{c} \text{(TY REMOVE)} \\ \hline \Gamma \vdash \text{remove } \ell(\bar{v}) : \text{unit} \mid \times \ell(\bar{v}) \end{array}
\end{array}$$

Fig. 13. Expression typing rules.

C Side Conditions

C.1 Typing – “causes”

The “causes” judgement $\Delta \vdash @x \Rightarrow F$ reads “a change in x is a sufficient cause for the effect F ”. It is the side condition that ties the declared effect environment Δ to what a declaration may do: a state, let, effect, or subcomponent declaration type-checks only when every effect it performs is caused, according to Δ , by a variable it depends on. The three rules cover the direct case (where x ’s own entry in Δ dominates F), the dependency case (a change in x propagates to any y that lists x among its dependencies), and closure under the $*$ combinator.

$$\begin{array}{c}
\frac{(x \text{ CAUSES } F) \quad x[\dots] \mid F' \in \Delta \quad F \leq F'}{\Delta \vdash @x \Rightarrow F} \quad \frac{(x \text{ CAUSES } y) \quad y[\dots, x, \dots] \mid F' \in \Delta}{\Delta \vdash @x \Rightarrow @y} \\
\frac{(x \text{ CAUSES } F_1 * F_2) \quad \Delta \vdash @x \Rightarrow F'_1 \quad \Delta \vdash @x \Rightarrow F'_2 \quad F_1 \leq F'_1 \quad F_2 \leq F'_2}{\Delta \vdash @x \Rightarrow F_1 * F_2}
\end{array}$$

Fig. 14. The “causes” judgement.

C.2 Sub-Effecting

Sub-effecting is the subtyping relation $F \leq F'$ on effects, read “ F is a sub-effect of F' ”: anywhere an effect F' is expected, an F with $F \leq F'$ may be supplied. The rules propagate sub-effecting through the $+$ and $*$ combinators, make the delay modality $\circ^t$ monotone in its body and splittable and additive in its grade t , provide reflexivity and transitivity, and let the empty effect $\cdot$ be refined into any single leaf effect (a state change, an event, an event-guarded modality, or a teardown), so a computation may always be over-approximated by a larger effect.

$$\begin{array}{c}
\frac{(SE-PLUS-R) \quad F \leq F_1 \quad F \leq F_2}{F \leq F_1 + F_2} \quad \frac{(SE-PLUS-L1)}{F_1 + F_2 \leq F_1} \quad \frac{(SE-PLUS-L2)}{F_1 + F_2 \leq F_2} \quad \frac{(SE-MULT)}{F \leq F_{i \in \{1,2\}}} \quad \frac{(SE-ZERO-L)}{F \leq F'} \quad \frac{(SE-ZERO-R)}{F \leq F'} \\
\frac{(SE-DELAY) \quad (\circ^{t-t'} F) \leq F' \quad t \leq t'}{\circ^t F \leq \circ^{t'} F'} \quad \frac{(SE-SPLIT-L)}{\circ^{t_1+t_2} F \leq \circ^{t_1} \circ^{t_2} F} \quad \frac{(SE-SPLIT-R)}{\circ^{t_1} \circ^{t_2} F \leq \circ^{t_1+t_2} F} \quad \frac{(SE-EQ)}{F = F'} \\
\frac{(SE-TRANS) \quad F \leq F' \quad F' \leq F''}{F \leq F''} \quad \frac{(SE-DELAY-EQ) \quad F \leq F'}{\circ^t F \leq \circ^t F'} \quad \frac{(SE-SUBEFFECTING)}{\cdot \leq @x} \quad \frac{(SE-SUBEFFECTING-NSE)}{\cdot \leq \ell(\bar{v})} \\
\frac{(SE-SUBEFFECTING-EVENTUALLY)}{\cdot \leq \Diamond \ell(\bar{v})(F)} \quad \frac{(SE-SUBEFFECTING-ALWAYS)}{\cdot \leq \Box \ell(\bar{v})(F)} \\
\frac{(SE-EVENTUALLY-BODY) \quad F \leq F'}{\Diamond \ell(\bar{v})(F) \leq \Diamond \ell(\bar{v})(F')} \quad \frac{(SE-ALWAYS-BODY) \quad F \leq F'}{\Box \ell(\bar{v})(F) \leq \Box \ell(\bar{v})(F')} \\
\frac{(SE-SUBEFFECTING-CANCEL)}{\cdot \leq \Diamond \ell(\bar{v})} \quad \frac{(SE-SUBEFFECTING-REMOVE)}{\cdot \leq \Box \ell(\bar{v})}
\end{array}$$

Fig. 15. Sub-effecting.

D Semantics

D.1 External Scheduler

Three new objects are added to the component-level housekeeping semantics.

- $\mathcal{E}$, the *external scheduler*: an external scheduler presents an ordered list of *event firings*, each of the form $(\ell(\bar{v}_1), v_2)$ where $\ell(\bar{v}_1)$ is the full event label (label ℓ together with its discriminant value tuple $\bar{v}_1$) and v_2 is the *payload value* delivered to listeners when this firing is dispatched. The same event may occur more than once in $\mathcal{E}$, possibly with different

payloads, so a single render can see an event fire multiple times. New elements may be present in $\mathcal{E}$ at the beginning of each housekeeping phase. $\mathcal{E}$ is treated as an unknown black box: well-typedness of configurations requires only that each emitted payload v_2 inhabit $\Sigma_E(\ell(\overline{v_1}))$, the data type declared for that event in the global event signature.

- $\mathcal{L}$, the *listener map*: a partial function $\mathcal{L} : \ell(\overline{v}) \rightarrow \mathcal{P}_{fin}(Closure \times \{\diamond, \square\} \times Effect)$ mapping exact events to finite sets of registered listener records. Each record (c, m, F) pairs the closure c with a mode tag — $\diamond$ for a one-shot listener and $\square$ for a persistent one — and the closure’s body effect F , recorded at the point of registration. The effect component F is an instrumented annotation only: it is used by well-typedness of configurations to witness the produced update queue when the listener fires, and is erased in bare execution. If $\ell(\overline{v}) \notin \text{dom}(\mathcal{L})$ we treat $\mathcal{L}(\ell(\overline{v})) = \emptyset$.
- $\mathcal{C}$, the *cancellation multiset*: a finite multiset of exact events $\ell(\overline{v})$. When POP EVENT is about to dispatch an event, it first consults $\mathcal{C}$: if that event has multiplicity ≥ 1 in $\mathcal{C}$, the dispatch is dropped and the event’s multiplicity in $\mathcal{C}$ is decremented by one; otherwise the event is dispatched normally. Because $\mathcal{C}$ is a multiset, the same event may be cancelled more than once, in which case each cancellation drops one subsequent dispatch. We write $\{e\}$ for a singleton multiset, $\uplus$ for multiset union (which adds multiplicities), and $\setminus$ for multiset difference (which subtracts multiplicities).

D.1.1 External task finished.

This judgement, $\mathcal{E} \Rightarrow_{\text{ext}} U_{\mathcal{E}}$, reads the pending event firings out of the external scheduler $\mathcal{E}$ as a queue of update-queue items to be prepended at the start of a housekeeping phase. Because $\mathcal{E}$ is an opaque black box, the single rule simply exposes whatever ordered list of firings it currently holds.

$$\frac{\text{(EXTERNAL TASK FINISHED)} \quad \mathcal{E} = (\ell_1(\overline{v_{11}}), v_{21}), \dots, (\ell_n(\overline{v_{1n}}), v_{2n})}{\mathcal{E} \Rightarrow_{\text{ext}} (\ell_1(\overline{v_{11}}), v_{21}), \dots, (\ell_n(\overline{v_{1n}}), v_{2n})}$$

Fig. 16. External tasks finishing.

D.1.2 Listener-safe queues.

A listener fires in response to an event, so it must not be able to inject a fresh labelled event back into the queue. It may, however, enqueue the internal housekeeping items: state updates, cancellations, listener removals, and new listener registrations. The judgement $\vdash U \text{ lsafe}$ holds exactly when U contains no labelled events.

$$\frac{\begin{array}{c} \text{(LSAFE EMPTY)} \\ \vdash \cdot \text{ lsafe} \end{array} \quad \frac{\begin{array}{c} \text{(LSAFE SETTER)} \\ \vdash U \text{ lsafe} \end{array} \quad \frac{\begin{array}{c} \text{(LSAFE CANCEL)} \\ \vdash U \text{ lsafe} \end{array}}{\vdash \text{cancel}(\ell(\overline{v})), U \text{ lsafe}}}{\vdash \text{setter}_y(f), U \text{ lsafe}} \quad \frac{\begin{array}{c} \text{(LSAFE LISTEN)} \\ \vdash U \text{ lsafe} \end{array}}{\vdash \text{listen}(\ell(\overline{v}), c, m, F), U \text{ lsafe}} \\ \frac{\vdash \text{remove}(\ell(\overline{v})), U \text{ lsafe}}{\vdash \text{remove}(\ell(\overline{v})), U \text{ lsafe}}$$

Fig. 17. Listener-safe queues.

Lemma (Expression evaluation is listener-safe). If $\Sigma ; V \vdash e \Downarrow v ; U$, then $\vdash U \text{ lsafe}$.

Proof. By induction on the expression-level semantic derivation. We check that every rule in EXPRESSION above either emits the empty queue $\cdot$ or extends with one of the four item kinds admitted by the listener-safe judgement:

- VAR, CONSTANT, CLOSURE CREATE: $U = \cdot$; apply LSAFE EMPTY.
- FST, SND: U is the queue of a subderivation; apply the IH to that subderivation.
- SEQ, PROD, BRANCH1, BRANCH2, FUNCTION APP: U is the concatenation of subderivation queues; closure of the listener-safe judgement under concatenation (each LSAFE rule extends a listener-safe tail with one admitted item, so an induction on the first queue gives concatenation closure) combined with the IH on each subderivation discharges the case.
- SETTER APP: $U = U_1, U_2, \text{setter}_x((\lambda y.e_3, V'))$. U_1 and U_2 are listener-safe by the IH; the trailing item is admitted by LSAFE SETTER.
- CANCEL, REMOVE: U is a single cancel or remove item; admitted by LSAFE CANCEL / LSAFE REMOVE.
- BIND, ONCE: $U = U', \text{listen}(\dots)$. U' is listener-safe by the IH; the trailing listen item is admitted by LSAFE LISTEN.

No expression-level rule emits an event-firing item $(\ell(\overline{v_1}), v_2)$; labelled events enter the working queue only via FLUSH EXTERNAL EVENTS at the component level, never by user code. $\square$

D.2 Component

The extended judgment $\Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash C$ status U' threads $\mathcal{L}$ and $\mathcal{C}$ as store-passed state through all component rules. Updated values of $\mathcal{C}$ or $\mathcal{L}$ appear in the conclusion's corresponding argument positions; rules that do not modify them pass the same value through unchanged.

$$\begin{array}{c}
 \text{(INIT)} \\
 \hline
 \Sigma ; [\cdot | \cdot] ; x_1 : v_1, \dots, x_n : v_n ; \text{"} \vdash p \Downarrow v_r ; U ; V \\
 \hline
 \Sigma ; \emptyset ; V ; V ; \emptyset \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ rendered } U \\
 \text{(NO UPDATES)} \\
 \hline
 \Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ rendered } \cdot \\
 \hline
 \Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ waiting } \cdot \\
 \text{(FLUSH EXTERNAL EVENTS)} \\
 \hline
 \Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ rendered } U \\
 \mathcal{E} \Rightarrow_{\text{ext}} U_{\mathcal{E}} \\
 \hline
 \Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ rendered } U_{\mathcal{E}}, U \\
 \text{(POP SETTER)} \\
 \hline
 \Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ rendered } \text{setter}_y(f), U' \\
 V_s \Rightarrow \text{setter}_y(f) \Rightarrow V'_s \\
 \hline
 \Sigma ; \mathcal{L} ; V_e ; V'_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ rendered } U'
 \end{array}$$

$$\begin{array}{c}
\text{(POP EVENT)} \\
\hline
\Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ rendered } (\ell(\overline{v_1}), v_2), U' \\
\mathcal{C}(\ell(\overline{v_1})) = 0 \\
\mathcal{L}(\ell(\overline{v_1})) = \{((\lambda x_1.e_1, V_1), m_1, F_1), \dots, ((\lambda x_k.e_k, V_k), m_k, F_k)\} \quad \overline{m_i} \in \{\diamond, \square\} \\
\frac{\Sigma ; V_i, x_i = v_2 \vdash e_i \Downarrow () ; U_i \quad \vdash U_i \text{ lsafe}}{\mathcal{L}' = \mathcal{L}[\ell(\overline{v_1}) \mapsto \{((\lambda x_i.e_i, V_i), m_i, F_i) \mid m_i = \square\}]} \\
\hline
\Sigma ; \mathcal{L}' ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ rendered } U', U_1, \dots, U_k \\
\text{(POP EVENT CANCELLED)} \\
\Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ rendered } (\ell(\overline{v_1}), v_2), U' \\
\mathcal{C}(\ell(\overline{v_1})) \geq 1 \quad \mathcal{C}' = \mathcal{C} \setminus \{\ell(\overline{v_1})\} \\
\hline
\Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C}' \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ rendered } U' \\
\text{(POP CANCEL)} \\
\Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v_1}) : \tau \{ p \} \text{ rendered cancel}(\ell(\overline{v_2})), U' \\
\hline
\Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \uplus \{\ell(\overline{v_2})\} \vdash \text{comp } C(\overline{x = v_1}) : \tau \{ p \} \text{ rendered } U' \\
\text{(POP LISTEN)} \\
\Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v_1}) : \tau \{ p \} \text{ rendered listen}(\ell(\overline{v_2}), c, m, F), U' \\
m \in \{\diamond, \square\} \\
\mathcal{L}' = \mathcal{L}[\ell(\overline{v_2}) \mapsto \mathcal{L}(\ell(\overline{v_2})) \cup \{(c, m, F)\}] \\
\hline
\Sigma ; \mathcal{L}' ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v_1}) : \tau \{ p \} \text{ rendered } U' \\
\text{(POP REMOVE LISTENER)} \\
\Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v_1}) : \tau \{ p \} \text{ rendered remove}(\ell(\overline{v_2})), U' \\
\mathcal{L}' = \mathcal{L}[\ell(\overline{v_2}) \mapsto \emptyset] \\
\hline
\Sigma ; \mathcal{L}' ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v_1}) : \tau \{ p \} \text{ rendered } U' \\
\text{(WAITING TO RENDERED)} \\
\Sigma ; \mathcal{L} ; V_e ; V_s ; \mathcal{C} \vdash \text{comp } C(\overline{x = v}) : \tau \{ p \} \text{ waiting} \cdot \\
\Sigma ; [V_e \mid V_s] ; x = v' ; \text{"} \vdash p \Downarrow v ; U' ; V' \quad \exists i \text{ s.t. } v_i \neq v'_i \\
\hline
\Sigma ; \mathcal{L} ; V' ; V' ; \mathcal{C} \vdash \text{comp } C(\overline{x = v'}) : \tau \{ p \} \text{ rendered } U'
\end{array}$$

Fig. 18. Component-level stepping.

$$\begin{array}{c}
\text{(VALID SETTER)} \\
\hline
V' = \text{removeSetters}(V) \quad \Sigma ; V', x = v \vdash e \Downarrow v' ; \cdot \\
\hline
V_s, x = v \Rightarrow \text{setter}_x((\lambda x.e, V)) \Rightarrow V_s, x = v'
\end{array}$$

Fig. 19. Setter validity.

D.3 Declarations

The declaration evaluation judgement $\Sigma ; [V_e \mid V_s] ; V ; c \vdash p \Downarrow v ; U ; V'$ is the big-step operational semantics of a declaration block. Under signature Σ , previous-render environment V_e , and setter store V_s , it evaluates the block p in the local environment V to a return value v , emitting an update queue U and a final environment V' . The two rules here handle a plain `VAR`

(let-bound) declaration and the terminating RETURN; the INITIALIZE and RERENDER rules below cover the remaining declaration forms in their two evaluation modes.

$$\begin{array}{c}
 \text{(VAR)} \\
 \frac{\Sigma ; V \vdash e \Downarrow v_0 ; \cdot \quad \Sigma ; [V_e \mid V_s] ; V, x = v_0 ; c \vdash p \Downarrow v ; U ; V'}{\Sigma ; [V_e \mid V_s] ; V ; c \vdash \text{let } x = e ; p \Downarrow v ; U ; V'} \\
 \text{(RETURN)} \\
 \frac{\Sigma ; V \vdash x \Downarrow v ; \cdot}{\Sigma ; [V_e \mid V_s] ; V ; c \vdash \text{return } x \Downarrow v ; \cdot ; V}
 \end{array}$$

Fig. 20. Declaration evaluation: VAR and RETURN.

D.3.1 Initialize.

These rules give the *first-render* evaluation of state, effect, and subcomponent declarations, taken when the component is first mounted (the previous-render environments are empty, $V_e = V_s = \cdot$). A state declaration installs its initial value and setter, an effect block always runs once and enqueues its on closure, and a subcomponent is recursively initialized with its arguments and has its resulting environment merged in under the subcomponent's prefix.

$$\begin{array}{c}
 \text{(STATE INIT)} \\
 \frac{\Sigma ; V \vdash e \Downarrow v_0 ; \cdot \quad \Sigma ; [\cdot \mid \cdot] ; V, x = v_0, \text{setX} = \text{setter}_{c.x} ; c \vdash p \Downarrow v ; U ; V'}{\Sigma ; [\cdot \mid \cdot] ; V ; c \vdash \text{state } x, \text{setX default } e ; p \Downarrow v ; U ; V'} \\
 \text{(EFFECT INIT)} \\
 \frac{\Sigma ; [\cdot \mid \cdot] ; V ; c \vdash p \Downarrow v ; U' ; V'}{\Sigma ; [\cdot \mid \cdot] ; V ; c \vdash \text{on } x_1, x_2, \dots, x_n \text{ do } \{e\} ; p \Downarrow v ; \text{on}(\langle \lambda_e, V \rangle), U' ; V'} \\
 \text{(SUBCOMP INIT)} \\
 \frac{\Sigma ; [\cdot \mid \cdot] ; \bar{x}' = v ; c.y \vdash p_A \Downarrow v_A ; U_y ; V'_y \quad \Sigma ; [\cdot \mid \cdot] ; V, y = v_A ; c \vdash p \Downarrow v ; U ; V' \quad V''_y = \text{addPrefix}("y.", V'_y) \quad \bar{x} = \bar{v} \in V \quad A : (\text{comp } A(\bar{x}') : \tau \{p_A\}, \Delta, \Gamma) \in \Sigma}{\Sigma ; [\cdot \mid \cdot] ; V ; c \vdash \text{comp } y = A(\bar{x}) ; p \Downarrow v ; U_y, U ; V' \cup V''_y}
 \end{array}$$

Fig. 21. Declaration evaluation: initialization.

D.3.2 Rerender.

These rules give the *re-render* evaluation of the same declaration forms, taken on every render after the first. They are distinguished from initialization by comparing the previous-render environment V_e against the current values: a state declaration reuses its stored value, and an effect block re-runs its body only when one of its watched variables $x_1, \dots, x_n$ has actually changed (EFFECT RERENDER, YES CHANGES), otherwise it is skipped (EFFECT RERENDER, NO CHANGES).

$$\begin{array}{c}
 \text{(STATE RERENDER)} \\
 \frac{\Sigma ; [V_e \mid V_s] ; V, x = v_0, \text{setX} = \text{setter}_{c.x} ; c \vdash p \Downarrow v ; U ; V' \quad x = v_0 \in V_s, x = v_e \in V_e}{\Sigma ; [V_e \mid V_s] ; V ; c \vdash \text{state } x, \text{setX default } e ; p \Downarrow v ; U ; V'} \\
 \text{(EFFECT RERENDER, NO CHANGES)} \\
 \frac{\Sigma ; [V_e \mid V_s] ; V ; c \vdash p \Downarrow v ; U ; V' \quad x_1 = v'_1, x_2 = v'_2, \dots, x_n = v'_n \in V \quad x_1 = v_1, x_2 = v_2, \dots, x_n = v_n \in V_e \quad \forall i \in [n], v_i = v'_i}{\Sigma ; [V_e \mid V_s] ; V ; c \vdash \text{on } x_1, x_2, \dots, x_n \text{ do } \{e\} ; p \Downarrow v ; U ; V'}
 \end{array}$$

$$\begin{array}{c}
\text{(EFFECT RERENDER, YES CHANGES)} \\
\frac{\Sigma ; [V_e \mid V_s] ; V ; c \vdash p \Downarrow v ; U' ; V' \quad \Sigma ; V \vdash e \Downarrow v_e ; U \quad x_1 = v'_1, x_2 = v'_2, \dots, x_n = v'_n \in V \quad x_1 = v_1, x_2 = v_2, \dots, x_n = v_n \in V_e \quad \exists i \in [n], v_i \neq v'_i}{\Sigma ; [V_e \mid V_s] ; V ; c \vdash \text{on } x_1, x_2, \dots, x_n \text{ do } \{e\} ; p \Downarrow v ; U, U' ; V'} \\
\text{(SUBCOMP RERENDER)} \\
\frac{\Sigma ; [V'_e \mid V'_s] ; \overline{x'} = \overline{v} ; c.y \vdash p_A \Downarrow v_A ; U_y ; V'_y \quad \Sigma ; [V_e \mid V_s] ; V, y = v_A ; c \vdash p \Downarrow v ; U ; V' \quad V'_e = \text{removePrefix}("y.", \text{startsWith}("y.", V_e)) \quad V'_s = \text{removePrefix}("y.", \text{startsWith}("y.", V_s)) \quad V''_y = \text{addPrefix}("y.", V'_y) \quad \overline{x} = \overline{v} \in V \quad A : (\text{comp } A(\overline{x'}) : \tau \{p_A\}, \Delta, \Gamma) \in \Sigma}{\Sigma ; [V_e \mid V_s] ; V ; c \vdash \text{comp } y = A(\overline{x}) ; p \Downarrow v ; U_y, U ; V' \cup V''_y}
\end{array}$$

Fig. 22. Declaration evaluation: rerender.

D.4 Expression

The expression evaluation judgement $\Sigma ; V \vdash e \Downarrow v ; U$ evaluates expression e in environment V to a value v , collecting into the update queue U every side-effecting item the evaluation produces – setter applications, cancellations, removals, and the listener registrations emitted by bind and once. It is a standard call-by-value big-step semantics; the queue U is the operational counterpart of the effect F assigned to e by expression typing.

$$\begin{array}{c}
\text{(VAR)} \quad \frac{}{\Sigma ; V, x = v \vdash x \Downarrow v ; \cdot} \quad \text{(CONSTANT)} \quad \frac{c \in \alpha \cup \{\text{true}, \text{false}\} \cup \{()\}}{\Sigma ; V \vdash c \Downarrow c ; \cdot} \\
\text{(SETTER APP)} \quad \frac{\Sigma ; V \vdash e_1 \Downarrow \text{setter}_x ; U_1 \quad \Sigma ; V \vdash e_2 \Downarrow \langle \lambda y. e_3, V' \rangle ; U_2}{\Sigma ; V \vdash e_1 e_2 \Downarrow () ; U_1, U_2, \text{setter}_x(\langle \lambda y. e_3, V' \rangle)} \\
\text{(FUNCTION APP)} \quad \frac{\Sigma ; V \vdash e_1 \Downarrow \langle \lambda x. e_3, V' \rangle ; U_1 \quad \Sigma ; V \vdash e_2 \Downarrow v_2 ; U_2 \quad \Sigma ; V', x : v_2 \vdash e_3 \Downarrow v_3 ; U_3}{\Sigma ; V \vdash e_1 e_2 \Downarrow v_3 ; U_1, U_2, U_3} \\
\text{(CLOSURE CREATE)} \quad \frac{}{\Sigma ; V \vdash \lambda x. e \Downarrow \langle \lambda x. e, V \rangle ; \cdot} \quad \text{(SEQ)} \quad \frac{\Sigma ; V \vdash e_1 \Downarrow v_1 ; U_1 \quad \Sigma ; V \vdash e_2 \Downarrow v_2 ; U_2}{\Sigma ; V \vdash e_1 ; e_2 \Downarrow v_2 ; U_1, U_2} \\
\text{(BRANCH1)} \quad \frac{\Sigma ; V \vdash e_1 \Downarrow \text{true} ; U_1 \quad \Sigma ; V \vdash e_2 \Downarrow v_2 ; U_2}{\Sigma ; V \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow v_2 ; U_1, U_2} \\
\text{(BRANCH2)} \quad \frac{\Sigma ; V \vdash e_1 \Downarrow \text{false} ; U_1 \quad \Sigma ; V \vdash e_3 \Downarrow v_3 ; U_3}{\Sigma ; V \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow v_3 ; U_1, U_3} \\
\text{(PROD)} \quad \frac{\Sigma ; V \vdash e_1 \Downarrow v_1 ; U_1 \quad \Sigma ; V \vdash e_2 \Downarrow v_2 ; U_2}{\Sigma ; V \vdash (e_1, e_2) \Downarrow (v_1, v_2) ; U_1, U_2} \quad \text{(FST)} \quad \frac{\Sigma ; V \vdash e \Downarrow (v_1, v_2) ; U}{\Sigma ; V \vdash \text{fst } e \Downarrow v_1 ; U}
\end{array}$$

$$\begin{array}{c}
\text{(SND)} \\
\frac{\Sigma ; V \vdash e \Downarrow (v_1, v_2) ; U}{\Sigma ; V \vdash \text{snd } e \Downarrow v_2 ; U} \\
\text{(REMOVE)} \\
\frac{\Sigma ; V \vdash \text{cancel } \ell(\bar{v}) \Downarrow () ; \text{cancel}(\ell(\bar{v}))}{\Sigma ; V \vdash \text{remove } \ell(\bar{v}) \Downarrow () ; \text{remove}(\ell(\bar{v}))} \\
\text{(BIND)} \\
\frac{\Sigma ; V \vdash e \Downarrow (\lambda x.e', V') ; U \quad \Sigma \models (\lambda x.e', V') : \tau \rightarrow \text{unit} \mid F}{\Sigma ; V \vdash \text{bind } \ell(\bar{v}_1) e \Downarrow () ; U, \text{listen}(\ell(\bar{v}_1), (\lambda x.e', V'), \square, F)} \\
\text{(ONCE)} \\
\frac{\Sigma ; V \vdash e \Downarrow (\lambda x.e', V') ; U \quad \Sigma \models (\lambda x.e', V') : \tau \rightarrow \text{unit} \mid F}{\Sigma ; V \vdash \text{once } \ell(\bar{v}_1) e \Downarrow () ; U, \text{listen}(\ell(\bar{v}_1), (\lambda x.e', V'), \diamond, F)}
\end{array}$$

Fig. 23. Expression evaluation.

E Instrumented Semantics

E.1 Declarations

The instrumented declaration judgement $\Gamma ; X ; V_s ; V ; c \vdash_\Sigma p \Downarrow v ; T ; V'$ mirrors the ordinary declaration semantics but additionally records a *trace* T — a structured log of the render with one entry per declaration (*LET*, *EFF*, *RET*, and subcomponent *ARGs*) — and is parameterized by the set X of variables that changed this render. The trace is the object that the metatheory’s *justification* judgements later check against Δ ; it is instrumented bookkeeping only and is erased in ordinary execution. These two rules cover the *VAR* (let-bound) and *RETURN* declarations.

$$\begin{array}{c}
\text{(VAR)} \\
\frac{\Sigma ; V \vdash e \Downarrow v_0 ; \cdot \quad \Gamma ; X ; V_s ; V, x = v_0 ; c \vdash_\Sigma p \Downarrow v ; U' ; V'}{\Gamma ; X ; V_s ; V ; c \vdash_\Sigma \text{let } x = e ; p \Downarrow v ; \text{LET}(x) \leftarrow \{df(e)\}, U ; V'} \\
\text{(RETURN)} \\
\frac{x = v_s \in V_s \quad x = v \in V \quad \Sigma ; \Gamma \models_t V}{\Gamma ; X ; V_s ; V ; c \vdash_\Sigma \text{return } x \Downarrow v ; \text{RET } x ; V}
\end{array}$$

Fig. 24. Instrumented declaration evaluation: *VAR* and *RETURN*.

E.1.1 Rerender.

These are the re-render rules of the instrumented semantics, the trace-producing counterparts of the re-render rules of the ordinary semantics. An effect block that fires (its watched set meets the changed set X) records an *EFF* entry carrying the update queue it produced (*EFFECT RERENDER*, *YES CHANGES*); one that does not fire records nothing new (*EFFECT RERENDER*, *NO CHANGES*), and subcomponents recurse with the appropriately re-prefixed changed set.

$$\begin{array}{c}
\text{(STATE RERENDER)} \\
\frac{\Gamma ; X ; V_s ; V, x = v, \text{setX} = \text{setter}_{c,x} ; c \vdash_{\Sigma} p \Downarrow v ; T ; V' \quad x : \tau \in \Gamma \quad x = v \in V_s \quad x = \text{setter}_{c,x} \in V_s \quad \Sigma \models v : \tau}{\Gamma ; X ; V_s ; V ; c \vdash_{\Sigma} \text{state } x, \text{setX default } e ; p \Downarrow v ; T ; V'} \\
\text{(EFFECT RERENDER, NO CHANGES)} \\
\frac{\Gamma ; X ; V_s ; V ; c \vdash_{\Sigma} p \Downarrow v ; T ; V' \quad \bar{x} \cap X = \emptyset}{\Gamma ; X ; V_s ; V ; c \vdash_{\Sigma} \text{on } \bar{x} \text{ do } \{e\} ; p \Downarrow v ; T ; V'} \\
\text{(EFFECT RERENDER, YES CHANGES)} \\
\frac{\Gamma ; X ; V_s ; V ; c \vdash_{\Sigma} p \Downarrow v ; T ; V' \quad \Sigma ; V \vdash e \Downarrow () ; U \quad \bar{x} \cap X \neq \emptyset \quad \exists \Gamma'. (\Gamma' \subseteq \Gamma, \Sigma ; \Gamma' \models_t V)}{\Gamma ; X ; V_s ; V ; c \vdash_{\Sigma} \text{on } \bar{x} \text{ do } \{e\} ; p \Downarrow v ; \text{EFF}(\bar{x} \cap X) \rightarrow \{U\} ; T ; V'} \\
\text{(SUBCOMP RERENDER)} \\
\frac{\begin{array}{c} \Gamma_{sF} ; X' ; V'_s ; x' = v ; c.y \vdash_{\Sigma} p_F \Downarrow v_F ; T_y, \text{RET } z ; V'_y \\ \Gamma_s ; X ; V_s ; V, y = v_F ; c \vdash_{\Sigma} p \Downarrow v ; T ; V' \\ \bar{x} = \bar{v} \in V \quad F : (\text{comp } F(\bar{x}') : \tau_{rF} \{p_F\}, \Delta_F, \Gamma_{sF}) \in \Sigma \\ X' = \text{removePrefix}("y.", \text{startsWith}("y.", X)) \\ V'_s = \text{removePrefix}("y.", \text{startsWith}("y.", V_s)) \quad V''_y = \text{addPrefix}("y.", V'_y) \end{array}}{\Gamma ; X ; V_s ; V ; c \vdash_{\Sigma} \text{comp } y = A(\bar{x}) ; p \Downarrow v ; T_y, T ; V' \cup V''_y}
\end{array}$$

Fig. 25. Instrumented declaration evaluation: rerender.

F Preservation

F.1 Values and Types

These rules define when a runtime value is well-typed. The value-typing judgement $\Sigma \models v : \tau$ assigns a closed value v its type τ (closures also carry the latent effect F of their body), and the value-environment judgement $\Sigma ; \Gamma \models_t V$ lifts this pointwise, holding when every binding in the runtime environment V matches its declared type in Γ . These are the semantic typing relations used throughout the preservation proof.

$$\begin{array}{c}
\begin{array}{cccc}
\text{(VAL-UNIT)} & \text{(VAL-TRUE)} & \text{(VAL-FALSE)} & \text{(VAL-CONST)} \\
\hline
\Sigma \models () : \text{unit} & \Sigma \models \text{true} : \text{bool} & \Sigma \models \text{false} : \text{bool} & \Sigma \models c : \alpha
\end{array} \\
\begin{array}{cc}
\text{(VAL-PAIR)} & \text{(VAL-CLOSURE)} \\
\hline
\Sigma \models v_1 : \tau_1 \quad \Sigma \models v_2 : \tau_2 & \exists \Gamma \quad \Sigma ; \Gamma \models_t V \quad \Gamma, x : \tau_1 \vdash e : \tau_2 \mid F \\
\hline
\Sigma \models (v_1, v_2) : \tau_1 \times \tau_2 & \Sigma \models (\lambda x. e, V) : (\tau_1 \rightarrow \tau_2 \mid F)
\end{array} \\
\text{(VAL-SETTER)} \\
\frac{\text{Main} : (\text{comp Main}, \Delta, \Gamma) \in \Sigma \quad x : \tau \in \Gamma}{\Sigma \models \text{setter}_x : ((\tau \rightarrow \tau \mid \cdot) \rightarrow \text{unit} \mid \circ^{1r} \otimes x)}
\end{array}$$

Fig. 26. Value typing.

$$\begin{array}{c}
\text{(VALENV-EMPTY)} \\
\hline
\Sigma ; \cdot \vdash_t \cdot
\end{array}
\quad
\begin{array}{c}
\text{(VALENV-CONS)} \\
\hline
\Sigma ; \Gamma \vdash_t V \quad \Sigma \vdash v : \tau \\
\hline
\Sigma ; \Gamma, x : \tau \vdash_t V, x = v
\end{array}$$

Fig. 27. Value environment typing.

F.2 Side Conditions

F.2.1 Typing Update Queues.

We first give the precise effect of a single queue item, one rule per kind of event the queue may carry, then assemble the effect of a whole queue with TQ NIL and TQ CONS. Subtyping (including *-splitting and +-branching across the queue) is handled by the rules in *Derived from Subtyping Relations* below.

Single events.

$$\begin{array}{c}
\text{(TQ SETTER)} \quad \text{(TQ EVENT)} \quad \text{(TQ CANCEL)} \quad \text{(TQ REMOVE)} \\
\hline
\begin{array}{c}
\circ^{1r}_{@x} \vdash_e \text{setter}_x(f) \quad \ell(\bar{v}_1) \vdash_e (\ell(\bar{v}_1), v_2) \quad \emptyset \ell(\bar{v}) \vdash_e \text{cancel}(\ell(\bar{v})) \quad \mathbf{x} \ell(\bar{v}) \vdash_e \text{remove}(\ell(\bar{v})) \\
\text{(TQ LISTEN ONCE)} \quad \text{(TQ LISTEN ALWAYS)} \\
\Sigma \vdash c : \tau \rightarrow \text{unit} \mid F \quad \tau = \Sigma_E(\ell(\bar{v})) \quad \Sigma \vdash c : \tau \rightarrow \text{unit} \mid F \quad \tau = \Sigma_E(\ell(\bar{v})) \\
\hline
\diamond^{\ell(\bar{v})}(F) \vdash_e \text{listen}(\ell(\bar{v}), c, \diamond, F) \quad \square^{\ell(\bar{v})}(F) \vdash_e \text{listen}(\ell(\bar{v}), c, \square, F)
\end{array}
\end{array}$$

Fig. 28. Queue-item typing: single events.

Queue assembly.

$$\begin{array}{c}
\text{(TQ NIL)} \quad \text{(TQ CONS)} \\
\hline
\cdot \vdash_e \cdot \quad \frac{F_1 \vdash_e e \quad F_2 \vdash_e U}{F_1 * F_2 \vdash_e e, U}
\end{array}$$

Fig. 29. Queue-item typing: queue assembly.

F.2.2 Derived from Subtyping Relations.

These two rules extend queue-item typing to whole queues using the sub-effecting structure of effects. TQ-MERGE types a concatenated queue with the product of its parts' effects, and TQ-PICK lets a queue that realises one branch of a + effect be typed at the whole choice. Together with the base rules above, they make $F \vdash_e U$ closed under the * and + connectives, mirroring sub-effecting on the effect side.

$$\begin{array}{c}
\text{(TQ-MERGE)} \quad \text{(TQ-PICK)} \\
\hline
\frac{F_1 \vdash_e U_1 \quad F_2 \vdash_e U_2}{F_1 * F_2 \vdash_e U_1, U_2} \quad \frac{F_i \vdash_e U \quad i \in \{1, 2\}}{F_1 + F_2 \vdash_e U}
\end{array}$$

Fig. 30. Queue typing derived from subtyping.

Lemma (Listen typing inversion). If $F' \vdash_e U$ and $\text{listen}(\ell(\bar{v}), c, m, F) \in U$, then $\Sigma \vdash c : \Sigma_E(\ell(\bar{v})) \rightarrow \text{unit} \mid F$.

Proof. By induction on the derivation of $F' \models_e U$.

- TQ NIL: $U = \cdot$ contains no items; vacuous.
- TQ EVENT, TQ SETTER, TQ CANCEL, TQ REMOVE: U is a single non-listen item; vacuous.
- TQ LISTEN ONCE / TQ LISTEN ALWAYS: $U = \text{listen}(\ell(\bar{v}), c, m, F)$. The rule's premises include $\Sigma \models c : \tau \rightarrow \text{unit} \mid F$ together with $\tau = \Sigma_E(\ell(\bar{v}))$, which is the goal.
- TQ CONS: $U = e, U''$ derived from $F_1 \models_e e$ and $F_2 \models_e U''$ with $F' = F_1 * F_2$. The listen item is either e (apply the IH to the smaller derivation $F_1 \models_e e$) or in U'' (apply the IH to $F_2 \models_e U''$).
- Subtyping rules ($*$ -merge, $+$ -pick): structurally smaller $F'' \models_e U$ premises witness the same items; apply the IH.

□

F.2.3 Justifying Traces.

The justification judgement $(\Delta, X) \succ T$ reads “the effect environment Δ , together with the changed-variable set X , justifies the trace T ”. It is the central soundness relation of the metatheory: it walks the trace produced by a render and checks that every recorded effect (each *EFF*, *LET*, *RET*, external firing, and success continuation) was permitted by Δ — that is, caused by a variable that actually changed — so that a well-typed program only ever fires effects its declared dependencies allow.

$$\begin{array}{c}
 \text{(JUSTIFY-ARGS)} \quad \frac{(\Delta, X) \succ T}{(\Delta, X) \succ \text{ARG } \bar{x} = \bar{v}_1 \rightarrow \bar{v}_2, T} \quad \text{(JUSTIFY-RETURN)} \quad \frac{\Delta \vdash @x \Rightarrow @\text{return}}{(\Delta, X) \succ \text{RET } x} \\
 \\
 \text{(JUSTIFY-EXT)} \quad \frac{\forall i. R_i \in \{ \text{FIRE}(\ell_i(\bar{v}_{1i}), v_{2i}), \text{FIRE-CXL}(\ell_i(\bar{v}_{1i}), v_{2i}), \text{FIRE-SUC}(\ell_i(\bar{v}_{1i}), v_{2i}) \rightarrow \{ (U_{i1}, F_{i1}), \dots, (U_{i,k_i}, F_{i,k_i}) \} \} \\
 \quad \forall i \text{ with } R_i = \text{FIRE-SUC}(\ell_i(\bar{v}_{1i}), v_{2i}) \rightarrow \{ (U_{i1}, F_{i1}), \dots, (U_{i,k_i}, F_{i,k_i}) \}. \forall j. (\\
 \quad F_{ij} \models_e U_{ij} \quad \exists c_{ij}. \Sigma \models c_{ij} : \Sigma_E(\ell_i(\bar{v}_{1i})) \rightarrow \text{unit} \mid F_{ij}) \quad (\Delta, X) \succ T}{(\Delta, X) \succ \text{EXT}(\ell_1(\bar{v}_{11}), v_{21}), (\ell_2(\bar{v}_{12}), v_{22}), \dots, (\ell_n(\bar{v}_{1n}), v_{2n}), R_1, R_2, \dots, R_n, T} \\
 \\
 \text{(JUSTIFY-LET)} \quad \frac{\Delta \vdash @x_i \Rightarrow @y \quad (\Delta, X) \succ T}{(\Delta, X) \succ \text{LET}(y) \leftarrow \{ \bar{x} \}, T} \quad \text{(JUSTIFY-ON)} \quad \frac{\bar{x} \subseteq X \quad \exists F. (\Delta \vdash @x_i \Rightarrow F \text{ and } F \models_e U) \quad (\Delta, X) \succ T}{(\Delta, X) \succ \text{EFF}(\bar{x}) \rightarrow \{ U \}, T}
 \end{array}$$

Fig. 31. Justifying traces.

F.2.4 Justifying Variable Change Sets.

The change-set judgement $(\Delta, X_1) \succ T \succ X_2$ computes, from a trace T evaluated under the incoming changed set X_1 , the set X_2 of variables whose state was actually changed during that render. It reads the setter applications out of the trace's *EFF* and *FIRE-SUC* entries (and the changed arguments out of an *ARG* entry); the resulting X_2 becomes the changed set against which the next render's justification is checked.

$$\begin{array}{c}
 \text{(JVS-EMPTY)} \quad \frac{}{(\Delta, X_1) \succ \cdot \succ \emptyset} \quad \text{(JVS-RETURN)} \quad \frac{(\Delta, X_1) \succ T \succ X_2}{(\Delta, X_1) \succ T, \text{RET } x \succ X_2} \\
 \text{(JVS-ON)} \quad \frac{(\Delta, X_1) \succ T \succ X_2}{(\Delta, X_1) \succ T, \text{EFF}(\bar{x}) \rightarrow \{ U \} \succ X_2 \cup \{ y \mid \text{setter}_y(f) \in U \}}
 \end{array}$$

$$\begin{array}{c}
\text{(JVS-ARGS)} \\
\hline
(\Delta, X_1) \succ ARG \bar{x} = \bar{v}_1 \rightarrow \bar{v}_2 \succ \{x_i \mid v_{1i} \neq v_{2i}\} \\
\text{(JVS-EXT)} \\
\hline
(\Delta, X_1) \succ T \succ X_2 \\
\hline
(\Delta, X_1) \succ T, EXT(\ell_1 \langle \bar{v}_{11} \rangle, v_{21}), \dots, (\ell_n \langle \bar{v}_{1n} \rangle, v_{2n}) \succ X_2 \\
\text{(JVS-FIRECXI)} \\
\hline
(\Delta, X_1) \succ T \succ X_2 \\
\hline
(\Delta, X_1) \succ T, FIRE-CXL(\ell \langle \bar{v}_1 \rangle, v_2) \succ X_2 \\
\text{(JVS-FIRESUC)} \\
\hline
(\Delta, X_1) \succ T \succ X_2 \\
\hline
(\Delta, X_1) \succ T, FIRE-SUC(\ell \langle \bar{v}_1 \rangle, v_2) \rightarrow \{(U_1, F_1), \dots, (U_k, F_k)\} \succ X_2 \cup \{y \mid \exists j. setter_y(f) \in U_j\}
\end{array}$$

Fig. 32. Justifying variable change sets.

F.2.5 Flush Queue.

The flush-queue judgment threads the component state $(X, V_s, \mathcal{L}, \mathcal{C})$ through an update queue U , applying each item to the appropriate carrier: setters update V_s and X , cancels extend $\mathcal{C}$, removes reset listener buckets in $\mathcal{L}$, and listener registrations extend $\mathcal{L}$. The judgment is defined only on listener-safe queues ($\vdash U$ lsafe from LSAFE): the four rules below partition the item constructors that the listener-safe judgement admits, so a derivation exists for U iff U is lsafe. Labelled events and effect-block evaluations are ruled out by construction — they are dispatched at the component level (via POP EVENT), not flushed here.

$$\begin{array}{c}
\text{(FQ NIL)} \\
\hline
(X, V_s, \mathcal{L}, \mathcal{C}) \Rightarrow \cdot \Rightarrow (X, V_s, \mathcal{L}, \mathcal{C}) \\
\\
\text{(FQ SETTER)} \\
\frac{V_{s1} \Rightarrow setter_x(f) \Rightarrow V_{s2} \quad (X_1 \cup \{x\}, V_{s2}, \mathcal{L}, \mathcal{C}) \Rightarrow U \Rightarrow (X_3, V_{s3}, \mathcal{L}', \mathcal{C}')}{(X_1, V_{s1}, \mathcal{L}, \mathcal{C}) \Rightarrow setter_x(f), U \Rightarrow (X_3, V_{s3}, \mathcal{L}', \mathcal{C}')} \\
\\
\text{(FQ CANCEL)} \\
\frac{(X_1, V_s, \mathcal{L}, \mathcal{C} \uplus \{\ell \langle \bar{v} \rangle\}) \Rightarrow U \Rightarrow (X_2, V'_s, \mathcal{L}', \mathcal{C}')}{(X_1, V_s, \mathcal{L}, \mathcal{C}) \Rightarrow cancel(\ell \langle \bar{v} \rangle), U \Rightarrow (X_2, V'_s, \mathcal{L}', \mathcal{C}')} \\
\\
\text{(FQ REMOVE)} \\
\frac{\mathcal{L}_1 = \mathcal{L}[\ell \langle \bar{v} \rangle \mapsto \emptyset] \quad (X_1, V_s, \mathcal{L}_1, \mathcal{C}) \Rightarrow U \Rightarrow (X_2, V'_s, \mathcal{L}', \mathcal{C}')}{(X_1, V_s, \mathcal{L}, \mathcal{C}) \Rightarrow remove(\ell \langle \bar{v} \rangle), U \Rightarrow (X_2, V'_s, \mathcal{L}', \mathcal{C}')} \\
\\
\text{(FQ LISTEN)} \\
\frac{m \in \{\Diamond, \Box\} \quad \tau = \Sigma_E(\ell \langle \bar{v} \rangle) \quad \Sigma \models c : \tau \rightarrow \text{unit} \mid F \quad \mathcal{L}_1 = \mathcal{L}[\ell \langle \bar{v} \rangle \mapsto \mathcal{L}(\ell \langle \bar{v} \rangle) \cup \{(c, m, F)\}]}{(X_1, V_s, \mathcal{L}, \mathcal{C}) \Rightarrow listen(\ell \langle \bar{v} \rangle, c, m, F), U \Rightarrow (X_2, V'_s, \mathcal{L}', \mathcal{C}')}
\end{array}$$

Fig. 33. Flush-queue judgment.

F.3 Instrumented Well Typed Configuration

A configuration $\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{status} \mid T_1 \mid T_2 \rangle$ is a whole-program runtime state: the component A with its current arguments, the state tuple $\mathcal{S} = (X_1, X_2, V_s, \mathcal{L}, \mathcal{C})$, a status of waiting or rendered, and the render's trace split into an already-processed prefix T_1 and a not-yet-processed suffix T_2 . WT-CONFIG defines when such a configuration is *well-typed*: the component checks, T_1 and T_2 are jointly justified by Δ , T_1 produces the recorded change set X_2 , and the store V_s together with every registered listener in $\mathcal{L}$ is well-typed. The remaining rules (INIT through WAITING TO RENDERED) are the configuration step relation $\rightarrow$, which drains the trace one entry at a time — flushing updates and lets, dispatching or cancelling external event firings, and re-rendering when arguments change. Preservation (Theorem F.1) shows this relation preserves well-typedness.

(WT-CONFIG)

$$\frac{\begin{array}{c} \mathcal{S} = (X_1, X_2, V_s, \mathcal{L}, \mathcal{C}) \\ \Sigma \Rightarrow \text{comp } A(\overline{x_i : \tau_i}) : \tau_r \{p\} \Rightarrow (\Delta, \Gamma_s) \\ (\Delta, X_1) \succ T_1, T_2 \quad (\Delta, X_1) \succ T_1 \succ X_2 \quad \Sigma ; \Gamma_s \models_t V_s \\ \overline{\Sigma \models v : \tau} \quad \forall \ell(\overline{v_1}) \in \text{dom}(\mathcal{L}), \forall (c, m, F) \in \mathcal{L}(\ell(\overline{v_1})). \Sigma \models c : \Sigma_E(\ell(\overline{v_1})) \rightarrow \text{unit} \mid F \end{array}}{\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{status} \mid T_1 \mid T_2 \rangle}$$

Fig. 34. Instrumented well-typed configuration.

(INIT)

$$\frac{A : (\text{comp } A(\overline{x : \tau}) : \tau_r \{p\}, \Delta, \Gamma_s) \in \Sigma \quad \Gamma_s ; \emptyset ; \cdot ; \overline{x = v} ; \text{"} \vdash_{\Sigma} p \Downarrow v_r ; T ; V}{\cdot \rightarrow \langle \Sigma; \Delta; \Gamma_s \parallel (\emptyset, \emptyset, V, \emptyset, \emptyset) \mid A \mid \overline{x = v} \mid \text{rendered} \mid \cdot \mid T \rangle}$$

(NO UPDATES)

$$\frac{\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1 \mid \text{RET } y \rangle}{\rightarrow \langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{waiting} \mid T_1, \text{RET } y \mid \cdot \rangle}$$

(FLUSH UPDATE)

$$\frac{\begin{array}{c} \mathcal{S} = (X_1, X_{21}, V_{s1}, \mathcal{L}_1, \mathcal{C}_1) \quad \mathcal{S}' = (X_1, X_{22}, V_{s2}, \mathcal{L}_2, \mathcal{C}_2) \\ \vdash U \text{ lsafe} \quad (X_{21}, V_{s1}, \mathcal{L}_1, \mathcal{C}_1) \Rightarrow U \Rightarrow (X_{22}, V_{s2}, \mathcal{L}_2, \mathcal{C}_2) \end{array}}{\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1 \mid \text{EFF}(\overline{y}) \rightarrow \{U\}, T_2 \rangle \rightarrow \langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S}' \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1, \text{EFF}(\overline{y}) \rightarrow \{U\} \mid T_2 \rangle}$$

(FLUSH LET)

$$\frac{\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1 \mid \text{LET}(y) \leftarrow \{\overline{z}\}, T_2 \rangle}{\rightarrow \langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1, \text{LET}(y) \leftarrow \{\overline{z}\} \mid T_2 \rangle}$$

(RECEIVE EXT)

$$\begin{array}{c}
\mathcal{S} = (X_1, X_2, V_s, \mathcal{L}, \mathcal{C}) \\
\hline
\frac{\mathcal{S}' = (X_1, X_2, V_s, \mathcal{L}, \mathcal{C}) \quad \mathcal{C} \Rightarrow_{\text{ext}} (\ell_1 \langle \overline{v_{11}} \rangle, v_{21}), (\ell_2 \langle \overline{v_{12}} \rangle, v_{22}), \dots, (\ell_n \langle \overline{v_{1n}} \rangle, v_{2n})}{\Sigma \models v_{2i} : \Sigma_E(\ell_i \langle \overline{v_{1i}} \rangle)} \quad T_{\text{fire}} = \text{FIRE}(\ell_1 \langle \overline{v_{11}} \rangle, v_{21}), \text{FIRE}(\ell_2 \langle \overline{v_{12}} \rangle, v_{22}), \dots, \text{FIRE}(\ell_n \langle \overline{v_{1n}} \rangle, v_{2n}) \\
T_{\text{ext}} = \text{EXT}(\ell_1 \langle \overline{v_{11}} \rangle, v_{21}), (\ell_2 \langle \overline{v_{12}} \rangle, v_{22}), \dots, (\ell_n \langle \overline{v_{1n}} \rangle, v_{2n}) \\
\hline
\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{rendered} \mid \text{ARG } \overline{x} = \overline{v'_1} \rightarrow \overline{v'_2}, T_2 \rangle \\
\rightarrow \\
\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S}' \mid A \mid \overline{x = v} \mid \text{rendered} \mid \text{ARG } \overline{x} = \overline{v'_1} \rightarrow \overline{v'_2}, T_{\text{ext}} \mid T_{\text{fire}}, T_2 \rangle
\end{array}$$

Fig. 35. Configuration stepping: initialization and flushing.

(FIRE CANCEL)

$$\begin{array}{c}
\mathcal{S} = (X_1, X_2, V_s, \mathcal{L}, \mathcal{C}) \quad \mathcal{S}' = (X_1, X_2, V_s, \mathcal{L}, \mathcal{C}') \quad \mathcal{C}(\ell \langle \overline{v_1} \rangle) \geq 1 \quad \mathcal{C}' = \mathcal{C} \setminus \{ \ell \langle \overline{v_1} \rangle \} \\
\hline
\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1 \mid \text{FIRE}(\ell \langle \overline{v_1} \rangle, v_2), T_2 \rangle \\
\rightarrow \\
\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S}' \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1, \text{FIRE-CXL}(\ell \langle \overline{v_1} \rangle, v_2) \mid T_2 \rangle
\end{array}$$

(FIRE SUCCESSFUL)

$$\begin{array}{c}
\mathcal{S} = (X_1, X_2, V_s, \mathcal{L}, \mathcal{C}) \quad \mathcal{S}' = (X_1, X'_2, V'_s, \mathcal{L}'', \mathcal{C}') \quad \mathcal{C}(\ell \langle \overline{v_1} \rangle) = 0 \\
\mathcal{L}(\ell \langle \overline{v_1} \rangle) = \{ (\langle \lambda x_1. e_1, V_1 \rangle, m_1, F_1), \dots, (\langle \lambda x_k. e_k, V_k \rangle, m_k, F_k) \} \\
\hline
\overline{m_i \in \{ \diamond, \square \}} \quad \Sigma \models v_2 : \Sigma_E(\ell \langle \overline{v_1} \rangle) \quad \Sigma; V_i, x_i = v_2 \vdash e_i \Downarrow (); U_i \vdash U_i \text{ lsafe} \quad \overline{F_i \models_e U_i} \\
\mathcal{L}' = \mathcal{L}[\ell \langle \overline{v_1} \rangle \mapsto \{ (\langle \lambda x_i. e_i, V_i \rangle, m_i, F_i) \mid m_i = \square \}] \\
(X_2, V_s, \mathcal{L}', \mathcal{C}) \Rightarrow U_1, \dots, U_k \Rightarrow (X'_2, V'_s, \mathcal{L}'', \mathcal{C}') \\
\hline
\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1 \mid \text{FIRE}(\ell \langle \overline{v_1} \rangle, v_2), T_2 \rangle \\
\rightarrow \\
\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S}' \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1, \text{FIRE-SUC}(\ell \langle \overline{v_1} \rangle, v_2) \rightarrow \{ (U_1, F_1), \dots, (U_k, F_k) \} \mid T_2 \rangle
\end{array}$$

Fig. 36. Configuration stepping: event firing.

(WAITING TO RENDERED)

$$\begin{array}{c}
A : (\text{comp } A(\overline{x : \tau}) : \tau_r \{ p \}, \Delta, \Gamma_s) \in \Sigma \\
\mathcal{S} = (X_1, X_2, V_{s1}, \mathcal{L}, \mathcal{C}) \quad \mathcal{S}' = (X_2 \cup \{ x_i \mid v_{1i} \neq v_{2i}, \{ x_i \mid v_{1i} \neq v_{2i} \}, V_{s2}, \mathcal{L}, \mathcal{C}) \\
\Gamma_s; X_2 \cup \{ x_i \mid v_{1i} \neq v_{2i} \}; V_{s1}; \overline{x = v_2}; \text{"} \vdash_{\Sigma} p \Downarrow v_r; T_2; V_{s2} \\
\exists i \text{ s.t. } v_{1i} \neq v_{2i} \quad \Sigma \models v_r : \tau_r \quad \Sigma \models v_2 : \tau_2 \\
\hline
\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v_1} \mid \text{waiting} \mid T_1 \mid \cdot \rangle \\
\rightarrow \\
\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S}' \mid A \mid \overline{x = v_2} \mid \text{rendered} \mid \text{ARG } \overline{x} = \overline{v_1} \rightarrow \overline{v_2} \mid T_2 \rangle
\end{array}$$

Fig. 37. Configuration stepping: re-rendering.

F.4 Configurations

Theorem F.1 (Preservation for Configurations). *Given a well-typed configuration*

$$\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{status}_1 \mid T_1 \mid T_2 \rangle$$

where $\mathcal{S} = (X_1, X_2, V_{s1}, \mathcal{L}, \mathcal{C})$

- $\Sigma \Rightarrow \text{comp } A \Rightarrow (\Delta, \Gamma_s)$
- $(\Delta, X_1) \succ T_1, T_2$
- $(\Delta, X_1) \succ T_1 \succ X_2$
- $\Sigma; \Gamma_s \models_t V_{s1}$
- $\overline{\Sigma} \models v : \tau$
- For every $\ell(\overline{v_1}) \in \text{dom}(\mathcal{L})$ and every $(c, m, F) \in \mathcal{L}(\ell(\overline{v_1}))$, $\Sigma \models c : \Sigma_E(\ell(\overline{v_1})) \rightarrow \text{unit} \mid F$.
- For every firing $(\ell(\overline{v_1}), v_2) \in \mathcal{E}$ supplied by the external scheduler, $\Sigma \models v_2 : \Sigma_E(\ell(\overline{v_1}))$.
- For every FIRE $(\ell(\overline{v_1}), v_2)$ record appearing in T_2 , $\Sigma \models v_2 : \Sigma_E(\ell(\overline{v_1}))$.

If the configuration makes a step

$$\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{status}_1 \mid T_1 \mid T_2 \rangle$$

$\rightarrow$

$$\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S}' \mid A \mid \overline{x = v'} \mid \text{status}_2 \mid T_3 \mid T_4 \rangle$$

where $\mathcal{S}' = (X_3, X_4, V_{s2}, \mathcal{L}', \mathcal{C}')$

Then the resulting configuration is also well-typed:

- $(\Delta, X_3) \succ T_3, T_4$
- $(\Delta, X_3) \succ T_3 \succ X_4$
- $\Sigma; \Gamma_s \models_t V_{s2}$
- $\overline{\Sigma} \models v' : \tau$
- For every $\ell(\overline{v_1}) \in \text{dom}(\mathcal{L}')$ and every $(c, m, F) \in \mathcal{L}'(\ell(\overline{v_1}))$, $\Sigma \models c : \Sigma_E(\ell(\overline{v_1})) \rightarrow \text{unit} \mid F$.
- For every FIRE $(\ell(\overline{v_1}), v_2)$ record appearing in T_4 , $\Sigma \models v_2 : \Sigma_E(\ell(\overline{v_1}))$.

PROOF. By case analysis on the configuration transition rule.

• **Case No Updates.**

$$\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1 \mid \text{RET } y \rangle$$

$\rightarrow$

$$\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{waiting} \mid T_1, \text{RET } y \mid \cdot \rangle$$

From the precondition, we have:

- $(\Delta, X_1) \succ T_1, \text{RET } y$
- $(\Delta, X_1) \succ T_1 \succ X_2$
- $\Sigma; \Gamma_s \models_t V_{s1}$
- $\overline{\Sigma} \models v : \tau$

The resulting configuration has $X_3 = X_1$, $X_4 = X_2$, $V_{s2} = V_s$, $T_3 = T_1, \text{RET } y$, and $T_4 = \cdot$.

We verify:

- $(\Delta, X_1) \succ T_1, \text{RET } y, \cdot$ – This is equivalent to $(\Delta, X_1) \succ T_1, \text{RET } y$, which holds by the precondition.
- $(\Delta, X_1) \succ T_1, \text{RET } y \succ X_2$ – By inversion on the justification rules for return traces, we have $(\Delta, X_1) \succ T_1 \succ X_2$ from the precondition, and the return trace does not modify the variable set.
- $\Sigma; \Gamma_s \models_t V_{s1}$ – unchanged from precondition.

- $\overline{\Sigma} \models v : \tau$ – unchanged from precondition.
- $\mathcal{L}$ well-formed – unchanged from precondition.

• **Case Flush Let.**

$$\frac{\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1 \mid LET(y) \leftarrow \{ \bar{z} \}, T_2 \rangle}{\rightarrow} \\ \langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1, LET(y) \leftarrow \{ \bar{z} \} \mid T_2 \rangle$$

From the precondition, we have:

- $(\Delta, X_1) \succ T_1, LET(y) \leftarrow \{ \bar{z} \}, T_2$
- $(\Delta, X_1) \succ T_1 \succ X_2$
- $\Sigma; \Gamma_s \models_t V_{s1}$
- $\overline{\Sigma} \models v : \tau$

The resulting configuration has $X_3 = X_1, X_4 = X_2, V_{s2} = V_{s1}, T_3 = T_1, LET(y) \leftarrow \{ \bar{z} \}$, and $T_4 = T_2$.

We verify:

- $(\Delta, X_1) \succ T_1, LET(y) \leftarrow \{ \bar{z} \}, T_2$ – This holds directly from the precondition.
- $(\Delta, X_1) \succ T_1, LET(y) \leftarrow \{ \bar{z} \} \succ X_2$ – By the justification rule for let traces, $(\Delta, X_1) \succ T_1, LET(y) \leftarrow \{ \bar{z} \} \succ X_2$ follows from $(\Delta, X_1) \succ T_1 \succ X_2$ (precondition).
- $\Sigma; \Gamma_s \models_t V_{s1}$ – unchanged.
- $\overline{\Sigma} \models v : \tau$ – unchanged.
- $\mathcal{L}$ well-formed – unchanged.

• **Case Flush Update.**

$$\frac{\begin{array}{l} \mathcal{S} = (X_1, X_{21}, V_{s1}, \mathcal{L}_1, \mathcal{C}_1) \quad \mathcal{S}' = (X_1, X_{22}, V_{s2}, \mathcal{L}_2, \mathcal{C}_2) \\ \vdash U \text{ safe} \quad (X_{21}, V_{s1}, \mathcal{L}_1, \mathcal{C}_1) \Rightarrow U \Rightarrow (X_{22}, V_{s2}, \mathcal{L}_2, \mathcal{C}_2) \end{array}}{\langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1 \mid EFF(\bar{y}) \rightarrow \{ U \}, T_2 \rangle} \\ \rightarrow \\ \langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S}' \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1, EFF(\bar{y}) \rightarrow \{ U \} \mid T_2 \rangle$$

From the precondition, we have:

- $(\Delta, X_1) \succ T_1, EFF(\bar{y}) \rightarrow \{ U \}, T_2$
- $(\Delta, X_1) \succ T_1 \succ X_{21}$
- $\Sigma; \Gamma_s \models_t V_{s1}$
- $\overline{\Sigma} \models v : \tau$

We need the following lemma:

Lemma (Flush Queue Preservation): If $\Sigma; \Gamma_s \models_t V_{s1}, \mathcal{L}_1$ is well-formed in the sense of the new well-typedness bullet – that is, for every $\ell(\bar{v}_1) \in \text{dom}(\mathcal{L}_1)$ and every $(c, m, F) \in \mathcal{L}_1(\ell(\bar{v}_1))$, $\Sigma \models c : \Sigma_E(\ell(\bar{v}_1)) \rightarrow \text{unit} \mid F$ – and $(X_1, V_{s1}, \mathcal{L}_1, \mathcal{C}_1) \Rightarrow U \Rightarrow (X_2, V_{s2}, \mathcal{L}_2, \mathcal{C}_2)$, then $\Sigma; \Gamma_s \models_t V_{s2}$ and $\mathcal{L}_2$ is well-formed in the same sense.

Proof of lemma: By induction on the derivation of $(X_1, V_{s1}, \mathcal{L}_1, \mathcal{C}_1) \Rightarrow U \Rightarrow (X_2, V_{s2}, \mathcal{L}_2, \mathcal{C}_2)$.

- Base case (FQ NIL): $U = \cdot$. Then $V_{s2} = V_{s1}$ and $\mathcal{L}_2 = \mathcal{L}_1$, so both results hold trivially.
- Inductive case (FQ SETTER): $U = \text{setter}_x(f), U'$. By the rule for setter queue derivation, we have $V_{s1} \Rightarrow \text{setter}_x(f) \Rightarrow V'_{s1}$ for some V'_{s1} , and $(X_1 \cup \{x\}, V'_{s1}, \mathcal{L}_1, \mathcal{C}_1) \Rightarrow U' \Rightarrow (X_2, V_{s2}, \mathcal{L}_2, \mathcal{C}_2)$.

From $\Sigma \models \text{setter}_x : ((\tau \rightarrow \tau \mid \cdot) \rightarrow \text{unit} \mid \circ^{1r} @x)$, we know that $f : \tau \rightarrow \tau$ where $x : \tau \in \Gamma_s$. The setter application applies f to the current value of x (of type τ), producing a new value of type τ . Thus $\Sigma; \Gamma_s \models_t V'_{s1}$. $\mathcal{L}$ is unchanged at this step, so its well-formedness carries through.

By the induction hypothesis, $\Sigma; \Gamma_s \models_t V_{s2}$ and $\mathcal{L}_2$ is well-formed.

- Inductive case (FQ CANCEL): $U = \text{cancel}(\ell(\bar{v})), U'$. V_s and $\mathcal{L}$ are unchanged at this step (only $\mathcal{C}$ gains a multiset entry), so the recursive premise still has a well-typed V_s and a well-formed $\mathcal{L}$, and the induction hypothesis yields the result.
- Inductive case (FQ REMOVE): $U = \text{remove}(\ell(\bar{v})), U'$. The rule sets $\mathcal{L}'_1 = \mathcal{L}_1[\ell(\bar{v}) \mapsto \emptyset]$. For the bucket at $\ell(\bar{v})$, well-formedness is vacuous because the bucket is empty; every other bucket is unchanged from $\mathcal{L}_1$, hence retains its well-formedness witness. V_s is unchanged. Induction hypothesis on the recursive premise $(X_1, V_s, \mathcal{L}'_1, \mathcal{C}) \Rightarrow U' \Rightarrow (X_2, V_{s2}, \mathcal{L}_2, \mathcal{C}_2)$ gives the result.
- Inductive case (FQ LISTEN): $U = \text{listen}(\ell(\bar{v}), c, m, F), U'$. The rule's own premise is $\Sigma \models c : \Sigma_E(\ell(\bar{v})) \rightarrow \text{unit} \mid F$, which is exactly the well-formedness obligation for the new entry. That premise is not free standing: at the lemma's call sites a witness for it is supplied by the *Listen typing inversion* lemma applied to the ambient $F_i \models_e U_i$ derivation (in Fire Successful this is the rule premise $\overline{F_i \models_e U_i}$ on each listener's evaluated queue). Every other listener record in $\mathcal{L}'_1 = \mathcal{L}_1[\ell(\bar{v}) \mapsto \mathcal{L}_1(\ell(\bar{v})) \cup \{(c, m, F)\}]$ was already in $\mathcal{L}_1$ and so is well-formed by the precondition. V_s is unchanged. Induction hypothesis on the recursive premise yields the result.

Applying the lemma, we get $\Sigma ; \Gamma_s \models_t V_{s2}$ and that $\mathcal{L}_2$ is well-formed.

By inversion on $(\Delta, X_1) \succ T_1, \text{EFF}(\bar{y}) \rightarrow \{U\}, T_2$:

- $\bar{y} \subseteq X_1$
- $\exists F. (\Delta \vdash @y_i \Rightarrow F \text{ and } F \models_e U)$

By the rule for justifying variable change sets with effect traces, if

$$U = \text{setter}_{x_1}(f_1), \dots, \text{setter}_{x_n}(f_n)$$

then flushing the update extends the variable set to include $\{x_1, \dots, x_n\}$. This matches $X_{22} = X_{21} \cup \{x_1, \dots, x_n\}$.

We verify:

- $(\Delta, X_1) \succ T_1, \text{EFF}(\bar{y}) \rightarrow \{U\}, T_2$ – holds by precondition.
- $(\Delta, X_1) \succ T_1, \text{EFF}(\bar{y}) \rightarrow \{U\} \succ X_{22}$ – From $(\Delta, X_1) \succ T_1 \succ X_{21}$ and the rule for effect traces, we get $(\Delta, X_1) \succ T_1, \text{EFF}(\bar{y}) \rightarrow \{U\} \succ X_{21} \cup \{x_1, \dots, x_n\} = X_{22}$.
- $\Sigma ; \Gamma_s \models_t V_{s2}$ – from lemma above.
- $\mathcal{L}_2$ is well-formed – from lemma above.
- $\Sigma \models v : \tau$ – unchanged.

• **Case waiting to rendered.**

$$\begin{array}{c}
 A : (\text{comp } A(\bar{x} : \tau) : \tau_r \{p\}, \Delta, \Gamma_s) \in \Sigma \\
 \mathcal{S} = (X_1, X_2, V_{s1}, \mathcal{L}, \mathcal{C}) \quad \mathcal{S}' = (X_2 \cup \{x_i \mid v_{1i} \neq v_{2i}\}, \{x_i \mid v_{1i} \neq v_{2i}\}, V_{s2}, \mathcal{L}, \mathcal{C}) \\
 \Gamma_s ; X_2 \cup \{x_i \mid v_{1i} \neq v_{2i}\} ; V_{s1} ; \bar{x} = \bar{v}_2 ; \text{"} \vdash_{\Sigma} p \Downarrow v_r ; T_2 ; V_{s2} \\
 \exists i \text{ s.t. } v_{1i} \neq v_{2i} \quad \Sigma \models v_r : \tau_r \quad \bar{\Sigma} \models v_2 : \tau_2 \\
 \hline
 \langle \Sigma ; \Delta ; \Gamma_s \parallel \mathcal{S} \mid A \mid \bar{x} = \bar{v}_1 \mid \text{waiting} \mid T_1 \mid \cdot \rangle \\
 \rightarrow \\
 \langle \Sigma ; \Delta ; \Gamma_s \parallel \mathcal{S}' \mid A \mid \bar{x} = \bar{v}_2 \mid \text{rendered} \mid \text{ARG } \bar{x} = \bar{v}_1 \rightarrow \bar{v}_2 \mid T_2 \rangle
 \end{array}$$

This is the re-render case. From the precondition:

- $(\Delta, X_1) \succ T_1, \cdot$
- $(\Delta, X_1) \succ T_1 \succ X_2$
- $\Sigma ; \Gamma_s \models_t V_{s1}$
- $\bar{\Sigma} \models v_1 : \tau$

From $\Sigma \Rightarrow \text{comp } A \Rightarrow (\Delta, \Gamma_s)$, by inversion:

- $\Sigma ; \Gamma ; \Delta \vdash p : \tau_r$ where Γ contains the argument bindings

The new arguments $\overline{x = v_2}$ are well-typed: $\overline{\Sigma \models v_2 : \tau}$ (given in the rule premises).

Let $X' = \{x_i \mid v_{1i} \neq v_{2i}\}$ be the set of changed arguments. Since $\exists i \text{ s.t. } v_{1i} \neq v_{2i}$, we have $X' \neq \emptyset$, so $X_2 \cup X' \neq \emptyset$ as well.

Apply Preservation for Declarations (Theorem F.2) to the semantic premise, taking the changed-variable set to be $X_2 \cup X'$ (matching the X used in the semantic derivation):

- $\Gamma_s ; X_2 \cup X' ; V_{s1} ; \overline{x = v_2} ; \text{"} \vdash_{\Sigma} p \Downarrow v_r ; T_2 ; V_{s2}$
- $\Sigma ; \Gamma ; \Delta \vdash p : \tau_r$
- $\Sigma ; \Gamma_s \models V_{s1}$ (from precondition)
- $\Sigma ; \Gamma \models_t \overline{x = v_2}$ (from well-typed new arguments)
- $X_2 \cup X' \neq \emptyset$

This gives us:

- $\Sigma \models v_r : \tau_r$
- $(\Delta, X_2 \cup X') \succ T_2$
- $\Sigma ; \Gamma_s \models_t V_{s2}$

The resulting configuration has:

- $X_3 = X_2 \cup X'$ — the union of the previous render's accumulated changed-variable set and this render's changed arguments. This is exactly the X under which the new body produced T_2 .
- $X_4 = X' = \{x_i \mid v_{1i} \neq v_{2i}\}$
- $T_3 = \text{ARG } \overline{x} = \overline{v_1} \rightarrow \overline{v_2}$
- $T_4 = T_2$

We verify:

- $(\Delta, X_2 \cup X') \succ \text{ARG } \overline{x} = \overline{v_1} \rightarrow \overline{v_2}, T_2$ — By the justification rule for argument traces (which adds an ARG record to any justified trace under the same X), this reduces to $(\Delta, X_2 \cup X') \succ T_2$, which we obtained from Declarations Preservation above.
- $(\Delta, X_2 \cup X') \succ \text{ARG } \overline{x} = \overline{v_1} \rightarrow \overline{v_2} \succ X'$ — By the justification rule for argument traces, the variable-change set produced by an ARG record is exactly $\{x_i \mid v_{1i} \neq v_{2i}\} = X'$, independent of the input X .
- $\Sigma ; \Gamma_s \models_t V_{s2}$ — from Declarations Preservation above.
- $\Sigma \models v_2 : \tau$ — from the rule premises.
- $\mathcal{L}$ well-formed — $\mathcal{L}$ is unchanged across this transition, so its well-formedness carries from the precondition.

• **Case Receive Ext.**

$$\frac{\begin{array}{l} \mathcal{S} = (X_1, X_2, V_s, \mathcal{L}, \mathcal{C}) \\ \mathcal{S}' = (X_1, X_2, V_s, \mathcal{L}, \mathcal{C}) \quad \mathcal{E} \Rightarrow_{\text{ext}} (\ell_1 \langle \overline{v_{11}} \rangle, v_{21}), \dots, (\ell_n \langle \overline{v_{1n}} \rangle, v_{2n}) \\ \overline{\Sigma \models v_{2i} : \Sigma_E(\ell_i \langle \overline{v_{1i}} \rangle)} \quad T_{\text{fire}} = \text{FIRE}(\ell_1 \langle \overline{v_{11}} \rangle, v_{21}), \dots, \text{FIRE}(\ell_n \langle \overline{v_{1n}} \rangle, v_{2n}) \\ T_{\text{ext}} = \text{EXT}(\ell_1 \langle \overline{v_{11}} \rangle, v_{21}), \dots, (\ell_n \langle \overline{v_{1n}} \rangle, v_{2n}) \end{array}}{\begin{array}{l} \langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{rendered} \mid \text{ARG } \overline{x} = \overline{v'_1} \rightarrow \overline{v'_2}, T_2 \rangle \\ \rightarrow \\ \langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S}' \mid A \mid \overline{x = v} \mid \text{rendered} \mid \text{ARG } \overline{x} = \overline{v'_1} \rightarrow \overline{v'_2}, T_{\text{ext}} \mid T_{\text{fire}}, T_2 \rangle \end{array}}$$

Let $T_1 = \text{ARG } \overline{x} = \overline{v'_1} \rightarrow \overline{v'_2}$ for brevity (the bare-subscript v_1, v_2 slots are reserved for the per-event discriminant and payload pattern). From the precondition:

- $(\Delta, X_1) \succ T_1, T_2$
- $(\Delta, X_1) \succ T_1 \succ X_2$
- $\Sigma ; \Gamma_s \models_t V_s, \Sigma \models v : \tau, \mathcal{L}$ well-formed

- Each $(\ell_i(\overline{v_{1i}}), v_{2i}) \in \mathcal{E}$ satisfies $\Sigma \models v_{2i} : \Sigma_E(\ell_i(\overline{v_{1i}}))$ – the external-scheduler well-typedness bullet.

The resulting configuration has $X_3 = X_1$, $X_4 = X_2$, and $V_s, \mathcal{L}, \mathcal{C}$ all unchanged; $T_3 = T_1, T_{ext}$, and $T_4 = T_{fire}, T_2$.

We verify:

- $(\Delta, X_1) \succ T_3, T_4$ – By the justification rule for the *EXT*-followed-by- R_i block, we can extend a justified tail with an *EXT* record followed by n records $R_1, \dots, R_n$ where each R_i is one of *FIRE*, *FIRE-CXL*, *FIRE-SUC* $\rightarrow \{ \}$. Choose each $R_i = \text{FIRE}(\ell_i(\overline{v_{1i}}), v_{2i})$; the additional witness obligation on *FIRE-SUC* $\rightarrow \{ \}$ records is vacuous because none are chosen. The tail-justification premise is $(\Delta, X_1) \succ T_2$, which follows from $(\Delta, X_1) \succ T_1, T_2$ by inversion using the justification rule for argument traces (which discharges the leading T_1).
- $(\Delta, X_1) \succ T_3 \succ X_2$ – The rules for variable-set justification on *EXT* and on *FIRE* are both the identity in the running variable set: extending the prefix by an *EXT* or by a *FIRE* preserves the set. From the precondition $(\Delta, X_1) \succ T_1 \succ X_2$, extending by one *EXT* record yields $(\Delta, X_1) \succ T_3 \succ X_2$.
- $\Sigma ; \Gamma_s \models_t V_s$ – unchanged.
- $\Sigma \models v : \tau$ – unchanged.
- $\mathcal{L}$ well-formed – $\mathcal{L}$ is unchanged.
- External scheduler payload well-typedness – the firings consumed by this step are precisely those whose payload-typing premise was discharged by the rule premise $\Sigma \models v_{2i} : \Sigma_E(\ell_i(\overline{v_{1i}}))$. Any further firings remaining in $\mathcal{E}$ continue to satisfy the precondition's well-typedness clause.

• **Case Fire Cancel.**

$$\begin{array}{c}
 \mathcal{S} = (X_1, X_2, V_s, \mathcal{L}, \mathcal{C}) \\
 \mathcal{S}' = (X_1, X_2, V_s, \mathcal{L}, \mathcal{C}') \quad \mathcal{C}(\ell(\overline{v_1})) \geq 1 \quad \mathcal{C}' = \mathcal{C} \setminus \{ \ell(\overline{v_1}) \} \\
 \hline
 \langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1 \mid \text{FIRE}(\ell(\overline{v_1}), v_2), T_2 \rangle \\
 \rightarrow \\
 \langle \Sigma; \Delta; \Gamma_s \parallel \mathcal{S}' \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1, \text{FIRE-CXL}(\ell(\overline{v_1}), v_2) \mid T_2 \rangle
 \end{array}$$

From the precondition:

- $(\Delta, X_1) \succ T_1, \text{FIRE}(\ell(\overline{v_1}), v_2), T_2$
- $(\Delta, X_1) \succ T_1 \succ X_2$
- $\Sigma ; \Gamma_s \models_t V_s, \Sigma \models v : \tau, \mathcal{L}$ well-formed

The resulting configuration has $X_3 = X_1$, $X_4 = X_2$, $V_{s2} = V_s$, $\mathcal{L}' = \mathcal{L}$, $\mathcal{C}' = \mathcal{C} \setminus \{ \ell(\overline{v_1}) \}$, $T_3 = T_1, \text{FIRE-CXL}(\ell(\overline{v_1}), v_2)$, and $T_4 = T_2$.

We verify:

- $(\Delta, X_1) \succ T_3, T_4$ – The combined trace is $T_1, \text{FIRE-CXL}(\ell(\overline{v_1}), v_2), T_2$. We use the *EXT/R*-block rule with the degenerate *single*-record extension: choose $R_1 = \text{FIRE-CXL}(\ell(\overline{v_1}), v_2)$, no *FIRE-SUC* $\rightarrow \{ \}$ records present, so the witness obligation is vacuous. We need to peel the *EXT*-prefix that the rule requires. Inversion on the precondition's justification of $T_1, \text{FIRE}(\ell(\overline{v_1}), v_2), T_2$ exposes the *EXT* prefix in T_1 (call it $T_1 = T'_1, \text{EXT} \dots$ followed by some *R*-block then a tail T'' with $\text{FIRE}(\ell(\overline{v_1}), v_2) \in$ the *R*-block, and then T_2); replace the *FIRE* at position of $(\ell(\overline{v_1}), v_2)$ by *FIRE-CXL* $(\ell(\overline{v_1}), v_2)$ inside that *R*-block: this is still a valid R_i choice, and no witness changes. The tail T'' followed by T_2 is justified by the inner premise of the original rule application. Therefore the new combined trace is also justified.

The inversion uniquely identifies the `EXT` prefix because at most one `EXT` record can appear in T_1 at any time within a render cycle: `Receive Ext` requires T_1 to be exactly the single argument record $ARG \dots = \dots \rightarrow \dots$, so once one `Ext` block has been deposited, a second `Receive Ext` cannot fire until the next waiting-to-rendered transition resets T_1 to $ARG \dots = \dots \rightarrow \dots$ alone. Hence the `Ext` prefix introducing this *FIRE* is uniquely the only one in T_1 .

- $(\Delta, X_1) \succ T_3 \succ X_2$ – The variable-set justification rule on *FIRE-CXL* is the identity in the variable set. From the precondition $(\Delta, X_1) \succ T_1 \succ X_2$, extending the prefix by *FIRE-CXL* $(\ell(\overline{v_1}), v_2)$ yields $(\Delta, X_1) \succ T_3 \succ X_2$.
- $\Sigma ; \Gamma_s \models_t V_s$ – unchanged.
- $\Sigma \models v : \tau$ – unchanged.
- $\mathcal{L}$ well-formed – $\mathcal{L}$ is unchanged; $\mathcal{C}$ is a multiset of event labels with no typing content, so its shrinkage carries no obligation.

• **Case Fire Successful.**

$$\begin{array}{c}
 \mathcal{S} = (X_1, X_2, V_s, \mathcal{L}, \mathcal{C}) \\
 \mathcal{S}' = (X_1, X_2', V_s', \mathcal{L}'', \mathcal{C}') \quad \mathcal{C}(\ell(\overline{v_1})) = 0 \quad \mathcal{L}(\ell(\overline{v_1})) = \{(\langle \lambda x_i.e_i, V_i \rangle, m_i, F_i)\}_{i=1}^k \\
 \frac{\overline{\vdash U_i \text{ lsafe}} \quad \Sigma \models v_2 : \Sigma_E(\ell(\overline{v_1})) \quad \Sigma ; V_i, x_i = v_2 \vdash e_i \Downarrow () ; U_i}{\overline{F_i \models_e U_i} \quad \mathcal{L}' = \mathcal{L}[\ell(\overline{v_1}) \mapsto \{(\langle \lambda x_i.e_i, V_i \rangle, m_i, F_i) \mid m_i = \square\}] \quad (X_2, V_s, \mathcal{L}', \mathcal{C}) \Rightarrow U_1, \dots, U_k \Rightarrow (X_2', V_s', \mathcal{L}'', \mathcal{C}')} \\
 \hline
 \langle \Sigma ; \Delta ; \Gamma_s \parallel \mathcal{S} \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1 \mid \text{FIRE}(\ell(\overline{v_1}), v_2), T_2 \rangle \\
 \rightarrow \\
 \langle \Sigma ; \Delta ; \Gamma_s \parallel \mathcal{S}' \mid A \mid \overline{x = v} \mid \text{rendered} \mid T_1, \text{FIRE-SUC}(\ell(\overline{v_1}), v_2) \rightarrow \{ (U_1, F_1), \dots, (U_k, F_k) \} \mid T_2 \rangle
 \end{array}$$

From the precondition:

- $(\Delta, X_1) \succ T_1, \text{FIRE}(\ell(\overline{v_1}), v_2), T_2$
- $(\Delta, X_1) \succ T_1 \succ X_2$
- $\Sigma ; \Gamma_s \models_t V_s, \Sigma \models v : \tau, \mathcal{L}$ well-formed
- External-scheduler payload well-typedness: the firing $(\ell(\overline{v_1}), v_2)$ this step consumes was introduced into the trace by a prior `Receive Ext` step that discharged $\Sigma \models v_2 : \Sigma_E(\ell(\overline{v_1}))$, which is therefore available here and matches the rule's payload-typing premise.

We establish each of the obligations for the resulting configuration in turn.

Step 1: each U_i has an effect-queue witness. Let $\tau = \Sigma_E(\ell(\overline{v_1}))$ be the payload type fixed by the signature at the firing label. This step is delivered directly by the rule premise $\overline{F_i \models_e U_i}$. As noted in the rule, this premise is justified by Theorem F.3 applied to e_i under the listener's argument-extended typing context $\Gamma_{V_i}, x_i : \tau$. Applying that theorem requires two facts:

- $\Sigma \models (\langle \lambda x_i.e_i, V_i \rangle) : \tau \rightarrow \text{unit} \mid F_i$ – the listener's closure has the expected arrow type. This is exactly the $\mathcal{L}$ -well-formedness bullet (from the precondition) instantiated at $\ell(\overline{v_1})$ for each $(c_i, m_i, F_i) \in \mathcal{L}(\ell(\overline{v_1}))$. The listener's arrow type is supplied by Σ_E pinning down the carried type.
- $\Sigma \models v_2 : \tau$ where v_2 is the payload actually delivered to the closure – i.e. the value assigned to x_i in the rule's premise $\Sigma ; V_i, x_i = v_2 \vdash e_i \Downarrow () ; U_i$. This is exactly the rule's payload-typing premise $\Sigma \models v_2 : \Sigma_E(\ell(\overline{v_1}))$, itself inherited from the precondition's external-scheduler well-typedness bullet (the firing was placed in the trace by an earlier `Receive Ext` step which discharged precisely this judgement). Because the payload occupies its own syntactic slot in the trace record, the discriminant tuple $\overline{v_1}$

never enters the listener context, and the typing obligation $\Sigma \models v_2 : \tau$ matches the rule's hypothesis directly.

Step 2: $\mathcal{L}'$ is well-formed. $\mathcal{L}'$ is obtained from $\mathcal{L}$ by replacing the bucket at $\ell(\overline{v_1})$ with the sub-set $\{(\langle \lambda x_i.e_i, V_i \rangle, m_i, F_i) \mid m_i = \square\}$. Every record retained in $\mathcal{L}'(\ell(\overline{v_1}))$ was already in $\mathcal{L}(\ell(\overline{v_1}))$ and so has a typing witness by the $\mathcal{L}$ -well-formedness assumption. All other buckets are unchanged. Hence $\mathcal{L}'$ is well-formed.

Step 3: V'_s well-typed and $\mathcal{L}''$ well-formed. Apply the Flush Queue Preservation lemma (extended above) to the rule premise $(X_2, V_s, \mathcal{L}', \mathcal{C}) \Rightarrow U_1, \dots, U_k \Rightarrow (X'_2, V'_s, \mathcal{L}'', \mathcal{C}')$. The lemma's preconditions are $\Sigma ; \Gamma_s \models_t V_s$ (from the precondition) and that $\mathcal{L}'$ is well-formed (from Step 2). The lemma's FQ LISTEN case requires a closure-typing witness for each newly registered listener; this is supplied by the *Listen typing inversion* lemma applied to the rule's premises $\overline{F_i} \models_e \overline{U_i}$, which establishes $\Sigma \models c : \Sigma_E(\ell(\overline{v})) \rightarrow \text{unit} \mid F$ for every listen item in any U_i . The lemma then yields $\Sigma ; \Gamma_s \models_t V'_s$ and $\mathcal{L}''$ well-formed.

Step 4: trace justification. The new combined trace is $T_1, \text{FIRE-SUC}(\ell(\overline{v_1}), v_2) \rightarrow \{(U_1, F_1), \dots, (U_k, F_k)\}$, where each F_j is the effect stored alongside the listener closure c_j in $\mathcal{L}(\ell(\overline{v_1}))$ at the moment of firing, lifted from the rule premise $\mathcal{L}(\ell(\overline{v_1})) = \{(c_j, m_j, F_j)\}_{j=1}^k$. We use the EXT/R-block rule. Invert the precondition's justification of $T_1, \text{FIRE}(\ell(\overline{v_1}), v_2), T_2$ by identifying the EXT prefix that introduced $\text{FIRE}(\ell(\overline{v_1}), v_2)$ as one of its R_i slots. As in Case Fire Cancel, this Ext prefix is unique: Receive Ext only fires when T_1 is the bare argument record, so T_1 contains at most one Ext block within a render cycle. In the new trace we replace that R_i choice by $R'_i = \text{FIRE-SUC}(\ell(\overline{v_1}), v_2) \rightarrow \{(U_1, F_1), \dots, (U_k, F_k)\}$, which is also a permitted R_i choice. The witness obligations imposed by the rule on $\text{FIRE-SUC} \rightarrow \{ \}$ records are, for each j :

- $F_j \models_e U_j$ — discharged directly by the rule premise $\overline{F_j} \models_e \overline{U_j}$ of FIRE SUCCESSFUL (the symbol F_j on both sides refers to the same listener-stored effect), itself supported by the witnesses produced in Step 1.
- $\exists c. \Sigma \models c : \Sigma_E(\ell(\overline{v_1})) \rightarrow \text{unit} \mid F_j$ — discharged by witnessing with the listener's own closure c_j . $\mathcal{L}$ -well-formedness (from the precondition) instantiated at $\ell(\overline{v_1})$ for the record (c_j, m_j, F_j) yields $\Sigma \models c_j : \Sigma_E(\ell(\overline{v_1})) \rightarrow \text{unit} \mid F_j$, tying F_j to the event's declared payload type in Σ_E .

All other parts of the justification derivation, including the inner tail-justification, are unchanged. Hence $(\Delta, X_1) \succ T_1, \text{FIRE-SUC}(\ell(\overline{v_1}), v_2) \rightarrow \{(U_1, F_1), \dots, (U_k, F_k)\}, T_2$.

Step 5: variable-set justification matches. The variable-set justification rule on FIRE-SUC extends X_2 to $X_2 \cup \{y \mid \exists j. \text{setter}_y(f) \in U_j\}$. We show this equals the X'_2 that the flush-queue derivation in Step 3 produced.

By induction on the flush-queue derivation $(X_2, V_s, \mathcal{L}', \mathcal{C}) \Rightarrow U_1, \dots, U_k \Rightarrow (X'_2, V'_s, \mathcal{L}'', \mathcal{C}')$, X'_2 is exactly X_2 extended by the set of x such that $\text{setter}_x(f)$ appears somewhere in $U_1, \dots, U_k$: only FQ SETTER extends the variable set, and it does so by adding the x of its $\text{setter}_x(f)$ item. Cancel/remove/listen items leave the set unchanged. Therefore $X'_2 = X_2 \cup \{y \mid \exists j. \text{setter}_y(f) \in U_j\}$, which matches the result of the variable-set justification rule for FIRE-SUC applied to the precondition $(\Delta, X_1) \succ T_1 \succ X_2$. Hence $(\Delta, X_1) \succ T_3 \succ X_4$ with $T_3 = T_1, \text{FIRE-SUC}(\ell(\overline{v_1}), v_2) \rightarrow \{(U_1, F_1), \dots, (U_k, F_k)\}$ and $X_4 = X'_2$.

Step 6: argument values. Unchanged.

□

F.5 Declarations

Theorem F.2 (Preservation for Declarations). *Given*

- $\Gamma_s ; X ; V_s ; V ; c \vdash_{\Sigma} p \Downarrow v ; T ; V'$
- $\Sigma ; \Gamma ; \Delta \vdash p : \tau$
- $\Sigma ; \Gamma_s \models_t V_s$
- $\Sigma ; \Gamma \models_t V$
- $X \neq \emptyset$

Then $\Sigma \models v : \tau$ and $(\Delta, X) \succ T$ and $\Sigma ; \Gamma_s \models_t V'$

PROOF. By induction on the size of the semantic derivation

- effect rerender, yes changes

$$\frac{\Gamma_s ; X ; V_s ; V ; c \vdash_{\Sigma} p \Downarrow v ; T ; V' \quad \Sigma ; V \vdash e \Downarrow () ; U \quad \bar{x} \cap X \neq \emptyset}{\frac{\Gamma_s ; X ; V_s ; V ; c \vdash_{\Sigma} \text{on } \bar{x} \text{ do } \{e\} ; p \Downarrow v ; \text{EFF}(\bar{x} \cap X) \rightarrow \{U\}, T ; V' \quad \Gamma \vdash e : \text{unit} \mid F \quad \forall i \in [n], \Delta \vdash @x_i \Rightarrow F}{\Sigma ; \Gamma ; \Delta \vdash \text{on } \bar{x} \text{ do } \{e\} ; \Rightarrow \Gamma} \quad \Sigma ; \Gamma ; \Delta \vdash p : \tau}$$

$$\Sigma ; \Gamma ; \Delta \vdash \text{on } \bar{x} \text{ do } \{e\} ; p : \tau$$

First, apply expressions preservation

- $\Sigma ; V \vdash e \Downarrow () ; U$
- $\Gamma \vdash e : \text{unit} \mid F$
- $\Sigma ; \Gamma \models_t V$

This gives us: (1) $\Sigma \models () : \text{unit}$ and (2) $F \models_e U$

Now apply inductive hypothesis

- $\Gamma_s ; X ; V_s ; V ; c \vdash_{\Sigma} p \Downarrow v ; T ; V'$
- $\Sigma ; \Gamma ; \Delta \vdash p : \tau$
- $\Sigma ; \Gamma_s \models_t V_s$
- $\Sigma ; \Gamma \models_t V$

which gives

- $\Sigma \models v : \tau$
- $(\Delta, X) \succ T$
- $\Sigma ; \Gamma_s \models_t V'$

this means we have:

$$\frac{\bar{x} \subseteq X \quad \exists F. (\Delta \vdash @x_i \Rightarrow F \text{ and } F \models_e U) \quad (\Delta, X) \succ T}{(\Delta, X) \succ (\text{EFF}(\bar{x} \cap X) \rightarrow \{U\}, T)}$$

so we have proved the result for this case:

- $\Sigma \models v : \tau$
- $(\Delta, X) \succ (\text{EFF}(\bar{x} \cap X) \rightarrow \{U\}, T)$
- $\Sigma ; \Gamma_s \models_t V'$

- effect rerender, no changes

$$\frac{\Gamma ; X ; V_s ; V ; c \vdash_{\Sigma} p \Downarrow v ; T ; V' \quad \bar{x} \cap X = \emptyset}{\frac{\Gamma ; X ; V_s ; V ; c \vdash_{\Sigma} \text{on } \bar{x} \text{ do } \{e\} ; p \Downarrow v ; T ; V' \quad \Gamma \vdash e : \text{unit} \mid F \quad \forall i \in [n], \Delta \vdash @x_i \Rightarrow F}{\Sigma ; \Gamma ; \Delta \vdash \text{on } \bar{x} \text{ do } \{e\} ; \Rightarrow \Gamma} \quad \Sigma ; \Gamma ; \Delta \vdash p : \tau}$$

$$\Sigma ; \Gamma ; \Delta \vdash \text{on } \bar{x} \text{ do } \{e\} ; p : \tau$$

Apply inductive hypothesis

- $\Gamma_s ; X ; V_s ; V ; c \vdash_{\Sigma} p \Downarrow v ; T ; V'$

- $\Sigma ; \Gamma ; \Delta \vdash p : \tau$
- $\Sigma ; \Gamma_s \Vdash_t V_s$
- $\Sigma ; \Gamma \Vdash_t V$

which gives

- $\Sigma \Vdash v : \tau$
- $(\Delta, X) \succ T$
- $\Sigma ; \Gamma_s \Vdash_t V'$

so we have proven the result for this case.

• state rerender

$$\frac{\begin{array}{c} \Gamma_s ; X ; V_s ; V, x = v, \text{setX} = \text{setter}_{c.x} ; c \vdash_{\Sigma} p \Downarrow v ; T ; V' \\ x : \tau \in \Gamma_s \quad x = v \in V_s \quad x = \text{setter}_{c.x} \in V_s \quad \Sigma \Vdash v : \tau \end{array}}{\Gamma ; X ; V_s ; V ; c \vdash_{\Sigma} \text{state } x, \text{setX default } e ; p \Downarrow v ; T ; V'} \quad \frac{\Gamma \vdash e : \tau \mid \cdot \quad x[] \mid F \in \Delta \quad \Gamma' = \Gamma, x : \tau, \text{setX} : ((\tau \rightarrow \tau \mid \cdot) \rightarrow \text{unit} \mid \circ^{1r}@x)}{\Sigma ; \Gamma ; \Delta \vdash \text{state } x, \text{setX default } e ; p \Rightarrow \Gamma'} \quad \frac{\Sigma ; \Gamma ; \Delta \vdash \text{state } x, \text{setX default } e ; p \Rightarrow \Gamma'}{\Sigma ; \Gamma ; \Delta \vdash p : \tau}$$

$$\Sigma ; \Gamma ; \Delta \vdash \text{state } x, \text{setX default } e ; p : \tau$$

Apply inversion on $\Sigma ; \Gamma_s \Vdash_t V_s$ to get $\Sigma \Vdash v : \tau$

Apply inductive hypothesis

- $\Gamma_s ; X ; V_s ; V, x = v, \text{setX} = \text{setter}_{c.x} ; c \vdash_{\Sigma} p \Downarrow v ; T ; V'$
- $\Sigma ; \Gamma ; \Delta \vdash p : \tau$
- $\Sigma ; \Gamma_s \Vdash_t V_s$
- $\Sigma ; \Gamma, x : \tau, \text{setX} : ((\tau \rightarrow \tau \mid \cdot) \rightarrow \text{unit} \mid \circ^{1r}@x) \Vdash_t V, x = v, \text{setX} = \text{setter}_{c.x}$

which proves the result for this case:

- $\Sigma \Vdash v : \tau$
- $(\Delta, X) \succ T$
- $\Sigma ; \Gamma_s \Vdash_t V'$

• return

$$\frac{x = v_s \in V_s \quad x = v \in V \quad \Sigma ; \Gamma_s \Vdash_t V}{\Gamma ; X ; V_s ; V ; c \vdash_{\Sigma} \text{return } x \Downarrow v ; \text{RET } x ; V} \quad \frac{\Delta \vdash @x \Rightarrow @\text{return}}{\Sigma ; \Gamma, x : \tau ; \Delta \vdash \text{return } x : \tau}$$

Base case.

- $\Sigma \Vdash v : \tau$
- $\Delta \vdash @x \Rightarrow @\text{return}$
- $(\Delta, X) \succ \text{RET } x$
- $\Sigma ; \Gamma_s \Vdash_t V$

□

F.6 Expressions

Theorem F.3 (Preservation for Expressions). *Given*

- $\Sigma ; V \vdash e \Downarrow v ; U$
- $\Gamma \vdash e : \tau \mid F$
- $\Sigma ; \Gamma \Vdash_t V$

Then $\Sigma \Vdash v : \tau$ and $F \Vdash_e U$

PROOF. By induction on the size of $\Sigma ; V \vdash e \Downarrow v ; U$.

$$\begin{array}{l}
\bullet \frac{}{\Sigma; V, x = v \vdash x \Downarrow v; \cdot} \quad \frac{}{\Gamma, x : \tau \vdash x : \tau \mid \cdot} \quad \frac{\Sigma \models v : \tau \quad \Sigma; \Gamma \models_t V}{\Sigma; \Gamma, x : \tau \models_t V, x = v} \\
\Sigma \models v : \tau \quad \cdot \models_e \cdot \\
\bullet \frac{}{\Sigma; V \vdash \lambda x.e \Downarrow \langle \lambda x.e, V \rangle; \cdot} \quad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2 \mid F}{\Gamma \vdash \lambda x.e : (\tau_1 \rightarrow \tau_2 \mid F) \mid \cdot} \quad \Sigma; \Gamma \models_t V \\
\frac{\Sigma; \Gamma \models_t V \quad \Gamma, x : \tau_1 \vdash e : \tau_2 \mid F}{\Sigma \models \langle \lambda x.e, V \rangle : (\tau_1 \rightarrow \tau_2 \mid F)} \quad \cdot \models_e \cdot \\
\bullet \frac{\Sigma; V \vdash e_1 \Downarrow \langle \lambda x.e_3, V' \rangle; U_1 \quad \Sigma; V \vdash e_2 \Downarrow v_2; U_2 \quad \Sigma; V', x=v_2 \vdash e_3 \Downarrow v_3; U_3}{\Sigma; V \vdash e_1 e_2 \Downarrow v_3; U_1, U_2, U_3} \\
\frac{\Gamma \vdash e_1 : (\tau_1 \rightarrow \tau_2 \mid F) \mid F_1 \quad \Gamma \vdash e_2 : \tau_1 \mid F_2}{\Gamma \vdash e_1 e_2 : \tau_2 \mid F_1 * F_2 * F} \\
\Sigma; \Gamma \models_t V \\
IH_1 \quad \Sigma \models \langle \lambda x.e_3, V' \rangle : (\tau_1 \rightarrow \tau_2 \mid F) \quad F_1 \models_e U_1 \\
IH_2 \quad \Sigma \models v_2 : \tau_1 \quad F_2 \models_e U_2 \\
\text{inversion on } IH_1 \quad \frac{\exists \Gamma'. \Sigma; \Gamma' \models_t V' \quad \Gamma', x : \tau_1 \vdash e_3 : \tau_2 \mid F}{\Sigma \models \langle \lambda x.e_3, V' \rangle : (\tau_1 \rightarrow \tau_2 \mid F)} \\
\text{apply IH to } \Sigma; V', x : v_2 \vdash e_3 \Downarrow v_3; U_3 \quad \Gamma', x : \tau_1 \vdash e_3 : \tau_2 \mid F \quad \Sigma; \Gamma' \models_t V' \\
IH_3 \quad \Sigma \models v_3 : \tau_2 \quad F_3 \models_e U_3 \\
\Sigma \models v_3 : \tau_2 \quad \frac{F_1 \models_e U_1 \quad \frac{F_2 \models_e U_2 \quad F_3 \models_e U_3}{F_2 * F_3 \models_e U_2, U_3}}{F_1 * F_2 * F_3 \models_e U_1, U_2, U_3} \\
\bullet \frac{\Sigma; V \vdash e_1 \Downarrow \text{setter}_x; U_1 \quad \Sigma; V \vdash e_2 \Downarrow \langle \lambda x.e_3, V' \rangle; U_2}{\Sigma; V \vdash e_1 e_2 \Downarrow (); U_1, U_2, \text{setter}_x(\langle \lambda x.e_3, V' \rangle)} \\
\frac{\Gamma \vdash e_1 : ((\tau \rightarrow \tau \mid \cdot) \rightarrow \text{unit} \mid \circ^{1r}_{@x}) \mid F_1 \quad \Gamma \vdash e_2 : (\tau \rightarrow \tau \mid \cdot) \mid F_2}{\Gamma \vdash e_1 e_2 : \tau_2 \mid F_1 * F_2 * \circ^{1r}_{@x}} \\
\Sigma; \Gamma \models_t V \\
IH_1 \quad \Sigma \models \text{setter}_x : (\tau \rightarrow \tau \mid \cdot) \rightarrow \text{unit} \mid \circ^{1r}_{@x} \quad F_1 \models_e U_1 \\
IH_2 \quad \frac{\exists \Gamma'. \Sigma; \Gamma' \models_t V' \quad \Gamma', x : \tau \vdash e_3 : \tau \mid \cdot}{\Sigma \models \langle \lambda x.e_3, V' \rangle : \tau \rightarrow \tau \mid \cdot} \quad F_2 \models_e U_2 \\
\frac{F_1 \models_e U_1 \quad \frac{F_2 \models_e U_2 \quad \circ^{1r}_{@x} \models_e \text{setter}_x(\langle \lambda x.e_3, V' \rangle)}{F_2 * \circ^{1r}_{@x} \models_e U_2, \text{setter}_x(\langle \lambda x.e_3, V' \rangle)}}{\Sigma \models () : \text{unit} \quad F_1 * F_2 * \circ^{1r}_{@x} \models_e U_1, U_2, \text{setter}_x(\langle \lambda x.e_3, V' \rangle)}
\end{array}$$

$$\begin{array}{c}
\bullet \frac{\Sigma ; V \vdash e_1 \Downarrow v_1 ; U_1 \quad \Sigma ; V \vdash e_2 \Downarrow v_2 ; U_2}{\frac{\Sigma ; V \vdash e_1 ; e_2 \Downarrow v_2 ; U_1, U_2}{\frac{\Gamma \vdash e_1 : \tau_1 \mid F_1 \quad \Gamma \vdash e_2 : \tau_2 \mid F_2}{\Gamma \vdash e_1; e_2 : \tau_2 \mid F_1 * F_2}}}}{\Sigma ; \Gamma \models_t V} \\
\\
IH_1 \quad \Sigma \models v_1 : \tau_1 \quad F_1 \models_e U_1 \\
IH_2 \quad \Sigma \models v_2 : \tau_2 \quad F_2 \models_e U_2 \\
\\
\Sigma \models v_2 : \tau_2 \quad \frac{F_1 \models_e U_1 \quad F_2 \models_e U_2}{F_1 * F_2 \models_e U_1, U_2} \\
\\
(\text{BRANCH1}) \\
\bullet \frac{\Sigma ; V \vdash e_1 \Downarrow \text{true} ; U_1 \quad \Sigma ; V \vdash e_2 \Downarrow v_2 ; U_2}{\Sigma ; V \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow v_2 ; U_1, U_2} \text{WLOG choose branch 1} \\
\frac{\Gamma \vdash e_1 : \text{bool} \mid F_1 \quad \Gamma \vdash e_2 : \tau \mid F_2 \quad \Gamma \vdash e_3 : \tau \mid F_3}{\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : \tau \mid F_1 * (F_2 + F_3)} \\
\\
IH_1 \quad \Sigma \models \text{true} : \text{bool} \quad F_1 \models_e U_1 \\
IH_2 \quad \Sigma \models v_2 : \tau \quad F_2 \models_e U_2 \\
\\
\Sigma \models v_2 : \tau \quad \frac{F_1 \models_e U_1 \quad \frac{F_2 \models_e U_2}{F_2 + F_3 \models_e U_2}}{F_1 * (F_2 + F_3) \models_e U_1, U_2} \\
\\
(\text{PROD}) \\
\bullet \frac{\Sigma ; V \vdash e_1 \Downarrow v_1 ; U_1 \quad \Sigma ; V \vdash e_2 \Downarrow v_2 ; U_2}{\Sigma ; V \vdash (e_1, e_2) \Downarrow (v_1, v_2) ; U_1, U_2} \quad \frac{\Gamma \vdash e_1 : \tau_1 \mid F_1 \quad \Gamma \vdash e_2 : \tau_2 \mid F_2}{\Gamma \vdash (e_1, e_2) : \tau_1 \times \tau_2 \mid F_1 * F_2} \\
\Sigma ; \Gamma \models_t V \\
\\
IH_1 \quad \Sigma \models v_1 : \tau_1 \quad F_1 \models_e U_1 \\
IH_2 \quad \Sigma \models v_2 : \tau_2 \quad F_2 \models_e U_2 \\
\\
\Sigma \models v_1 : \tau_1 \quad \Sigma \models v_2 : \tau_2 \quad \frac{F_1 \models_e U_1 \quad F_2 \models_e U_2}{F_1 * F_2 \models_e U_1, U_2} \\
\\
(\text{FST}) \\
\bullet \frac{\Sigma ; V \vdash e \Downarrow (v_1, v_2) ; U}{\Sigma ; V \vdash \text{fst } e \Downarrow v_1 ; U} \quad \frac{\Gamma \vdash e_1 : \tau_1 \times \tau_2 \mid F}{\Gamma \vdash \text{fst } e_1 : \tau_1 \mid F} \quad \Sigma ; \Gamma \models_t V \\
\text{WLOG fst case} \\
IH \quad \frac{\Sigma \models v_1 : \tau_1 \quad \Sigma \models v_2 : \tau_2}{\Sigma \models (v_1, v_2) : \tau_1 \times \tau_2} \quad F \models_e U \\
\Sigma \models v_1 : \tau_1 \quad F \models_e U
\end{array}$$

$$\begin{array}{c}
\text{(BIND)} \\
\frac{\Sigma ; V \vdash e \Downarrow \langle \lambda x.e', V' \rangle ; U \quad \Sigma \models \langle \lambda x.e', V' \rangle : \tau \rightarrow \text{unit} \mid F}{\Sigma ; V \vdash \text{bind } \ell(\overline{v_1}) e \Downarrow () ; U, \text{listen}(\ell(\overline{v_1}), \langle \lambda x.e', V' \rangle, \square, F)} \\
\text{(TY BIND)} \\
\frac{\Gamma \vdash e : (\tau \rightarrow \text{unit} \mid F) \mid F_e \quad \tau = \Sigma_E(\ell(\overline{v_1}))}{\Gamma \vdash \text{bind } \ell(\overline{v_1}) e : \text{unit} \mid F_e * \square^{\ell(\overline{v_1})}(F)} \\
\Sigma ; \Gamma \models_t V
\end{array}$$

apply IH to $\Sigma ; V \vdash e \Downarrow \langle \lambda x.e', V' \rangle ; U$ $\Gamma \vdash e : (\tau \rightarrow \text{unit} \mid F) \mid F_e$ $\Sigma ; \Gamma \models_t V$
 IH $\Sigma \models \langle \lambda x.e', V' \rangle : \tau \rightarrow \text{unit} \mid F$ $F_e \models_e U$

$$\begin{array}{c}
F_e \models_e U \quad \frac{\Sigma \models \langle \lambda x.e', V' \rangle : \tau \rightarrow \text{unit} \mid F \quad \tau = \Sigma_E(\ell(\overline{v_1}))}{\square^{\ell(\overline{v_1})}(F) \models_e \text{listen}(\ell(\overline{v_1}), \langle \lambda x.e', V' \rangle, \square, F)} \\
\Sigma \models () : \text{unit} \quad \frac{}{F_e * \square^{\ell(\overline{v_1})}(F) \models_e U, \text{listen}(\ell(\overline{v_1}), \langle \lambda x.e', V' \rangle, \square, F)}
\end{array}$$

Note that the closure typing $\Sigma \models \langle \lambda x.e', V' \rangle : \tau \rightarrow \text{unit} \mid F$ used to build the TQ LISTEN ALWAYS premise is exactly the value-typing conclusion delivered by the IH, so the type-level witness on the new listen item matches the latent effect F recorded by the instrumented BIND rule.

$$\begin{array}{c}
\text{(ONCE)} \\
\frac{\Sigma ; V \vdash e \Downarrow \langle \lambda x.e', V' \rangle ; U \quad \Sigma \models \langle \lambda x.e', V' \rangle : \tau \rightarrow \text{unit} \mid F}{\Sigma ; V \vdash \text{once } \ell(\overline{v_1}) e \Downarrow () ; U, \text{listen}(\ell(\overline{v_1}), \langle \lambda x.e', V' \rangle, \diamond, F)} \\
\text{(TY ONCE)} \\
\frac{\Gamma \vdash e : (\tau \rightarrow \text{unit} \mid F) \mid F_e \quad \tau = \Sigma_E(\ell(\overline{v_1}))}{\Gamma \vdash \text{once } \ell(\overline{v_1}) e : \text{unit} \mid F_e * \diamond^{\ell(\overline{v_1})}(F)} \\
\Sigma ; \Gamma \models_t V
\end{array}$$

Identical to the bind case modulo the mode tag $\diamond$ in place of $\square$: apply IH to the closure subderivation to obtain $\Sigma \models \langle \lambda x.e', V' \rangle : \tau \rightarrow \text{unit} \mid F$ and $F_e \models_e U$, then close with TQ LISTEN ONCE in place of TQ LISTEN ALWAYS.

$$\begin{array}{c}
F_e \models_e U \quad \frac{\Sigma \models \langle \lambda x.e', V' \rangle : \tau \rightarrow \text{unit} \mid F \quad \tau = \Sigma_E(\ell(\overline{v_1}))}{\diamond^{\ell(\overline{v_1})}(F) \models_e \text{listen}(\ell(\overline{v_1}), \langle \lambda x.e', V' \rangle, \diamond, F)} \\
\Sigma \models () : \text{unit} \quad \frac{}{F_e * \diamond^{\ell(\overline{v_1})}(F) \models_e U, \text{listen}(\ell(\overline{v_1}), \langle \lambda x.e', V' \rangle, \diamond, F)}
\end{array}$$

$$\begin{array}{c}
\text{(CANCEL)} \\
\frac{}{\Sigma ; V \vdash \text{cancel } \ell(\overline{v}) \Downarrow () ; \text{cancel}(\ell(\overline{v}))} \\
\text{(TY CANCEL)} \\
\frac{}{\Gamma \vdash \text{cancel } \ell(\overline{v}) : \text{unit} \mid \emptyset \ell(\overline{v})}
\end{array}$$

$$\Sigma ; \Gamma \models_t V$$

No subderivations; no IH to apply. The semantic rule produces $()$ with the singleton queue $\text{cancel}(\ell(\bar{v}))$, and the typing rule assigns it the effect $\oslash \ell(\bar{v})$. The matching queue typing rule TQ CANCEL relates the two directly.

$$\Sigma \models () : \text{unit} \quad \frac{}{\oslash \ell(\bar{v}) \models_e \text{cancel}(\ell(\bar{v}))}$$

(REMOVE)

$$\bullet \quad \frac{}{\Sigma ; V \vdash \text{remove } \ell(\bar{v}) \Downarrow () ; \text{remove}(\ell(\bar{v}))} \quad (\text{TY REMOVE})$$

$$\frac{}{\Gamma \vdash \text{remove } \ell(\bar{v}) : \text{unit} \mid \blacktimes \ell(\bar{v})} \quad \Sigma ; \Gamma \models_t V$$

Symmetric to the cancel case: the semantic rule produces $()$ with the singleton queue $\text{remove}(\ell(\bar{v}))$, the typing rule assigns effect $\blacktimes \ell(\bar{v})$, and TQ REMOVE relates them directly.

$$\Sigma \models () : \text{unit} \quad \frac{}{\blacktimes \ell(\bar{v}) \models_e \text{remove}(\ell(\bar{v}))}$$

$$\begin{aligned} & \bullet \text{ trivial cases:} \\ & \bullet \quad \frac{c \in \alpha \cup \{\text{true}, \text{false}\} \cup \{()\}}{\Sigma ; V \vdash c \Downarrow c ; \cdot} \quad \frac{c \in \{\text{true}, \text{false}\}}{\Gamma \vdash c : \text{bool} \mid \cdot} \quad \Sigma ; \Gamma \models_t V \\ & \bullet \quad \frac{c \in \alpha \cup \{\text{true}, \text{false}\} \cup \{()\}}{\Sigma ; V \vdash c \Downarrow c ; \cdot} \quad \frac{c \in \alpha}{\Gamma \vdash c : \alpha \mid \cdot} \quad \Sigma ; \Gamma \models_t V \\ & \bullet \quad \frac{() \in \alpha \cup \{\text{true}, \text{false}\} \cup \{()\}}{\Sigma ; V \vdash () \Downarrow () ; \cdot} \quad \frac{}{\Gamma \vdash () : \text{unit} \mid \cdot} \quad \Sigma ; \Gamma \models_t V \end{aligned}$$

□